Computer Networks

جلسه اول - ۱۴۰۰ ،۶/۲۸

بارم بندی: تمرین ها و پروژه ← ۸ پایانترم ← ۱۲

شبکه های پیچیده و سیستم های پیچیده

یک سیستم پیچیده سیستمی است که از تعدادی اشیاء و ارتباطات میان آن ها تشکیل شده است

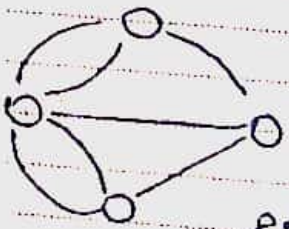
مثلا: هر خودرو اتومبیل، حوادث بشری ← هر فرد یک object است و با هم در ارتباط اند

سیستم های ارتباطاتی ← هر تلفن یک object، ارتباطاتی هم بینشون هست

داده بشری ← مثلا دیلی بدیا کشش قابل توجهی از داده را در خودش دارد و آن را به صورت organized ذخیره کرده، هر بیج کشی از این داده را در خودش دارد

ارتباطات بین آن ها، ارتباطات میان نورون های مغز انسان

سیستم های پیچیده توسط یک ابزار قوی بنام Network مدل سازی می شوند. object ها نودهای Network هستند و ارتباطات میان آن ها یال ها را نشان می دهد.



* مسئله ی ۷ پل کونیگسبرگ

آیا می توان از یکی از شهرها شروع کرد، از هر پل فقط یکبار عبور کرد و دوباره به همان شهر اول رسید؟ اویلر ثابت کرد برای وجود چنین دوری باید درجه هر راس زوج باشد (دور اویلری)

جلسه دوم - ۱۴۰۰ ،۶/۳۰

مثال دیگری از Network ← Map of science ← ژورنال های مختلف در حوزه های مختلف را در نظر می گیرند. هر نود یک paper است و بین دو تا نود یال داریم اگر یکی دیگری را cite کرده باشد.

Scanned with CamScanner

ما گراف و شبکه را یکسان تلقی می کنیم و سیستم های پیچیده را با آن ها مدل می کنیم.

چرا گراف های پیچیده الان مهم شده اند؟

① چون به تازگی توانسته ایم دیتاها را جمع آوری کنیم

② دیتاهای جمع شده توسط شبکه های گرافی بازنمایی شده اند

churn prediction (طغیان) ← با توجه به ویژگی هایی که کاربران دارند حد آستانه ای بالا برون قیمت خدمات چقدر است به طوری که کاربران باز هم از خدمات این شرکت استفاده کنند.

local community detection ← همسایگی یک کاربر خاص را detect می کنیم

Community detection ← به طور کلی اجتماع های dense را در گراف بیابیم

یک تسک مهم در شبکه های اجتماعی ← نودهای کلیدی را هم شناسایی کنیم

اهداف کلی در مطالعه شبکه های پیچیده:

① به دنبال ویژگی های آماری مختلف مشترک بین شبکه های مختلف را پیدا کنیم. مثلا یک ویژگی آماری مشترک ← توزیع درجه می توانی

② بعضی از ویژگی های ساختاری بین شبکه های پیچیده مشترک است. به دنبال pattern ها یا موتیف ها در شبکه های پیچیده مشترک اند در حالی که در شبکه های دست ساز و تصادفی دیده نمی شوند یا خیلی کم دیده می شوند.

③ اینکه بفهمیم چرا این ویژگی های مشترک وجود دارند؟ چرا شبکه های پیچیده از این مبانی و اصول پیروی می کنند؟ که جواب آن به ما کمک می کند در مورد این شبکه ها پیش بینی ای هم بدهیم.

Scanned with CamScanner

Objects : nodes, vertices — N
Interactions: links, edges — E
system: network, graph — G(N,E)

در شبکه بدون جهت:

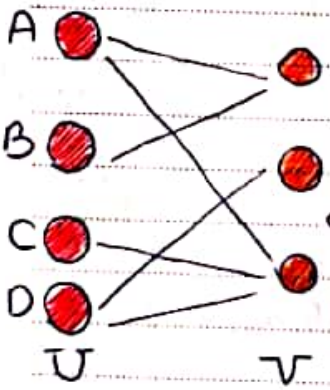
$$\text{Avg degree} = \bar{K} = \langle K \rangle = \frac{1}{N}\sum_{i=1}^{N} K_i = \frac{2E}{N}$$

در شبکه جهت‌دار:

$$\text{Avg degree } (\bar{k}) = \frac{E}{N} \qquad \overline{k^{in}} = \overline{k^{out}} = \frac{E}{N}$$

حداکثر تعداد یال‌ها در گراف غیرجهت‌دار: $\binom{n}{2} = \frac{n(n-1)}{2}$ (غیرجهت‌دار)، گراف این که این تعداد یال داشته باشد گراف کامل نامیده می‌شود. Avg. Degree در گراف کامل (جهت‌دار و غیرجهت‌دار) برابر $\frac{n-1}{2}$ است.

گراف دوبخشی Bipartite Graph



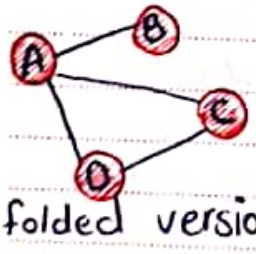
گرافی به ۲ بخش مانند U و V تقسیم می‌شود به طوری که نودهای بخش U فقط به نودهای بخش V و نه U یال دارند و برعکس

مثال: گراف author, paper

در user, movie اگر کاربر به فیلم امتیاز داده باشد یال وزن‌دار بینشون هست

خیلی اوقات می‌توان از گراف دوبخشی یک گراف folded بسازید

بین دو نود یال داریم اگر همسایه مشترکی در بخش دیگر گراف داشته باشند و متی گراف folded می‌تواند وزن‌دار باشد و وزن‌ها نشان‌دهنده تعداد همسایه‌های مشترک باشد



folded version

نمایش گراف‌ها

① ماتریس مجاورت (Adjacency Matrix)

ماتریس مجاورت گراف‌های بدون جهت متقارن است

معمولا ماتریس مجاورت روش خوبی برای ذخیره گراف‌های دنیای واقعی نیست چون $O(N^2)$ حافظه نیاز دارد

تقریبا امکان از عملیاتی که نیاز دارند ماتریس یک گراف به صورت صریح ساخته شود استفاده نکنید. ماتریس‌های مجاورت خیلی پرتک و sparse هستند، تعداد خیلی زیادی ۰ دارند در نتیجه بهینه نیستند. گراف‌های دنیای واقعی تعداد یال‌هاشون نسبت به max تعداد یال‌ها خیلی کم است.

Scanned with CamScanner

۱۴۰۰

space complexity: O(E)

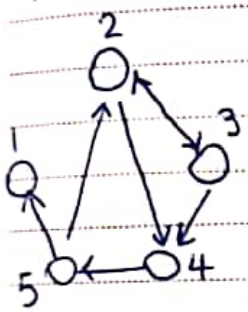
② لیست یال ها (Edge List)

مزیت این روش: حافظه کمی مصرف می‌کند O(E) و تعداد یال ها از n^2 معمولا خیلی کمتره

عیب این روش: در خیلی از عملیات هایی که روی گراف ها انجام می دهیم local عمل می‌کنیم و از یک نود وارد مجموعه همسایه هاش می‌شویم. خیلی خوبه که همسایه ها هر نود رو به صورت یکجا داشته باشیم اما این روش چنین امکانی را فراهم نمی‌کند.
در نتیجه لیست مجاورت پیشنهاد شده

③ لیست مجاورت (Adjacency List)

برای هر نود لیست همسایه هایش را نگه داریم. برای گراف های جهتدار لیست همسایه های خروجی یا ورودی را نگه داریم (یا هر دو در کنار هم)



1:
2: 3,4
3: 2,4
4: 5
5: 1,2

✓ space Complexity ← O(E) ← بهینه

✓ هم چنین خیلی بهینه به همسایه ها هر نود دسترسی داریم.

Edge Attributes ←

در بعضی گراف ها، یال ها یک سری ویژگی یا attribute دارند مثلا: وزن (مثلا در شبکه های اجتماعی فرکانس یک communication)، Ranking (دوست نزدیک، دومین دوست و ...)

Type (دوست، فامیل، ...)

sign (در گراف های علامتدار هر یال یا مثبته یا منفی - دشمن / دوست - اعتماد / عدم اعتماد)

گراف
- غیر وزن دار ← ماتریس مجاورت ۰ و ۱ است (zero_one)
- وزندار ← در همه نود همان درجه وزن دار است (جمع وزن یال های خروجی)

تعداد یال های گراف = حاصل جمع وزن های یال های مختلف، سعی کنید تا جای ممکن وزن منفی نداشته باشید در گراف هایی که برای کاربردهای مختلف می سازید.

روز آتش نشانی و ایمنی - روز بزرگداشت شمس

Scanned with CamScanner

جلسه سوم - ۱۴۰۰/۷/۴

سوال: چرا وجود وزن منفی تأثیر بدی دارد در گراف؟

order حل مسائل بالایی رود. مثلا برای longest shortest path از $O(nm)$ به مسئله‌ی np_hard تبدیل می‌شود. از polynomial به np_hard تبدیل می‌شود.

مسئله longest shortest path درحالتی که یال‌ها وزن ندارند ← $O(nm)$

درحالتی که یال‌ها وزن مثبت دارند ← $O(nm + n^2 \log n)$

وقتی اجازه بدهیم وزن‌های منفی در گراف به وجود بیاید ← np_hard ← راه حل چندجمله‌ای ندارد

پیچیدگی زمانی نمایی دارد برحسب n و m

همبندبودن گراف (Connectivity of Graphs)

یک گراف غیرجهت‌دار همبند یا متصل است اگر و تنها اگر بین هر دو نود حداقل یک مسیر وجود داشته باشد

Giant component ← وقتی گراف غیرهمبند است دارای تعدادی Component است

این کامپونتی که بیشترین تعداد نود را دارد Giant Component است

Largest component ← فرق آن با Giant Component این است که

Giant Component وقتی استفاده می‌شه که فقط یه دونه بزرگترین Component داشته باشید و هیچ کدام از کامپونت‌ها مهم نیست که یونیک باشد.

Largest Component هم نیست که یونیک باشد ولی

دیگه از نظر اندازه با آن قابل مقایسه نباشند

bridge Edge ← یالی است که اگر حذف شود تعداد Component ها زیاد شود یا به طور خاص اگر گراف همبند باشد غیرهمبند شود

Articulation point (نقطه مفصلی):

رأسی است که اگر حذف شود تعداد کامپونت‌های گراف افزایش پیدا می‌کند

Scanned with CamScanner

۱۴۰۰

در گراف های جهتدار دو تا مفهوم از اتصال (همبندی یا Connectivity) تعریف می شود:

① strongly connected directed graph:

یک گراف جهتدار به شدت متصل یا همبند است اگر و تنها اگر به ازای هر دو تا نود A و B، هم یک مسیر از A به B وجود داشته باشد و هم یک مسیر از B به A.

② weakly connected directed Graph:

یک گراف جهتدار متصل ضعیف است اگر و تنها اگر وقتی جهت یال ها را حذف کنیم (نادیده بگیریم) گراف غیر جهتدار حاصل، متصل باشد.

هر گرافی strongly connected باشد weakly connected است اما برعکسش نه.

تحلیل وب — با گراف جهتدار مدل می کنیم

یک شبکه پیچیده را انتخاب می کنیم، گراف متناظر با آن را می سازیم، با استفاده از الگوریتم های گراف آن را تحلیل می کنیم و یک سری معیارها را اندازه می گیریم، بعد یک سری آزمایش های تجربی روی آن انجام می دهیم و به سری نتایج به دست می آوریم و در انتها به یک مدل می رسیم.

نودها ← page ها یال ها ← hyperlink ها (جهتدار هستند)

تقریبا $\frac{2}{3}$ گراف های دنیای واقعی با گراف جهتدار بهتر مدل می شوند

$In(v)$ ← نودهایی که از آن ها به نود v یک مسیر وجود داشته باشد

$In(v) = \{w \mid w \text{ can reach } v\}$

$out(v)$ ← به نود در $out(v)$ هست اگر و تنها اگر از v به اون نود یک مسیر وجود داشته باشد

$out(v) = \{w \mid v \text{ can reach } w\}$

Scanned with CamScanner

* طریقه به دست آوردن out(v):

از نود v شروع می کنیم DFS یا BFS می زنیم، هرگره ای ملاقات بشه در out(v) قرار دارد

پس time Complexity آن O(E) (E تعداد یال ها) می باشد ← با فرض اینکه گراف وزندار نیست. اگر باشد وزن ها را نادیده می گیریم

* طریقه به دست آوردن in(v):

راه حل اولیه: از هرکدام از گره ها DFS بزنیم، اگر گره v دیده شد، اولین که DFS ازش شروع شده در in(v) قرار می گیرد. مرتبه زمانی آن O(nm) است (n تعداد گره ها، m تعداد یال ها)

عیب این روش: پیچیدگی زمانی بالا

یک روش دیگر: در لیست مجاورت همسایه های v و مجددا همسایه های همسایه های v و ... را بررسی کنیم

آن هایی که به v لینک دارند آن هایی که به همسایه ها v لینک دارند

راه حل بهتر: جهت یال ها را عوض کنیم و مجددا با شروع از v، DFS را اجرا کنیم. (یا BFS را)

. مرتبه زمانی ← O(E) (E تعداد یال ها)

در بین گراف های جهتدار 2 نوع گراف هم داریم:

① Strongly Connected ← بین هر دو نود یک مسیر از اولی به دومی و یک مسیر از دومی به اولی وجود داشته باشد

برای گراف هایی که کل گراف Strongly Comected هست یک ویژگی داریم: برای هر نود مثل A چون

هر نود دلخواه به A مسیر دارد و A هم به هر نود دلخواه مسیر دارد: $In(A) = Out(A) = \{$کل نودها$\}$

② (DAG) Directed Acyclic Graph

در این گراف ها اگر u در In(v) باشد دیگر v در In(u) نیست. چون اگر از u به v مسیر داشته باشیم

دیگر از v به u مسیر نداریم (برای اینکه دور نداشته باشد)

نکته مهم: هر گراف جهتدار دلخواه را می توانیم برحسب دو نوع گراف بالا بیان کنیم

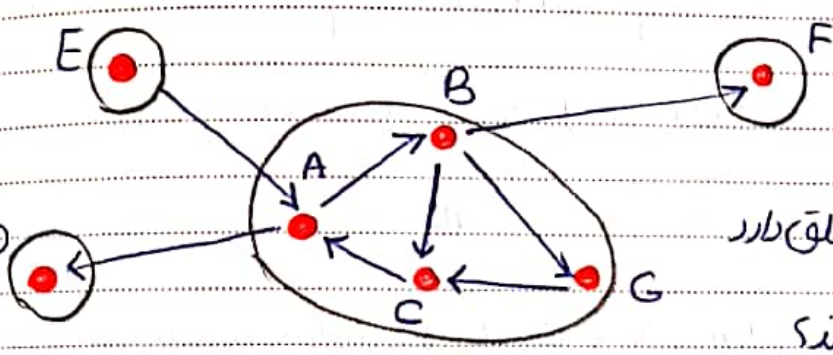
Scanned with CamScanner

Strongly Connected Component

یک SCC زیرمجموعه‌ای از نودهای گراف است که ۲ ویژگی دارد:

۱) گرافی که توسط این زیرمجموعه تولید می‌شه (به وجود آمده) strongly Connected است

۲) این زیرمجموعه maximal باشد یعنی هیچ ابرمجموعه‌ای نداشته باشد که آن هم strongly Connected باشد.

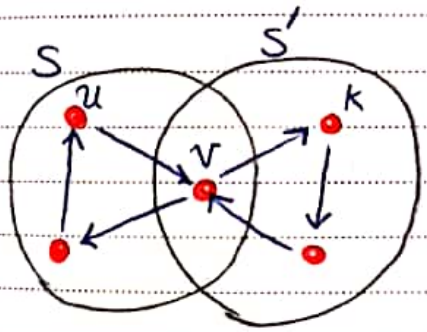


افراز کردن یا partition کردن گراف

یعنی هر نود از گراف دقیقاً به یک SCC تعلق دارد

آیا همیشه SCC ها یک افراز به ما می‌دهند؟

آیا ممکن است یک نود به دو تا SCC تعلق داشته باشد؟



برهان خلف: فرض کنیم نودی داریم که به دو تا SCC تعلق دارد (نود v)

یک نود دلخواه از S و یک نود دلخواه از S' را در نظر بگیرید مثل u و k

از u به v حتما مسیر وجود دارد چون داخل یک SCC هستند از v به k هم حتما یک مسیر وجود دارد چون در یک SCC هستند در نتیجه از u به k یک مسیر وجود دارد و همینطور از k به u با استدلال مشابه. پس u و k باید داخل یک SCC باشند. در واقع به ازای هر نود دلخواه از S و S' ثابت کردیم که این‌ها باید داخل یک SCC باشند. پس ترکیب S و S' باید یک SCC بسازد و S و S' نمی‌توانند SCC باشند و شرط maximal بودنشان نقض می‌شود.

← بنابراین SCC یک partitioning روی نودها اعمال می‌کند

نکته: هر گراف جهتدار را می‌توان به صورت یک DAG برحسب SCC هایش نشان داد.

① از روی گراف دلخواه، گراف SCC می‌سازیم. یعنی SCC های گراف را در گام اول پیدا می‌کنیم.

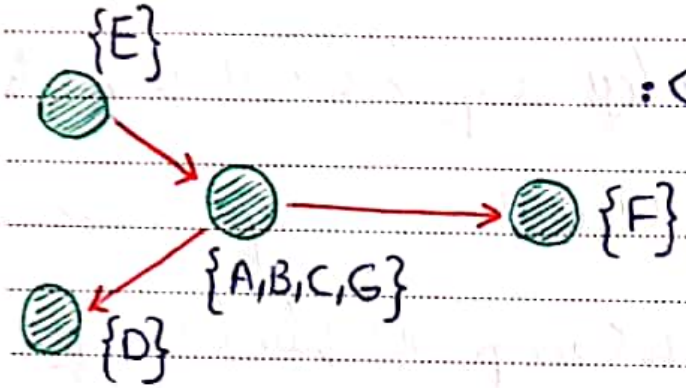
② در گام دوم هر SCC می‌شود یک نود در گراف SCC

روز وقف

Scanned with CamScanner

(۴) در گام سوم دو تا ابر یال را هم وصل می کنید اگر و تنها اگر یک یال در ابر یال اول و یک یال در ابر یال دوم وجود داشته باشد که در گراف اصلی به هم یال داشته باشند.

گراف SCC برای گرافی که در صفحه قبل بود به این صورت است:



ادعا: گراف SCC یک DAG است

گراف SCC جهت دار است چون گراف اول ما جهت دار بود

و این گراف جهت دار سایکل یا دور هم ندارد. چرا دور ندارد؟

برهان خلف: فرض کنیم سایکل وجود داشته باشیم مثلا از {F} به {E} یال وجود داشته باشد

از یک نود دلخواه در {E} به هر نود دلخواه در {F} مسیر وجود دارد، از طریق ابر نود {A,B,C,G}

هم چنین از هر نود دلخواه در {F} به هر نود دلخواه در {E} مسیر داریم به دلیل وجود یال. بنابراین {E} و

{F} باید داخل یک SCC باشند. در نتیجه {E} و {F} شرط maximality را نقض می کنند و نمی توانند

SCC باشند و به تناقض می رسیم.

Scanned with CamScanner

۱۴۰۰/ وب همواره در حال تغییر است. میایم یه snapshot از وب در یک زمان مشخص می‌گیریم

← فرآیند یافتن SCC های گراف وب چگونه است؟

یک نود دلخواه را انتخاب می‌کنیم و SCC ای که این نود را در بر دارد پیدا می‌کنیم و بعد این SCC را با خیال راحت حذف می‌کنیم چون SCC ها دارای افراز و پارتیشن بندی هستند می‌توانیم یک SCC را راحت حذف کنیم بدون اینکه نگران باشیم به نودی عکسه از SCC دیگری به این SCC تعلق داشته باشه.

بنابراین فرآیند یافتن SCC های یک گراف بصورت به این صورت است: یک نود دلخواه را برمیداریم، SCC این نود دلخواه را پیدا می‌کنیم و همه‌ی نودهای آن SCC را حذف می‌کنیم. مجددا از نودهای باقی مانده یک نود اتفاقی برمیداریم، SCC آن نود را حساب می‌کنیم و حذف می‌کنیم (SCC دوم) و این روند را ادامه می‌دهیم تا گراف تمام شود

← برای به دست آوردن SCC یک نود چگونه عمل کنیم؟

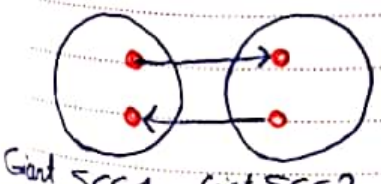
هر نودی که در SCC نود v وجود دارد به نود v مسیر دارد پس در In(v) وجود دارد. هم‌چنین از v به همه‌ی نودهایی که در SCC مربوط به v هستند، مسیر داریم پس آن نود در out(v) است. پس هر نود دلخواه در SCC مربوط به v هم در In(v) و هم در out(v) وجود دارد. پس SCC نود v می‌شود: $In(v) \cap out(v)$

قبلا هم گفتیم که In(v) را با برعکس کردن جهت یال‌ها می‌توانیم به دست آوریم:

$In(v) = out(v, \bar{G}) \Rightarrow SCC(v) = out(v,G) \cap out(v,\bar{G})$

در $O(m)$ می‌توان SCC نود را حساب کرد.

به طور کلی در شبکه‌های پیچیده یک giant SCC داریم (دوتا SCC با اندازه خیلی بزرگ و نزدیک به هم نداریم). علتش چیه؟



فرض کنید دوتا giant SCC به شکل روبه رو داریم. الله حداقل یه نود از SCC1 به یک نود از SCC2 و همینطور از یک نود در SCC2 به یک نود در SCC1 یال داشته باشیم، این دو SCC با هم ترکیب می‌شوند و یک SCC بزرگتر تشکیل می‌دهند و با توجه به اینکه تعداد نودها در هر کدام از SCC ها خیلی زیاد است احتمال وجود چنین جفت یال‌هایی بسیار بالا است

روز دامپزشکی

Scanned with CamScanner

largest weakly component

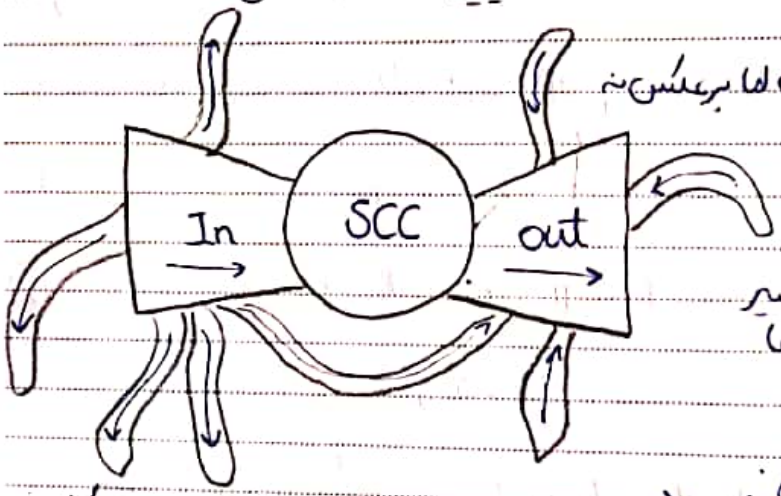
بهتر یال های دو راه حذف کردند. یک
در وب وجود داشت که ۹۱ درصد نودها در آن وجود دارند.

آیا نودهای hub هستند که گره ها را بهم وصل کرده اند؟

hub ← گره هایی که درجه شان از ما بزرگتر است. اهمیتشان بالا است. به این نتیجه رسیدند
که اهمیت بالایی ندارند چون با حذف hub ها هنوز حدود ۹۰ درصد گره ها در largest Component وجود داشتند.

ورژن مختصر گراف وب: ۲۸ درصد نودها به largest SCC تعلق دارند

هر نود به طور تصادفی حدود ۱۰۰ میلیون نود در out و ۱۰۰ میلیون نود در in اش هست



out: از نودهای SCC به آن ها مسیر هست اما برعکس نه

مدل هایی که پیشنهاد دادند:

In ← از آن ها به نودهای داخل SCC مسیر وجود دارد اما از نودهای SCC به این نودها مسیر وجود ندارد

✓ ساختار بندی موضوعی وب را مشخص می کند
اصل پایه ای دارد)

[این مدل چه چیزهایی را برای ما روشن می کند؟

مواردی که مشخص نمی کند:

۱) اینکه آیا همه نودها اهمیت یکسانی دارند؟ چگونه نودهای مهم را پیدا کنیم؟

۲) توزیع درجه به چه صورت است؟ آیا درجه نودها تردیک بهم هست؟

۳) داخل SCC چه اتفاقی می افتد؟ ساختار وب در داخل SCC چگونه است؟ ساختارهای مختلف درون SCC هست مثلا یک ساختار community structure است،

۴) داخل giant Component نودها بهم مسیر دارند. اما طول این مسیرها چقدر است؟
نودها بهم نزدیکند یا دورند؟ دنیای ما دنیای کوچکیه یا بزرگه داخل SCC؟

برای پاسخ به این سوال ها یه سری معیارها به وجود آمدند:

Scanned with CamScanner

① $p(k)$ توزیع درجه

② h path Length یا Avg path Length

فاصله نودها به طور متوسط از هر یک چقدر است؟ یا فاصله نودها در بدترین حالت از هم چقدر است؟

③ ضریب خوشه بندی (C) Clustering Coefficient

density (چگالی) محلی نودها در یک گراف چگونه است؟

گراف های دنیای واقعی به طور کلی sparse هستند اما نمی توان نتیجه گرفت که به صورت local (محلی) هم sparse باشند.

توزیع درجه (Degree Distribution)

به دو صورت تعریف می شود: تئوری و عملی

تعریف تئوری ← احتمال اینکه درجه یک نود تصادفی برابر با K باشد. $P(k)$

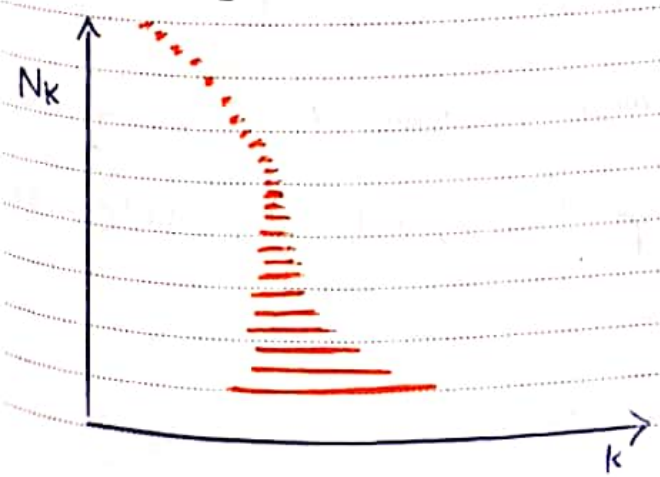
در مدل های ریاضی کاربرد دارد

تعریف عملی ← یک گراف موجود داریم. تعداد نودهایی که درجه k دارند

$$P(k) = \frac{N_k}{N}$$

تقسیم بر تعداد کل نودها می شود $p(k)$

گراف های واقعی توزیع درجه شان شکل خاصی دارد. نمودار log-log آن به این شکل است:



Scanned with CamScanner

paths In Graph

path (مسیر) : یک توالی از نودها به طوری بین هر دو تا نود پشت سر هم در توالی یک یال وجود داشته باشد.

$$P_n = \{i_0, i_1, i_2, \dots, i_n\} \qquad P_n = \{(i_0,i_1),(i_1,i_2),(i_2,i_3),\dots,(i_{n-1},i_n)\}$$

به طور کلی یک نود در یک مسیر ممکن است چند بار ظاهر شود

Simple path (مسیر ساده) ← مسیری که هر نود در آن حداکثر یک بار ظاهر شود

دو تا نود u و v داریم و می خواهیم تعداد مسیرهای بین آن ها را بشماریم:

① تعداد مسیرهای به طول 1 ← تعداد یال های بین u و v ، اگر گراف ساده باشد و بین u و v یال وجود داشته باشد این تعداد 1 است و اگر یال وجود نداشته باشد صفر است

② مسیرهای به طول 2 ← فرض کنیم گراف ساده باشد (یال های چندگانه نداریم)

در اینجا تک تک نودهای میانی بین u و v ، iterate می کنیم. برای هر نود میانی k اگر از u به k و هم چنین از k به v یال وجود داشته باشد در این صورت یک مسیر به طول 2 از u به v وجود دارد:

$$H^{(2)}_{uv} = \sum_{k=1}^{N} A_{uk} A_{kv} = [A^2]_{uv}$$

طبق تعریف ضرب ماتریسی اگر ماتریس مجاورت را به توان 2 برسانیم خانه uv آن تعداد مسیرهای به طول 2 بین u و v را نشان می دهد.

به طور کلی تعداد مسیرهای به طول h بین u و v برابر است با:

$$H^{(h)}_{uv} = [A^h]_{uv}$$

یعنی ماتریس مجاورت را به توان h برسانیم و خانه uv ام آن را در نظر بگیریم. این الگوریتم تعداد مسیرها را حساب می کند و کاری ندارد مسیر ساده هست یا نه و اینکه نود تکراری نباشد. حداکثر زمان آن همان مرتبه زمانی ضرب ماتریسی است که $O(n^3)$ است. پس در زمان نسبتاً بهینه ای می توان تعداد مسیرهای بین هر دو نود دلخواه را یافت.

طولانی ترین مسیر هم در زمان چند جمله ای قابل حل است اما یافتن مسیرهای ساده زمان غیر چندجمله ای می برد.

polynomial delay در حل می شود

Scanned with CamScanner

بدترین حالت روی گراف simple path ها نمایی است. نه تنها روی simple path ها نمایی است بلکه exponential است

وقتی یک Simple path را تولید کرده ایم دیگر نمی خواهیم Simple path بعدی را پیدا کنیم زمان نمایی لازم داریم

مهر | ۱۹ | 11 Oct 2021
دوشنبه | ۴ ربیع الاول ۱۴۴۳

۱۴۰۰

Distance in a Graph ← فاصله کوتاهترین مسیر بین دو تا نود

اگر دو تا نود disconnect باشن distance بینشون رو بی نهایت در نظر می گیریم

در گراف های جهت دار باید دقت کنیم که فاصله نود A از نود B برابر با فاصله نود B از نود A برابر نیست. چون عکسه طول shortest path برای AC متفاوت باشه با CA

قطر گراف (diameter) ← longest shortest path

بین هر دو تا نود shortest path را در نظر بگیریم، بین shortest path های مختلف اون که از همه طولانی تره میشه قطر گراف.

جلسه چهارم - ۱۴۰۰/۷/۶

دومین معیار مهم در تحلیل شبکه های پیچیده ← path Length ← طول متوسط مسیر بین نودهای مختلف

در برخی موارد Diameter یا قطر را در نظر می گیریم و بعضی جاها Average path Length

منظور از Avg path Length در واقع Avg shortest path Length هست یعنی بین هر دو نود در گراف (یا جداول در Connected Component گراف) فاصله شان را حساب کنیم و میانگین بگیریم

$$\bar{h} = \frac{1}{2E_{max}} \sum_{i \neq j \in i} h_{ij}$$

معیار سوم: ③ clustering coefficient

حکایتی حول یک نود را بیان می دهد. در واقع بیان می دهد که چه نسبتی از همسایه های یک نود خودشان به همدیگر شان لینک دارند

$$C_i = \frac{2e_i}{k_i(k_i-1)}$$ ← تعداد جفت نودهایی که همسایه نود i هستند (k_i درجه نود i است)

$\frac{k_i(k_i-1)}{2}$ = ← e_i تعداد جفت نودهایی که همسایه i هستند و بینشان یال وجود دارد

Clustering coefficient برای کل گراف به صورت میانگین Cc ها تعریف می شود:

$$C = \frac{1}{N} \sum_{i}^{N} C_i$$

Scanned with CamScanner

تحلیل MSN

یک giant Component داریم که تفاوت اندازه‌اش با بزرگترین Component کوچکتر از خودش خیلی زیاد است. هرچقدر اندازه Component ها کوچکتر است تعدادشان بیشتر است

توزیع درجه: هرچقدر درجه نودها بیشتر شود clustering coefficient نودها کمتری شود

✓ Degree Distribution → Avg Degree = 14.4 → کل گراف اسپارس است → عدد کوچکی است

✓ path Length = 6.6 → نودها خیلی کم تر دیگند → خیلی کمتر از تعداد نودهاست

✓ clustering Coefficient → Avg clustering coefficient = ۰/۱۱ → عدد بزرگی است

سوال: آیا این مقادیر قابل انتظارند؟

برای پاسخ به این سوال نیاز به یک مدل پایه داریم که این مدل را با آن مقایسه کنیم. مدل پایه‌ای که استفاده می‌کنیم مدل ER است

null_model

مدل (ER) Erdos_Renyi

Gnp ← n مقدار نودها ، p به احتمال است، یه عدد بین ۰ و ۱

اول میایم n تا نود ایجاد می‌کنیم بعد بین هر دو نود با احتمال p یک یال ایجاد می‌کنیم (توزیع ما در این حالت یکنواخت است. با احتمال p موفقیت داریم با احتمال p-۱ شکست) (گراف غیرجهتدار)

یک حالت دیگر از این مدل: Gn,m ← n مقدار نودها m ← مقدار یال‌ها

n تا نود ایجاد می‌کنیم، بعد m تا یال ایجاد می‌کنیم. برای ایجاد یال‌ها هر بار یک جفت نود را به صورت تصادفی بطوریکه انتخاب می‌کنیم و بینشان یک یال ایجاد می‌کنیم. این دو مدل در اکثر موارد رفتار یکسانی دارند.

تمرکز ما روی مدل اول است

روز بزرگداشت حافظ

Scanned with CamScanner

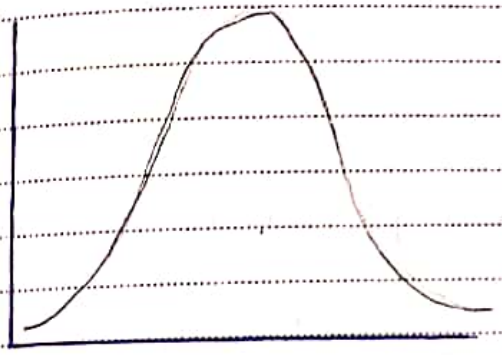
۱۴۰۰

۳ تا معیاری که گفتیم رای خواهیم در مدل ER بررسی کنیم.

$P(E)$ ← احتمال اینکه E تا یال داشته باشیم، از توزیع دوجمله‌ای پیروی می‌کند:

$$P(E) = \binom{E_{max}}{E} p^{E} (1-p)^{E_{max}-E}$$

E_{max} ← تعداد حفت نودها $= \frac{n(n-1)}{2}$



اگر تعداد آزمایش‌ها به سمت بی‌نهایت میل کند توزیع دوجمله‌ای مثل توزیع نرمال می‌شود

← Expected degree برای یک نود چقدر است؟
(امید ریاضی درجه یک نود)

$$X_V = X_{V,1} + X_{V,2} + \dots + X_{V,n-1}$$

$$E[X_V] = E[X_{V,1} + X_{V,2} + \dots + X_{V,n-1}] = E\left[\sum_{u=1}^{n-1} X_{Vu}\right] = \sum_{u=1}^{n-1} E[X_{Vu}]$$

$$= \sum_{u=1}^{n-1} p = (n-1)p$$

توزیع درجه

$P(k)$ ← از بین $n-1$ همسایه‌ای که یک نود میتونه داشته باشه k تا ش پیش اینک داشته باشن

$$P(k) = \binom{n-1}{k} p^{k} (1-p)^{n-1-k}$$

میانگین درجه $= \bar{k} = p(n-1)$

واریانس در توزیع دوجمله‌ای از این راه به دست می‌آید:
تعداد آزمایش‌ها $\times$ احتمال شکست $\times$ احتمال موفقیت

واریانس $= \sigma^2 = p(1-p)(n-1)$

Scanned with CamScanner

معیار coefficiental variation

$$\frac{\sigma}{\bar{k}} = \left[\frac{1-p}{p}\,\frac{1}{(n-1)}\right]^{1/2} \approx \frac{1}{(n-1)^{1/2}}$$

اگر از معیار میزان کجش بورن

مقادیر را در توزیع نشان می دهد.

اینکه k های ما چقدر کجش اند. اما اگر از معیار به مشکلی دارد: scale (مقیاس) دیتا را در نظر نمی گیرد.

مثلا میانگین ۱۰۰۰ باشد و به دیتا ۱۰۰۲ دارید، در حالت دیگری میانگین ۱ است و به دیتای ۳ دارید

در هر دو حالت اگر از معیار سکسانه اما کجش بورن در حالت دوم بیشتر و چند برابر میانگین هست

به همین دلیل از نسبت اگر از معیار به میانگین استفاده می کنند: $\frac{\sigma}{\mu}$

این نسبت یعنی $\frac{1}{(n-1)^{1/2}}$ نشان می دهد که وقتی n خیلی بزرگ می شود به صفر میل می کند یعنی

وقتی n خیلی بزرگ است میزان کجش درجه ها خیلی کم است و همه حول میانگین قرار می گیرند

در این حالت نودهای با درجه های خیلی بالا نخواهیم داشت اما در شبکه دنیای واقعی روز جهانی استاندارد

۲۳ | مهر | جمعه

15 Oct ■ ۸ ربیع الاول

MSN که بررسی می کردیم نودهای با درجه خیلی بالا داشتیم. بنابراین این

مسئله یک عدم سازگاری بین $p_{n,n}$ و گراف دنیای واقعی را

نشان می دهد. به اصطلاح گفته می شود که گراف های دنیای واقعی در نمودار توزیع درجه شان

یک دم کلفت دارند به این معنی که نودهای با درجه خیلی بالا دارند

Scanned with CamScanner

Clustering Coefficient

$C_i = \frac{2e_i}{k_i(k_i-1)}$

e_i یک Random Variable است

که به جای آن expected value را قرار می‌دهیم

$e_i = p\frac{k_i(k_i-1)}{2}$ ← تعداد حفت یال‌ها

↑ احتمال موفقیت یک جفت یال

$C = \frac{p.k_i(k_i-1)}{k_i(k_i-1)} = p = \frac{\bar{k}}{n-1} \simeq \frac{\bar{k}}{n} = \frac{\text{avg degree}}{n}$

چون همه C_iها با هم برابرند ↙

متوسط‌شان هم همین می‌شود

C را برحسب clustering coefficient و avg degree بیان کردیم و برحسب p بیان نکردیم چون در دنیای واقعی p نداریم avg degree و n را داریم و می‌خواهیم این جا با دنیای واقعی مقایسه کنیم $G_{n,p}$ را.

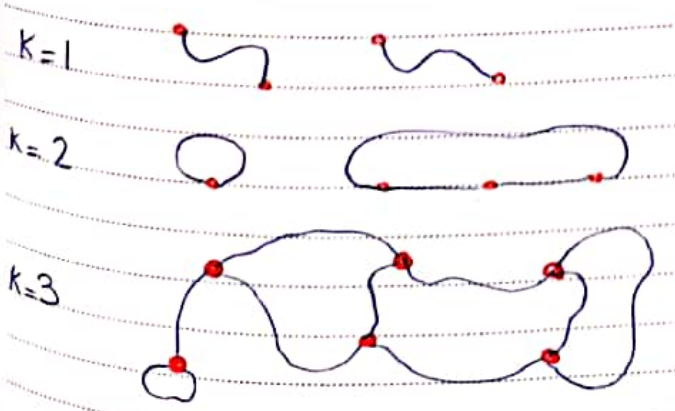
path Length

به جای اینکه گراف‌های $G_{n,p}$ را در نظر بگیریم یک کلاس خاصی از گراف‌های تصادفی به نام "Random K-Regular Graph" را در نظر می‌گیریم.

k-Regular را در نظر می‌گیریم.

در این خانواده درجه همه نودها k هست و یال‌ها به صورت تصادفی به وجود آمده‌اند.

برای هر نود k تا نیم یال در نظر می‌گیریم و بعد به طور تصادفی این نیم یال‌ها را هم متصل می‌کنیم.



سوال: آیا مقدار n و k هر مقدار دلخواهی می‌توانند باشند؟

خیر. مجموع درجه نودها باید زوج باشد. پس $n \times k$ باید زوج باشد بنابراین حداقل یکی از n و k باید زوج باشد.

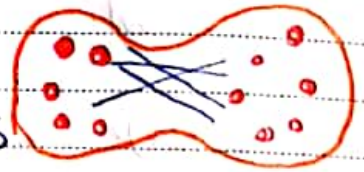
روز ملی پارالمپیک

Scanned with CamScanner

Expansion

گراف $G(V, E)$ دارای α Expantion به صورت زیر است:

$$\alpha = \min_{S \subseteq V} \frac{\#\text{ edges leaving } S}{\min(|S|, |V \setminus S|)}$$



یک زیرمجموعه خاص از نودها مثل S را در نظر بگیرید.

expan ion نسبت به S که با α_S نشان می دهیم برابر است با تعداد یال هایی که یک طرفشان در S است و یک طرفشان در بقیه نودها $(V-S)$ است. برای نرمال کردن تقسیم بر min اندازه |S| و اندازه $|V-S|$ می شود.

expansion برای گراف می شود min α_S ها. برای S های مختلف α را حساب می کنیم min آن ها می شود α برای کل گراف.

α معیاری است از اینکه چقدر شبکه در disconnected می شود. اینکه با حذف یال چقدر دیر می شکند

Scanned with CamScanner

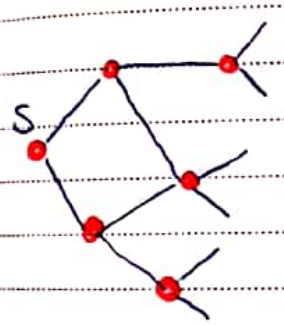
۱۴۰۰

می خواهیم برای گراف های تصادفی k_Regular Random تخمین average path length را انجام بدهیم.

دو نود دلخواه s و t را در گراف k_Regular رندومی که n تا نود دارد در نظر می گیریم و α آن هم مشخص است و نشان می دهیم طول مسیر بین s و t متعلق به $O((\log n)/\alpha)$ است.

یعنی طول مسیر تابعی لگاریتمی از تعداد نودهاست

استراتژی کلی



حول هرکدام از s و t تعدادی توپ (ball) در نظر می گیریم:

S_0 ← توپی که فاصله صفر از نود s دارد ← فقط نود s

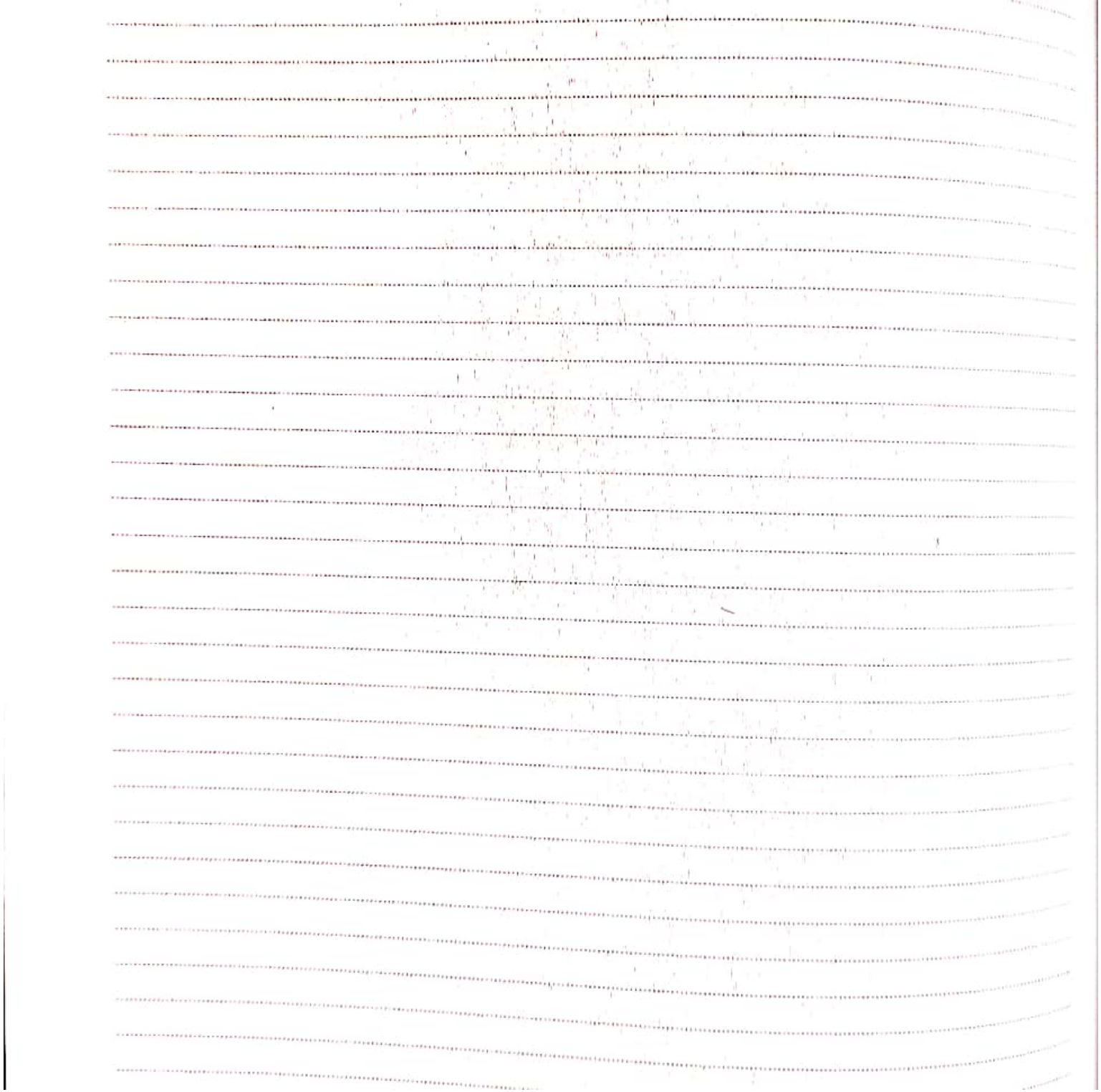
S_1 ← این هایی که فاصله حداکثر 1 از نود s دارند که میشه خود s و همسایه هاش

S_i ← توپی بهرکزیت s که شامل نودهایی است که فاصله شان از s حداکثر برابر با i است

این کار را برای t هم انجام میدهیم و توپ ها را مرحله به مرحله بزرگ تر می کنیم تا جایی که این دو تا توپ که یکی از s شروع شده و رشد کرده و یکی از t، هرکدامشان حداقل نصف نودهای گراف را در بر داشته باشند (حداقل $\frac{n}{2}$)

در این حالت شعاع توپی که از s و t شروع شده هرکدام $\log n$ هست و چون این دو تا توپ هرکدامشان حداقل $\frac{n}{2}$ از نودهای گراف را در بر دارند، کل نودهای گراف در بدترین حالت توسط این دو تا cover می شود و یک مسیر به طول $2\log n$ از s به t وجود دارد

Scanned with CamScanner



Scanned with CamScanner

Scanned with CamScanner

جلسه پنجم – ۱۴۰۰/۷/۱۱

به دنبال رابطه‌ای برای $|S_{j+1}|$ برحسب $|S_j|$ بودیم که به این رابطه رسیدیم:

$$|S_{j+1}| \geq |S_j|\left(1+\frac{\alpha}{k}\right) = \left(1+\frac{\alpha}{k}\right)^{j+1}$$

α ضریب expansion گراف و k درجه نود x است

دنبال این هستیم که بفهمیم اندازه‌ی کره به مرکز s چه‌زمانی بزرگتر مساوی $n/2$ می‌شود و هم‌چنین برای کره به مرکز t. اگر این اتفاق بیفتد فاصله s و t در بدترین حالت می‌شود جمع شعاع کره‌ها به اضافه ۱. (چون گراف متقل است)

می‌خواهیم نشان بدهیم وقتی که $j = \frac{k\log n}{\alpha}$ (شعاع کره) تعداد نودهای ما حداقل $n/2$ می‌شود

$$\left(1+\frac{\alpha}{k}\right)^{\frac{k\log_2 n}{\alpha}} \geq 2^{\log_2 n} = n > \frac{n}{2}$$

پس وقتی شعاع ما می‌شود $\frac{k\log n}{\alpha}$، اندازه کره حداقل $\frac{n}{2}$ است، برای t هم برقراره. بنابراین چیزی که دنبالش بودیم اثبات می‌شود: فاصله s و t در بدترین حالت می‌شود اندازه شعاع کره به مرکز s به اضافه اندازه شعاع کره به مرکز t به اضافه ۱:

$$\frac{k\log n}{\alpha} + \frac{k\log n}{\alpha} + 1 \in O\left(\frac{\log n}{\alpha}\right)$$

α را هم می‌توانیم ثابت در نظر بگیریم. اگر α تا حدی که خواهی بتواند کوچک شود

می‌توانیم از O حذف شود اما فرض می‌کنیم تا حدی که نمی‌تواند کوچک شود و فاصله بین دو نود $O(\log n)$ می‌شود.

نکته حاشیه‌ای: اثبات اینکه چرا $\left(1+\frac{\alpha}{12}\right)^{\frac{k\log n}{\alpha}} \geq 2^{\log_2 n}$

k و n ثابت‌اند. α همیشه برای گراف‌های تصادفی مختلف باهم فرق داشته باشد. باید نشان بدهیم $\left(1+\frac{\alpha}{12}\right)^{\frac{k\log_2 n}{\alpha}}$ را به ازای مقادیر مختلف α این نامساوی برقرار است. اگر مشتق تابع برحسب α بگیریم، مشتق همیشه منفی درمی‌آید. از طرفی $0 < \alpha < k$. در بازه ۰ تا k مشتق ما همیشه منفی است. وقتی مشتق همیشه منفی است یعنی تابع نزولی اکید است و در این بازه نقاط بحرانی نداریم.

روز صادرات

۳۰ | مهر / جمعه

22 Oct ■ ۱۵ ربیع الاول

Scanned with CamScanner

۱۴۰۰

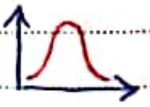
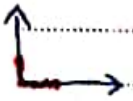
نقاط max و min را ابتدا و انتهای بازه قرار می‌گیرند و برای یافتن حداقل تابع کافی است ابتدا و انتهای بازه را چک کنیم. به ازای $\alpha=0$ و $\alpha=k$ اگر مقدار تابع را حساب کنیم، در هر دو حالت مقدار تابع برابر $2^{\log n}$ می‌شود و تابع ما اکیدا نزولی است پس تابع ما همیشه بزرگتر مساوی $2^{\log n}$ می‌شود.

خلاصه تحلیل ما برای گراف Gnp:

Degree Distribution	$p(k)=\binom{n-1}{k} p^k (1-p)^{n-1-k}$
path Length	$O(\log n)$
clustering Coefficient	$C=p=\bar{k}/n$

Scanned with CamScanner

مقایسه MSN و G_{np}

	MSN	G_{np}
Degree Distribution ↓ با هم متفاوت اند	توزیع درجه ای (binomial) 	از توزیع توانی (power law) پیروی می کند 
path Length ↓ تقریبا با هم برابرند	$O(\log n)$ ← در مدل G_{np} اگر یک گراف تصادفی تولید کنیم که تعداد نودهایش برابر MSN باشد و Density اش هم تقریبا برابر باشد، 8.2 بدست می آید	6.6 ← مقدار path length که تقریبا با 6.6 یکسانه
clustering coefficient ↓	$\bar{k}/n \approx 8.10^{-8}$	0.11

خیلی تفاوت دارند: گراف های دنیای واقعی از جمله MSN مقدار clustering Coefficient شان خیلی بالاست

که نشان می دهد density محلی خیلی بالایی دارند اما در G_{np} مقدارش خیلی خیلی پایینه

← آیا گراف های دنیای واقعی گراف های تصادفی هستند؟ خیر. در برخی موارد شبیه اند اما در برخی موارد تفاوت دارند.

✓ Giant connected component ← از این نقطه نظر شبیه هستند، در هر دو جزء متصل غول پیکر داریم

✓ Average path Length ← شبیه به هم هستند

X clustering coefficient ← تفاوت زیادی دارند

X Degree Distribution ← تفاوت وجود دارد

phase transition

در گراف های دنیای واقعی ما معمولا phase transition نداریم. به این معناست که یکی از پارامترهای ورودی را مقدار کمی (ε) تغییر دهیم، خروجی و رفتار سیستم به شدت تغییر پیدا می کند. در مدل G_{np} این پدیده را برای یک Giant Component داریم، به این معنی که مقدار density را مقدار کمی که زیادی کنیم Giant Component ε ظاهر می شود و تا یکم کم می کنیم ناپدید می شود. (به ناگهان ناپدید می شود اینگونه نیست که به تدریج کوچک شود)

Scanned with CamScanner

۱۴۰۰

اما گراف های دنیای واقعی ... phase transition ... نماینده‌ی پدیده‌هایشان

به صورت تدریجی رخ می دهد. پیدایش و نابودشدن Giant component هم تدریجی است.

گراف های دنیای واقعی از نظر ساخت تصادفی نیستند. بین دو نود با احتمال یکسانی یال به وجود

نمی آید، به همین دلیل Gnp مدل خیلی درستی نیست اما مفید است.

بررسی بیشتر Gnp

امید ریاضی درجه‌ی یک نود دلخواه ← $E[X_v] = p(n-1)$

حالا فرض کنید که $p = \frac{c}{n-1}$، حالا برای c های مختلف می خواهیم بررسی کنیم چه اتفاقی می افتد:

یادآوری: $\lim_{x\to\infty}(1+\frac{1}{x})^x = e$

احتمال اینکه درجه نود v صفر باشد چقدر است؟

$$P[v \text{ has degree } 0] = (1-p)^{n-1} = \left(1-\frac{c}{n-1}\right)^{n-1} \xrightarrow[n\to\infty]{} e^{-c}$$

پس احتمال اینکه یک نود مشخص درجه‌اش صفر باشد: e^{-c}

event I ← احتمال اینکه حداقل یک نود درجه‌اش صفر باشد

$I = \bigcup_{v\in N} I_v$ ← یا نود اول درجه‌اش ۰ باشد یا نود دوم و ...

$$P(I) = P\left(\bigcup_{v\in N} I_v\right) \leq \sum_{v\in N} P(I_v) = ne^{-c}$$

یادآوری: $P(A\cup B) \leq P(A) + P(B)$

$P(I)$ را با ne^{-c} تقریبی می‌زنیم $P(I) = ne^{-c}$

مثال:

$c = \ln n \Rightarrow ne^{-c} = ne^{-\ln n} = n \times \frac{1}{n} = 1$

وقتی c را برابر $2\ln n$ قرار دهیم یعنی density را دو برابر کرده ایم

احتمال اینکه حداقل یک نود با درجه صفر داشته باشیم ۱ است

Scanned with CamScanner

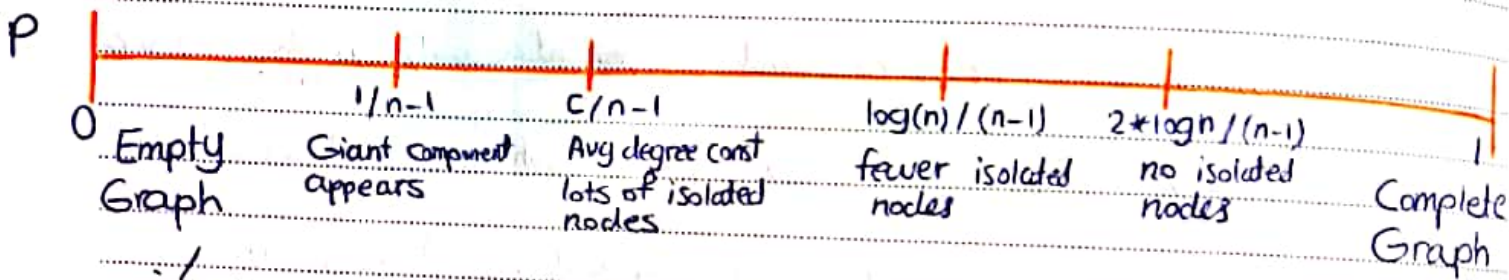
$c = 2\ln n \Rightarrow ne^{-2\ln n} = n.\frac{1}{n^2} = \frac{1}{n} \xrightarrow{n\to\infty} 0$

پس وقتی حطابی ۲ برابر شود، احتمال اینکه نود ایزوله داشته باشیم می‌شود صفر (برای nهای بزرگ)

وقتی احتمال اینکه یک نود ایزوله باشد برابر e^{-c} باشد، امید ریاضی مقدار نودهای با درجه صفر

می‌شود ne^{-c}

Evolution of a Random Graph



در این طایف سری مشخصات گراف Gnp برای pهای مختلف (densityهای مختلف) را بررسی می‌کنیم

وقتی $p=0$ ← با احتمال صفر یال ایجاد می‌کنیم ← گراف Empty

وقتی $p=1$ ← گراف کامل

وقتی $p=\frac{1}{n-1}$ ← اولین بار Giant Component ما به وجود می‌آید. در این حا اتفاق phase transition

می‌افتد. اگر $p=\frac{1-\varepsilon}{n-1}$، Giant Component ناگهان ناپدید می‌شود، اگر $p=\frac{1+\varepsilon}{n-1}$ ناگهان ظاهر می‌شود. ε یک عدد

خیلی کوچک است، نزدیک به صفر.

وقتی $p=\frac{c}{n-1}$ (که c ثابتی بزرگتر از ۱ است) ← مقدار خیلی زیادی نود ایزوله داریم

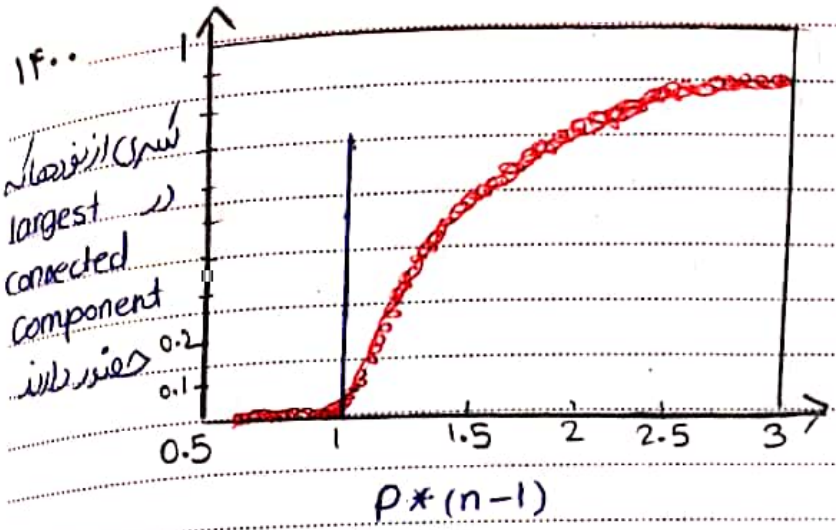
اگر $p=\frac{\log(n)}{n-1}$ ← احتمال اینکه حداقل یک نود ایزوله داشته باشیم ۱ است یعنی حتما یک نود ایزوله

داریم ولی تعداد نودهای ایزوله خیلی کم است

اگر $p=\frac{2*\log(n)}{n-1}$ ← احتمال اینکه نود ایزوله داشته باشیم به سمت صفر میل می‌کند

نکته: افزایش تعداد نودها باعث افزایش Avg path Length به صورت linear می‌شود. یا sublinear یا logarithmic اتفاق می‌افتد این افزایش.

Scanned with CamScanner



در این جا به صورت تجربی مورد بررسی قرار داده شده. محور افقی $p*(n-1)$ است که در واقع Avg degree را نشان می دهد. با افزایش محور افقی $p*(n-1)$ برابر یک است یعنی p برابر $\frac{1}{n-1}$ بزرگتر است، Giant Component به وجود می آید. خط آبی رنگ جایی را نشان می دهد که phase transition داریم. p به اندازه از $\frac{1}{n-1}$ بزرگتر که بیشتر Giant Component به وجود می آید. در $p = \frac{1-\varepsilon}{n-1}$، Giant Component یکدفعه از هم می پاشد. در حالت $p = \frac{1}{n-1}$ ناپایدار داریم. بین این دو وضعیت سوئیچ می کند.

Configuration Model

برای حل مشکل عدم سازگاری توزیع درجه این روش پیشنهاد شد

یک توالی درجه از یک گراف دنیای واقعی داریم می خواهیم یک گراف با همان توزیع درجه بسازیم

توالی درجه گراف دنیای واقعی : $k_1, k_2, \dots k_N$

از مدل نیم یال استفاده می کنیم. N تا نود ایجاد می کنیم، نود اول k_1 تا نیم یال دارد، نود دوم k_2 تا نیم یال، نود سوم k_3 تا یال و ... این نیم یال ها را به صورت تصادفی کم وصل می کنیم و تبدیل به تمام یال می کنیم. در نتیجه گراف خواهیم داشت که اتلا تصادفی است و دوما توالی و توزیع درجه این دقیقاً مثل یک گراف دنیای واقعی است

Scanned with CamScanner

پنجشنبه

پدیده‌ی دنیای کوچک (Small-world)

مشکل آن روی حل مستقل کم بودن clustering coefficient در مدل Gnp است

مفهوم این پدیده: نودهای شبکه بیشتر از چیزی که مورد انتظار هست کنار هم نزدیک‌ترند.

این پدیده در حالتهای مختلف دیده شد. از جمله در گراف هالیوود، گراف ریاضیدان‌ها و...

در نهایت به ۶ درجه‌ی جدایی (6 degree of seperation) رسیدند

سوال: آیا این پدیده مورد انتظار است؟

یک مثال:

step 1: 100 people

step 2: reach 100*100 = 10,000 people

step 3: reach 100*100*100 = 1,000,000 people

Step 4: reach 100*100*100*100 = 100M people

با شروع از ۱۰۰ نفر می‌توانیم به ۱۰۰ میلیون نفر در ۴ گام برسیم

آیا این مثال درست است؟

۷ | آبان | جمعه

29 Oct ■ ۲۲ ربیع الاول

خیر چون افرادی که می‌بینیم جدید نیستند. خیلی از افرادی که در گام‌های بعدی می‌بینیم، قبلا دیده شده‌اند، چون دوست‌های دوستان ما، دوست ما هم هستند.

در واقع باوجود clustering coefficient بالا هست که پدیده‌ی small world پدیده‌ی جالبیه

همین که دوستان دوست ما، دوست ما هم هستند

حتی شاید وجود clustering coefficient بالا در کنار path Length پایین تناقض جزیی به نظر برسد: چون در پدیده‌ی small world از یک طرف انتظار داریم که یال‌های تصادفی داشته باشیم زیرا یال‌های تصادفی موجبی می‌شود که تعداد گام‌های مختلف گراف کم و معقول شوند

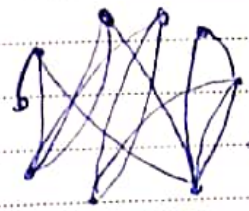
Scanned with CamScanner

۱۴۰۰

در نتیجه expansion ما نمی‌تواند خیلی پایین بشود، یه قسمتی از گراف را نمی‌توانیم جدا کنیم
یه قسمت از گراف خیلی well_seperated نیست از بقیه گراف. در نتیجه وجود یال های تصادفی
کمکی می‌کند که small world داشته باشیم. در واقع این یال ها مثل shortcut عمل می‌کنند
و قسمت های مختلف گراف را کجم وصل می‌کنند.

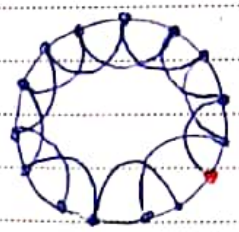
از طرف دیگر برای اینکه clustering coefficient بالا داشته باشیم باید یال های خیلی
منظم داشته باشیم و هر دو نودی که یک همسایه مشترک دارند باید هم وصل باشند.
اینگونه نباشد که یک طرف گراف را به طرف دیگه وصل کنند. یال های ما به اصطلاح
باید خیلی محلی (local) باشند.

در نتیجه این ۲ مورد با هم تناقض دارند. چگونه ممکن است این ۲ مورد را با هم داشته باشیم؟
یک جواب طبیعی این است که هر دو نوع یال را با هم داشته باشیم



Low clustering
Low diameter

یال ها کاملا تصادفی اضافه شده
مرتب شده فاصله ها خیلی کم باشد از مرتبه O(log n)
و clustering coefficient خیلی پایین



High clustering
High diameter
(گرید ۱ بعدی)

یال ها خیلی منظم و دقیق چیده شده اند
بین اکثر نودها این که همسایه مشترک دارند یال اضافه شده
اما فاصله ها زیاد شده. از مرتبه O(n) شده

باید دو حالت را ترکیب کنیم.

مدل Small world

این مدل ۳ تا پارامتر دارد ← ① تعداد نودها (n) ② تعداد همسایه‌های محلی هر نود (r)

③ p ← احتمال ← عددی بین ۰ و ۱ که با آن randomness ایجاد می‌کنیم

Scanned with CamScanner

ابتدا یک grid با n تا نود ایجاد می‌کنیم (n تا نود را روی دایره می‌چینیم)

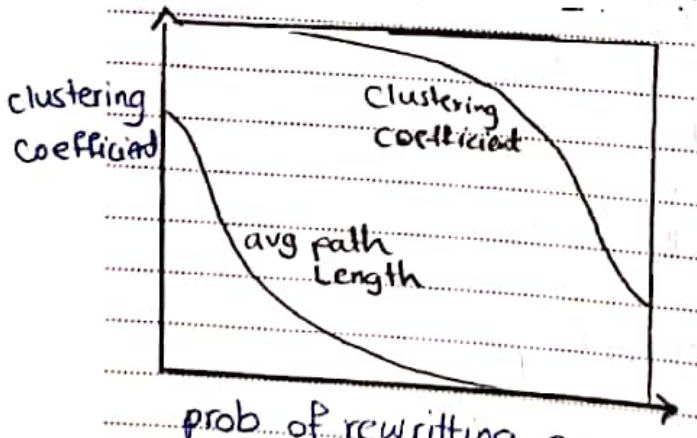
به هر نود را به k تا نود قبل و k تا نود بعد از خودش وصل می‌کنیم. تا این جا یک grid یک بعدی منظم ایجاد کرده‌ایم که هر نود k همسایه دارد و یال‌های ما خیلی محلی هستند و ضریب خوشه‌بندی بالایی دارند.

در گام بعد rewire یا سیم‌کشی مجدد انجام می‌دهیم: برای هر یال مستقل از یال‌های دیگر با احتمال p یکی از endpoint های آن یال را ثابت نگه می‌داریم و endpoint دیگر آن را به صورت تصادفی انتخاب می‌کنیم. با احتمال 1-p یال را دست نمی‌زنیم.

← به صورت uniformly at random از n-1 نود دیگر انتخاب می‌کنیم

میزان تصادفی بودن گراف تولید شده را با پارامتر p تعیین می‌کنیم هرچقدر p بزرگتر باشد میزان تصادفی بودن بیشتر است

سوال: چرا این مدل کار می‌کند؟ چرا هم ضریب خوشه‌بندی بالایی دارد هم Avg path length پایین؟



توضیح ← احتمال rewire کردن

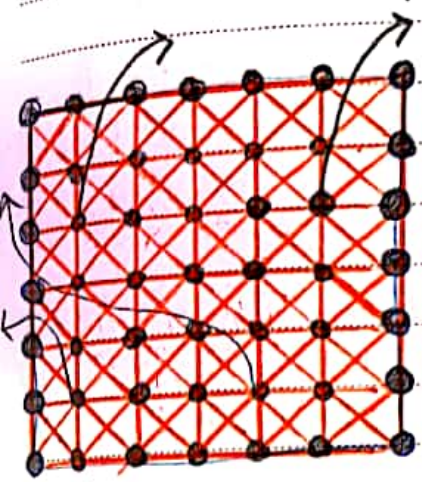
وقتی p=0 ← گراف منظم داریم ← ضریب خوشه‌بندی و Avg path Length هر دوشان بالا هستند

وقتی p را کمی زیاد می‌کنیم، Avg path Length به شدت کاهش پیدا می‌کند اما ضریب خوشه‌بندی به آرامی کاهش پیدا می‌کند. پس به ناحیه‌هایی برای p داریم که توش Avg path length شدیدا کم شده، از مرتبه O(n) به O(log n) رسیده ولی ضریب خوشه‌بندی خیلی کم کاهش پیدا کرده.

Scanned with CamScanner

جلسه ششم ـ ۱۴۰۰/۷/۱۸

یک مدل ساده‌تری از مدل walts_strogatz را می‌خواهیم بررسی کنیم:



یک grid دوبُعدی داریم. در ابتدای کار یک grid دوبعدی می‌سازیم بعد برای هر نود یک نیم‌یال ایجاد می‌کنیم و بعد این نیم‌یال‌ها را به صورت تصادفی جفت می‌کنیم

بررسی Clustering Coefficient:

برای نودهای داخلی ← $C_i = \frac{2e_i}{k_i(k_i-1)} = \frac{2\times 12}{9\times 8} \geq 0,33$

← ۸ یال منظم + یک یال تصادفی

تعداد نودهای داخلی از مرتبه n^2 هستند، تعداد نودهای مرزی از مرتبه n هستند بین رفتن میانگین بگیریم نودهای داخلی غلبه دارند. درنتیجه این گراف ضریب خوشه‌بندی بالایی دارد.

بررسی Avg. path length:

هر ۴ تا نود را یک ابرنود در نظر می‌گیریم. بین دو تا ابرنود یک یال ایجاد می‌شود اگر و فقط اگر بین یکی از نودهای ابرنود اول و یکی از نودهای ابرنود دوم، بینشان یک یال تصادفی وجود داشته باشد (یال‌های منظم را ignore می‌کنیم). بنابراین گراف حاصل، یک گراف منظم تصادفی است. همینه‌ای اثبات کردیم که در k_regular random graph مقدار path length از مرتبه $o(\log n)$ است. بنابراین در گراف حاصل، بین هر دو تا نود یک مسیر به طول $o(\log n)$ داریم که n تعداد نودهای گراف است

Scanned with CamScanner

می خواهیم نشان بدهیم بین یک نود دلخواه از یک ابرنود و یک نود دلخواه از ابرنود دیگر، مسیری به طول $O(\log n)$ وجود دارد.

لمّه اول: طبق آن می‌گفتیم بین هر دو ابرنود یک مسیر به طول $O(\log n)$ وجود دارد

لمّه دوم: داخل هر ابرنود، همه‌ی نودها کم متصل اند (گراف ما کامل است)

بنابراین از ابرنود مربوط به نود دلخواه اول به ابرنود مربوط به نود دلخواه دوم در $O(\log n)$ می‌توان رسید و چون داخل هر ابرنود گراف کامل داریم با پیمودن یال‌های تصادفی بعد از آن یال‌های درون هر ابرنود به نود دلخواه در $O(\log n)$ می‌رسیم. در واقع از مسیر $\log n$ ای که توسط یال‌های تصادفی به وجود آمده است استفاده کردیم.

مدل watts_strogatz نشان داد که امکان جمع شدن ضریب خوشه بندی بالا در کنار Avg path length پایین وجود دارد. به این صورت که کافی است علاوه بر یال‌های منظم، مقداری یال‌های تصادفی داشته باشیم.

اما این مدل یک عیب دارد، اینکه توزیع درجه‌ی آن (degree distribution) سازگار با گراف‌های دنیای واقعی نیست. گراف‌های دنیای واقعی توزیع درجه‌شان power low است اما توزیع درجه این مدل از خانواده توزیع‌های نرمال (شبیه توزیع دوجمله‌ای) است. توزیع جملی پیچیده‌ای دارد اما power low نیست و خیلی شبیه به توزیع گراف‌های ER است.

یک عیب دیگر این مدل این است که گراف حاصل امکان Navigation و مسیریابی low efficient را نمی‌دهد. اصطلاحاً گفته می‌شود Searchable نیست یا Social Search به صورت ریشه روی این گراف قابل انجام نیست. Social Search و decentralized search به چه معناست؟

Scanned with CamScanner

Social Search

شما مکان جغرافیایی خودتان و مقصدتان را دانید. دوستان خودتان که با آن‌ها لینک دارید را می‌شناسید و موقعیت جغرافیایی هرکدام از دوستانتان را هم می‌دانید.

بین دوستانتان بهتر است به کسی می‌دهید که از نظر جغرافیایی به مقصد نزدیک‌تر است. به این فرآیند decentralized search (یا Social Search) می‌گویند.

اگر گراف ما به گونه‌ای باشد که تعداد گام‌هایی که در بدترین حالت طی می‌شود تابعی بسته از نقطه شروع و نقطه مقصد باشد، به گراف Searchable است.

یکی از ویژگی‌های گراف‌های دنیای واقعی این است که Searchable هستند.

اگر فرآیند Search از مرتبه $O(n)$ یا توانی از n باشد می‌گوییم گراف Searchable نیست.

همانطور که گفتیم گراف‌های دنیای واقعی طول مسیر بین دو نود دلخواه $O(\log n)$ است. بنابراین وقتی گراف Searchable است به این معناست که مکانیسم Search برحسب طول مسیر، خطی (linear) است یا در بدترین حالت polynomial است.

اما وقتی فرآیند Search، $O(n)$ طول می‌کشد به این معناست که زمان جست‌وجو برحسب طول مسیر، غایی است.

Searchable	Not Searchable
Search Time: $O((\log n)^{\beta})$	Search Time: $O(n^{\alpha})$
Kleinberg's Model	watts_strogatz → $O(n^{2/3})$
$O((\log n)^2)$	Erdös-Rényi → $O(n)$

Scanned with CamScanner

اگر تغییر کوچکی در گراف Small_world بهیم مدل Kleinberg بپیوندی آید که هنوز در کلاس های
خوب مدل Small_world باقی باشد به اضافه ی اینکه Searchable هم می شود. در گراف Small world
مقدیال تعدادی را به صورت uniformly at Random انتخاب می کردیم. در مدل Kleinberg مقدیال
تعدادی متناسب با عکس فاصله نودها با نود مبدأ انتخاب می شود. [دقت کنید که هنوز مسئله ی توزیع درجه
حل نکرده ایم]
با یک احتمالی

حل دو مسئله در شبکه های پیچیده

مسئله اول: بیشینه سازی تأثیر (influence maximization)

یک شرکتی برای اینکه موفق باشد می خواهد تبلیغ اش تا جای ممکن در شبکه های اجتماعی گسترش
پیدا کند ← viral marketing (بازاریابی ویروسی)

گام اول این است که افراد influential را پیدا کنید در شبکه اجتماعی

گام دوم اینکه این افراد را قانع کنید محصول شما را معرفی کنند و بخرند

گام سوم چون این افراد موثر هستند، تا پیدا آن ها در شبکه اجتماعی
خیلی گسترش پیدا می کند.

گام چهارم: در نتیجه اگر فرد خیلی موثر باشد تبلیغ شما بسیار گسترش پیدا می کند

سوال کلیدی این است که نودهای تاثیر گذار را چگونه پیدا کنیم؟

مدل گراف (یک شبکه پیچیده) داریم. می خواهیم نودهایی را پیدا کنیم که اگر پدیده ای را از این مجموعه
نودها شروع کنیم، این پدیده ما بیشترین انتشار را پیدا می کند.

اینکه این مجموعه نودها چه نودهایی هستند به خود فرآیند انتشار هم بستگی دارد. به اینکه انتشار ما روی این
گراف، چگونه انجام می گیرد. مدل های مختلفی برای مدلسازی انتشار وجود دارد. ما این جا یک مدل خیلی ساده و پرکاربرد
را مطرح می کنیم و فرض می کنیم پدیده انتشار ما از این مدل پیروی می کند:

Scanned with CamScanner

۱۴۰۰

(مدل آبشاری مستقل) Independent Cascade Model

در این مدل فرض می‌کنیم گراف زیر ما که پدیده انتشار روی آن صورت می‌گیرد جهتدار است

یک مجموعه آغازین از نودها داریم که Active (فعال) هستند ← S

این گراف ما یک گراف وزن دار احتمالی است یعنی به هر یال (u,v) یک احتمال P_{uv} نسبت می‌دهیم

P_{uv} نشان می‌دهد اگر source این یال که v است فعال شد، با چه احتمالی نود مقصد که u است

فعال خواهد شد. به عبارت دیگر اگر نود v فعال شده باشد نود u دقیقاً یک بار این شانس را پیدا

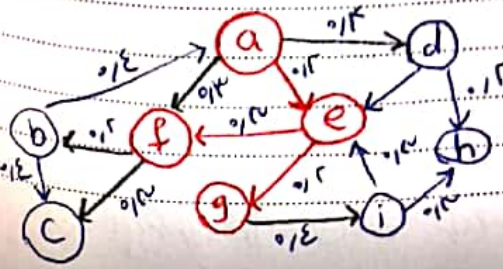
می‌کند که از طریق نود v فعال شود با احتمال P_{vu}.

زمان‌بندی از یک جهت مهم است و از جهت دیگر مهم نیست:

① فرض کنید نود u دو همسایه v_1 و v_2 دارد که فعال هستند. نود u این احتمال را پیدا می‌کند که توسط v_1 با احتمال P_{v_1u} و توسط نود v_2 با احتمال P_{v_2u} فعال شود. ما در این جا زمان‌بندی (scheduling) برایمان مهم نیست. اینکه ابتدا کدام یک u را فعال می‌کند مهم نیست.

② اما ما در یک سری گام‌های زمانی گسسته جلو می‌رویم، در هر گام زمانی یک سری نود فعال و یک سری نود غیرفعال داریم. تا زمانی که همه‌ی نودهای ما فعال شوند یا تا زمانی که همه‌ی یال‌هایی که متصل اند به نودهای ما، شانس فعال کردن متقابلشان انجام گرفته باشد. در هر مرحله نودهایی که غیرفعال هستند و یک یال از نودهای فعال به این نودها وجود دارد و این یال هنوز یک تلاشش برای فعال شدن انجام نگرفته، می‌آیم این را انجام می‌دهیم. به صورت time step های گسسته جلو می‌رویم و این فعال‌سازی را انجام می‌دهیم

مثال:



Scanned with CamScanner

S: مجموعه نودهای فعال آغازین

f(S): اندازه مورد انتظار مجموعه فعال کنایی اگر از S شروع کنیم

X_u: همان f(S) است درصورتی که S ما یک نود باشد ← u (برای هر نود یک X تعریف می‌کنیم)

مسئله: ما می‌خواهیم بین تمام زیرمجموعه های با اندازه K که در گراف وجود دارد، بهترین مجموعه S ای با اندازه K را انتخاب کنیم به طوری که f(S) ما کسیمم شود. (اندازه مورد انتظار انتشار ما اگر از S شروع شود max شود)

$$\max_{S \text{ of size } k} f(S)$$

expected size ← در عمل چگونه پیاده می‌کنیم؟

یک سری realization از گراف رندوم در نظر می‌گیریم. تصمیم می‌گیریم با یه احتمالی این یال بماند یا از بین برود

بنابراین گراف باقی مانده یک گراف قطعی است. گراف احتمالی ما را به گراف قطعی تبدیل می‌کنیم.

بعد ما روی یال هایی که باقی مانده اند به صورت قطعی انتشار می‌یابد.

گراف حاصل یک گراف قطعی است. روی هر یال قطعی پیده ما با احتمال ⊥ انتشار می‌یابد.

این کار را یکبار انجام نمی‌دهیم بلکه مقدار زیادی تکرار می‌کنیم. به اصطلاح مقداری realization از گراف ایجاد می‌کنیم. برای اینکه خطا کاهش پیدا کند.

$$f(S) = \sum_{\text{Random Realization } i} f_i(S)$$

$f_i(S)$ ← Score ای که S دارد روی گراف i ام.

هرکدام از تها یک Realization هستند، یکی از گراف های قطعی هستند

سوال: این مسئله از نظر hardness چگونه است؟

راه حل بدیهی: اینکه بیاییم تک تک زیرمجموعه های K تایی از نودها را در نظر بگیریم، f را برای تک تک آن‌ها حساب کنیم و ببینیم که f این max میشه به عنوان خروجی بفرستیم.

time Complexity این راه حل برحسب K و n عبارتی (exponential) است. در واقع این مسئله، یک مسئله np_hard است.

Scanned with CamScanner

۱۴۰۰

اینکه راه‌حل بدهی شما exponential است همیشه به شما نمی‌گوید که این مسئله np_hard است np_hard است.

اما ثابت شده که این مسئله برحسب K (تعداد رئوس‌های مجموعه S) و n (تعداد کل نودها) np_hard است.

ورژن decision (تصمیم) آن np_complete است و ورژن optimization این np_hard است.

به طور کلی اگر مسئله‌ای ورژن optimization این np_hard باشد ورژن decision این np_complete است.

در ورژن decision مسئله اینطور مطرح می‌شود:

آیا مجموعه‌ای با اندازه K وجود دارد به طوری که f آن بیشینه شود؟ (f آن برای fmax شود)

fmax یعنی انتشار را ما کل نودها را در بگیرد. (با شروع از S)

جواب مسئله تصمیم (decision) به صورت yes یا no است.

وقتی مسئله decision، np_complete است verify کردن جواب (درستی آزمایی جواب) در زمان polynomial قابل انجام است ولی پیدا کردن خود جواب در زمان exponential انجام می‌شود.

سوال: چگونه اثبات می‌شود که یک مسئله np_hard است؟

با همکاری reduction (کاهش) است. کاهش یعنی شما به تعدادی مسئله شناخته شده دارید که می‌دانید np_hard است، شما مسئله‌تان را یکی از این مسائل شناخته شده کاهش می‌دهید. این reduction در زمان polynomial انجام می‌شود.

از آن جایی که این تبدیل در زمان poly nomial انجام می‌گیرد، مسئله قدیم هم در زمان exponential حل می‌شود، پس مسئله جدید شما هم np_hard خواهد بود و در زمان exponential حل می‌شود.

مسئله جدید شما نمی‌تواند polynomial باشد.

Scanned with CamScanner

مسئله Set Cover

یک مجموعه U داریم $U = \{u_1, \dots, u_n\}$

m تا X_i داریم که زیرمجموعه U هستند $X_1, \dots, X_m \subseteq U$

سوال: آیا k تا از این X_iها وجود دارند به طوری که اجتماع آن‌ها با U را بپوشانند (کل U را cover کنند)

یک نمونه از مسئله set cover داریم. در زمان polynomial آن را تبدیل به مسئله influence maximization می‌کنیم. این تبدیل باید در زمان polynomial انجام بشود.

بعد مسئله influence Maximization را حل می‌کنیم بعد باید بیاییم جواب مسئله influence Maximization را شان بدهیم که در زمان polynomial قابل تبدیل به جواب مسئله set cover است.

چون این تبدیلات در زمان polynomial انجام می‌گیرد، ما فقط آغاز زمان یک نمونه از مسئله set cover و نقطه پایانش جواب مسئله set cover است. تبدیلات وسط هم polynomial است. فقط در این بین، حل مسئله influence Maximization باقی می‌ماند. اگر پیدا کردن جواب influence Maximization هم در زمان polynomial انجام بگیرد، یعنی کل این مسئله در آمد در زمان polynomial انجام گرفته پس مسئله set cover به polynomial هم در influence Maximization است و این غیرممکن و متناقض است. در نتیجه حل مسئله influence Maximization هم در زمان polynomial قابل انجام نیست و np_hard است.

بنابراین این موارد را باید بتوانیم بدهیم:

① یک نمونه از مسئله Set Cover در زمان polynomial به یک نمونه از مسئله IM تبدیل می‌شود

② جواب مسئله IM در زمان polynomial به جواب مسئله set cover تبدیل می‌شود.

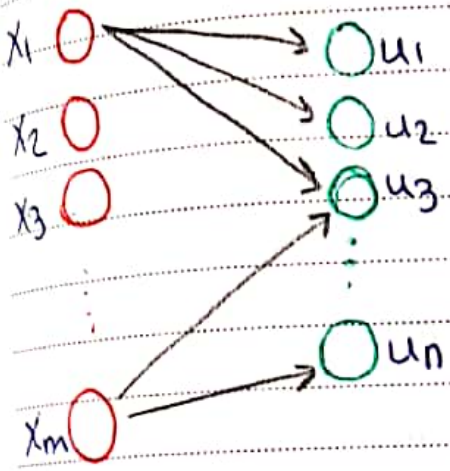
Scanned with CamScanner

۱۴۰۰

فرض کنید یک نمونه از مسئله set Cover به ما داده شده:

$X_1, X_2, \dots, X_m$ $\quad U=\{u_1, u_2, \dots, u_n\}$

از روی این مسئله می‌خواهیم یک مسئله IM بسازیم:



یک گراف دوبخشی به صورت روبه رو ایجاد می‌کنیم.

یک یال از X_i به u_j ایجاد می‌کنیم اگر u_j عضوی از X_i باشد. مثال: $X_1=\{u_1, u_2, u_3\}$

مسئله IM روی این گراف به این صورت است:

آیا k تا نود در رنگ وجود دارد به طوری که influence این نودها برابر با max مقدار $(k+n)$ باشد؟

$k+n \leftarrow$ خود k تا نود در به اضافه کل نودهای u

بنابراین مسئله set cover را در زمان polynomial تبدیل کردیم به IM. حالا باید جواب IM را در زمان polynomial تبدیل کنیم به جواب مسئله set cover.

فرض کنید یک جواب برای مسئله IM یافته‌ایم. یعنی یک مجموعه k تایی از X_iها یافته‌ایم که آن‌ها برابر $k+n$ شده. معنیش اینه خودِ k تا X_i که داریم کل u_1 تا u_n را cover می‌کنند چون به u_1 تا u_n یال دارند بنابراین u_1 تا u_n عضو این k تا X_i ها هستند در نتیجه کل U را cover می‌کنند بنابراین خیلی مستقیم و در زمان polynomial جواب مسئله IM به جواب مسئله set cover تبدیل می‌شود.

⇐ مسئله Influence Maximization، np_hard است.

Scanned with CamScanner

جلسه هفتم - ۱۴۰۰/۷/۲۰

گفتیم مسئله‌ی IM، np_hard است و پیچیدگی زمانی نمایی دارد. اما الگوریتم‌های تقریبی هم برای حل آن وجود دارند که کارایی خطایی به صورت روبه رو دارند: $f(S) > 0.63 * f(OPT)$

یعنی اندازه‌ی دامنه‌ی انتشار S بزرگتر از ۰/۶۳ اندازه دامنه انتشار OPT (بهینه) است.

الگوریتم تقریبی:

یک الگوریتم greedy (حریصانه) است.

وروری: دامنه‌ی انتشار هر نود u: $X_u \leftarrow$ مجموعه نودهایی که با شروع از نود u به آن‌های رسیم (out u)

در گام اول به عنوان اولین عضو مجموعه S نودی را انتخاب می‌کنیم که بیشترین اندازه out را دارد. بیشترین اندازه نودهای خروجی را دارد. (یعنی X اش از همه بزرگتر است)

بعد در هر iteration، فرض کنید که عضو اول تا i-1 ام مجموعه S را پیدا کرده‌اید ← S_{i-1} را یافته‌اید

برای انتخاب نود i ام، آن نودی را انتخاب می‌کنیم که بین همه‌ی نودهای اضافه نشده به S، بیشترین marginal gain را داشته باشد.

۲۱ | آبان | جمعه

12 Nov ■ ۶ ربیع الثانی

یعنی چنین عبارتی را max کند: $\max_u f(S_{i-1} \cup \{u\})$

بنابراین در هر iteration، i ام، برای تک تک نودهای u که باقی مانده‌اند، آن‌ها را به S_{i-1} اضافه می‌کنیم و f این مجموعه حاصل را حساب می‌کنیم. درنهایت u ای را انتخاب می‌کنیم که $f(S_{i-1} \cup \{u\})$ را max کند

← این جا دوتا حلقه‌ی تودرتو داریم، یک حلقه روی iteration های مختلف ← $i = 1, 2, 3, \ldots, k$ ی خواهیم یک مجموعه با اندازه k تولید کنیم: $S_1, S_2, \ldots, S_k$. یک حلقه دیگر هم داریم که روی هر iteration، i ام، میایم تک تک نودها را انتخاب می‌کنیم. marginal gain تک تک نودها را حساب می‌کنیم.

(نیازی به connected بودن گراف نداریم برای این الگوریتم)

Algorithm:
- start with $S_0 = \{\}$
- For $i = 1, \ldots, k$
  - Activate node u that $\max f(S_{i-1} \cup \{u\})$
  - let $S_i = S_{i-1} \cup \{u\}$

Scanned with CamScanner

تحلیل تئوری الگوریتم:

$$f(S) \geq \left(1-\frac{1}{e}\right) * f(OPT)$$

چرا این رابطه برقرار است؟

این تحلیل که این جا ارائه می دهیم مستقل از هر مسئله ای کار می کند به شرط اینکه f شما دو تا ویژگی داشته باشد. یعنی این الگوریتم می تواند برای مسائلی به غیر از IM هم استفاده شود به شرط اینکه f که در حقیقت امتیاز یا نمره یک مجموعه S را تعیین می کند، این دو ویژگی را دارا باشد:

① f شما monotone باشد: if $S \subseteq T$ then $f(S) \leq f(T)$ and $f(\{\})=0$

یعنی وقتی عضو جدید به مجموعه S اضافه می کنید f اش بزرگتر شود یا ثابت بماند، کمتر نشود.

این ویژگی در مسئله IM برقرار هست، S_i را داریم و عضو $i+1$ ام را اضافه می کنیم، $f(S_{i+1})$ بزرگتر مساوی $f(S_i)$ می شود چون نود جدید یک سری out دارد که ممکن است در out نودهای قبلی نباشد

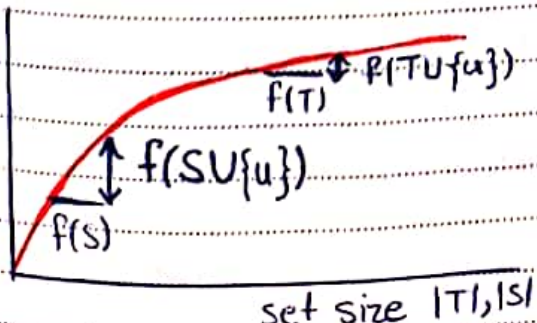
② f باید submodular باشد: فرض کنید S و T دو مجموعه هستند که $S \subseteq T$ و u عضوی است که نه در S و نه در T هست، آن گاه:

$$f(S \cup \{u\}) - f(S) \geq f(T \cup \{u\}) - f(T)$$

(باید داشته باشیم)

اگر تابع f این ویژگی را برآورده کند می گوییم f submodular است

یعنی اگر یک عضو جدید u را به زیرمجموعه S اضافه کنید marginal gain بیشتری به دست می آورید نسبت به زمانی که u را به T اضافه کنید.



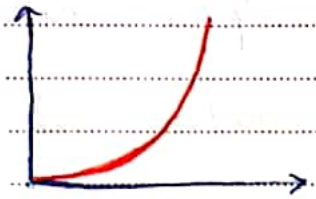
محور افقی اندازه مجموعه ها را با توجه به زیرمجموعه بودن بالا نشان می دهد. محور عمودی مقدار f را نشان می دهد.

اگر نمودار f چنین شکلی داشته باشد آن گاه:

marginal gain T $\leq$ marginal gain S

Scanned with CamScanner

خاصیت submodularity برقرار است به این خاصیت Diminishing Returns هم گفته می‌شود (برگشت کاهشی) ← یعنی هرچقدر اندازه مجموعه‌ها بزرگتر می‌شود میزان برگشت که همان marginal gain است کوچکتر می‌شود.



مثال از f ای که submodular نباشد: در ابتدا میزان افزایش کم باشد بعد کم شود.

در ادامه باید نشان دهیم f ما مونوتن، submodular است و $f(S) \geq 0.63 f(OPT)$

اثبات submodularity: ابتدا یک ویژگی در مورد توابع submodular بیان می‌کنیم:

فرض کنید $f_1(x), f_2(x), \dots, f_K(x)$ توابع submodular باشند. در این صورت هر ترکیب خطی غیرمنفی از آن‌ها submodular است

$$F(x) = \sum_i c_i \cdot f_i(x) \quad c_1, \dots, c_K \geq 0$$

ویژگی دوم:

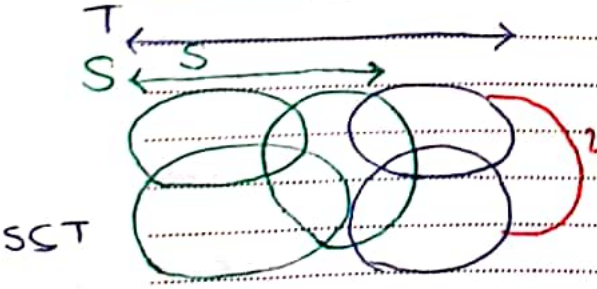
فرض کنید مجموعه‌های $X_1, X_2, \dots, X_m$ را داشته باشیم که هرکدام مثلا مجموعه‌ای از نودها هستند

فرض کنید S مجموعه‌ای از اینکس‌های 1 تا m باشد. f این زیرمجموعه از اینکس‌ها را به این شکل تعریف می‌کنیم:

$$f(S) = \left| \bigcup_{K \in S} X_K \right|$$

یعنی به ازای هر اینکس در S، X مربوطه آن اینکس را در نظر بگیرید، اندازه اجتماع X_K های مختلف می‌شود f(S).

← f(S) ای که به این شکل تعریف می‌شود submodular است



با توجه به شکل بعد، u اضافه شده به S و T.

X_u هم X_S و هم X_T، union می‌شود. هرعنصری که به این ترتیب به X_T اضافه شود حتما به X_S هم اضافه می‌شود.

نتیجه: marginal gain f در S ≤ marginal gain T

Scanned with CamScanner

همانطور که گفتیم برای محاسبه $f(S)$ چند تا realization برای گراف احتمالی مان تولید می‌کنیم و بعد روی هرکدام از این گراف‌ها $f_i(S)$ را حساب می‌کنیم و

$$f(S) = \sum f_i(S)$$

$f_i(S)$ ← اندازه انتشار (cascade) وقتی که مجموعه آغازین ما S باشد و روی i امین realization

در حقیقت داریم: $f_i(S) = |U_{v \in S} X_v^i|$

میایم تک‌تک اینفلوئنس‌ها یا نودهایی که در S هستند را در نظر می‌گیریم، به ازای هر اینفلوئنس یا نودی که در S هست (X_v)، میایم X_v در i امین realization را در نظر می‌گیریم و این‌ها را با هم union می‌کنیم و در نهایت اندازه آن را حساب می‌کنیم که $f_i(S)$ است. همانطور که نشان دادیم پس $f_i(S)$ submodular است.

$f(S)$ هم یک ترکیب خطی از f_i هاست به طوری که $c_i = 1$

$$f(S) = \sum_{i \in I} f_i(S)$$

در نتیجه $f(S)$ هم طبق ویژگی اول، submodular است

نحوه‌ی اجرای الگوریتم:

میایم چند تا realization برای گراف ورودی ایجاد می‌کنیم، بعد روی تک تک آن‌ها، برای هرکدام از نودها یا هر زیرمجموعه از نودهایی می‌توانیم یک gain تعریف کنیم.

مثلا $f(a)$ را روی گراف‌های اول، دوم، سوم و ... حساب می‌کنیم

بعد $f(b)$ را " " " " " " " " و همینطور الی آخر.

اون نودی که بیشترین جمع f روی داشته باشه به عنوان نود شروع انتخاب می‌کنیم.

در واقع در هر iteration، نودی را انتخاب می‌کنیم که حاصل جمع marginal gain اش در realization ها مختلف max باشد.

روز کتاب ، کتاب خوانی و کتابدار

Scanned with CamScanner

فرض کنید i-1 تا نود را انتخاب کرده‌ایم و حالا نود i ام را می‌خواهیم انتخاب کنیم. میانگین برای تک تک نودهای باقی مانده، حساب می‌کنیم که دست تک realization ها چقدر marginal gain تولید می‌کند، آن‌ها را با هم جمع می‌کنیم و در نهایت از نودی که جمع marginal gain هاش در realization های مختلف از همه بیشتر بود انتخاب می‌کنیم و به S اضافه می‌کنیم

اثبات باند الگوریتم:

می‌خواهیم اثبات کنیم در حالت کلی اگر f ما submodular باشد، monotone و S هم متوسط یک الگوریتم greedy انتخاب شود، در این صورت داریم: $f(S) \geq (1-\frac{1}{e}) \cdot f(OPT)$

فرض کنیم $OPT=\{t_1, \dots, t_K\}$ ← جواب بهینه

اثبات ما برای K تا کام است

گام اول: فرض کنید یک مجموعه A و یک مجموعه B داریم. gain ای که B با افتاده شدنش به A به ما می‌دهد اگر بخواهد همیشه کوچکترمساوی gain ای که تک تک اعضای B به ما می‌دهند

$$f(A \cup B) - f(A) \leq \sum_{j=1}^{k} [f(A \cup \{b_j\}) - f(A)]$$

فرض: $B = \{b_1, b_2, \dots, b_k\}$

اثبات:

تعریف می‌کنیم: $B_i = \{b_1, b_2, \dots, b_i\}$ بنابراین $B_1, B_2, \dots, B_k$ را داریم

$$f(A \cup B) - f(A) = \sum_{i=1}^{K} [f(A \cup B_i) - f(A \cup B_{i-1})]$$

$$= \sum_{i=1}^{k} [f(A \cup B_{i-1} \cup \{b_i\}) - f(A \cup B_{i-1})]$$

$$\leq \sum_{i=1}^{k} [f(A \cup \{b_i\}) - f(A)]$$

$f(A \cup B_1) - f(A \cup B_0)$
$+ f(A \cup B_2) - f(A \cup B_1)$
$+ f(A \cup B_3) - f(A \cup B_2) \dots$
$+ f(A \cup B_K) - f(A \cup B_{K-1})$

وفات حضرت معصومه (س) (۲۰۱ هـ ق)

Scanned with CamScanner

۱۴۰۰

گام دوم: میاریم حدپایین (حداقل مقدار) برای marginal gain ای که در $i+1$ امین iteration پیدا می‌کنیم، به ما می‌دهد.

$$\delta_{i+1} \geq \frac{1}{k}\left[f(OPT) - f(S_i)\right]$$

$$\delta_i = f(S_i) - f(S_{i-1})$$

ما در هر iteration میایم نودی را انتخاب می‌کنیم که از بین نودهای باقی مانده بیشترین gain را به ما می‌دهد و حریصانه عمل می‌کنیم. میزان gain ای که به دست می‌آوریم یک حدپایین دارد.

اثبات:

$$f(OPT) \leq f(S_i \cup OPT) \rightarrow \text{به دلیل خاصیت monotone بودن}$$

$$= \underline{f(S_i \cup OPT) - f(S_i)} + f(S_i)$$

$$\leq \sum_{j=1}^{k}\left[f(S_i \cup \{t_j\}) - f(S_i)\right] + f(S_i) \rightarrow \text{به دلیل آنچه در گام اول اثبات کردیم}$$

$$\leq \sum_{j=1}^{k}\left[\delta_{i+1}\right] + f(S_i)$$

δ_{i+1} حداکثر gain ای است که در $i+1$ امین تکرار به دست می‌آوریم بنابراین gain ای که عنصر t_j از OPT به ما می‌دهد حتما کوچکتر مساوی δ_{i+1} است

$$= f(S_i) + k\delta_{i+1} \Rightarrow f(OPT) \leq f(S_i) + k\delta_{i+1}$$

$$\Rightarrow \quad \delta_{i+1} \geq \frac{1}{k}\left[f(OPT) - f(S_i)\right]$$

گام سوم: در این گام از حدپایینی که برای δ_{i+1} در گام قبلی به دست آوردیم استفاده می‌کنیم و یک حدپایین برای $f(S_{i+1})$ پیدا می‌کنیم:

$$f(S_{i+1}) = f(S_i) + \delta_{i+1} \geq f(S_i) + \frac{1}{k}\left[f(OPT) - f(S_i)\right]$$

$$\Rightarrow \quad f(S_{i+1}) \geq \left(1 - \frac{1}{k}\right) f(S_i) + \frac{1}{k} f(OPT)$$

Scanned with CamScanner

گام چهارم: در این گام یک حد اقل مقدار برای $f(S_k)$ پیدا می‌کنیم. (S_k همان S نهایی است)

در گام قبلی یک رابطه بازگشتی به دست آمد که $f(S_{i+1})$ را برحسب $f(S_i)$ نشان می‌دهد

ادعا می‌کنیم این رابطه بازگشتی به این صورت حل می‌شود:

$$f(S_i) \geq \left[1-\left(1-\frac{1}{k}\right)^i\right] f(OPT)$$

اثبات با استقرا:

پایه استقرا ←

$$i=0 \qquad f(S_0)=f(\{\})=0$$

$$\left[1-\left(1-\frac{1}{k}\right)^0\right] f(OPT)=0$$

فرض می‌کنیم برای i برقرار است، می‌خواهیم نشان دهیم برای $i+1$ هم برقرار است.

$$f(S_{i+1}) \geq \left(1-\frac{1}{k}\right) f(S_i) + \frac{1}{k} f(OPT)$$

$$\geq \left(1-\frac{1}{k}\right)\left[1-\left(1-\frac{1}{k}\right)^i\right] f(OPT) + \frac{1}{k} f(OPT) = \left[1-\left(1-\frac{1}{k}\right)^{i+1}\right] f(OPT) \checkmark$$

بنابراین قضیه اثبات می‌شود

حالا اگر به جای i، k قرار دهیم داریم:

$$f(S)=f(S_k) \geq \left[1-\underbrace{\left(1-\frac{1}{k}\right)^k}_{\leq 1/e}\right] f(OPT)$$

$$\boxed{f(S_k) \geq \left(1-\frac{1}{e}\right) f(OPT)} \qquad f(S) \geq 0.63\, f(OPT)$$

این بازی که به دست آمد data independent است یعنی به بیتای شما بستگی ندارد، به خود S بستگی ندارد و به گراف شما هم بستگی ندارد. همیشه یک مقدار ثابت است

در عمل معمولا خیلی بهتر از این حدپایین کار می‌کند

۲۸ | آبان | جمعه
19 Nov ■ ۱۳ ربیع الثانی

Scanned with CamScanner

جلسه هشتم - ۱۴۰۰/۷/۲۵

گفتیم که الگوریتم greedy مسئله IM را در زمان چندجمله‌ای حل می‌کند و یک حد پایین هم برای آن پیدا کردیم. البته این حد برای هر مسئله‌ای که f (تابعی که امتیازی دهد به مجموعه نودها) submodular و monotone باشد قابل استفاده است.

بررسی عملی این الگوریتم:

افرادی که این الگوریتم را پیشنهاد دادند به سری تست روی شبکه‌ی همکاری علمی در فیلد فیزیک اقدام کردند. نودها افرادی هستند و اگر با هم همکاری داشته باشند بین شان یال وجود دارد.

انواع cascade ها را می‌توانیم در این شبکه داشته باشیم مثلا اینکه یک Term یا اصطلاح از چه کسانی شروع شده و چگونه گسترش پیدا می‌کند یا یک تکنیک چگونه گسترش پیدا کرده.

فرض می‌کنیم cascade های مختلف از مدل آبشاری مستقل پیروی می‌کند و دو حالت را در نظر می‌گیریم:

① به هر یال یک احتمال ثابت p می‌دهیم ② به یالی که از v به w وجود دارد احتمال $\frac{1}{deg(w)}$ می‌دهیم

۱۰,۰۰۰ تا realization می‌سازد و متد حریصانه را با ۳ تا معیار اکتشافی دیگر مقایسه می‌کنیم:

① k تا یال را با نودهای با درجه بالا پر کنیم. k تا نودی که بالاترین درجه را دارند انتخاب کنیم.

② استفاده از closeness centrality برای هر نود فاصله متوسط اون نود از بقیه نودها را حساب کنیم، k تا نودی که کمترین فاصله متوسط از بقیه را دارند در S قرار بدهیم.

③ k تا نود تصادفی

در هر دو setting نتایج شان می‌دهد که greedy بهتر از همه عمل می‌کند بعد درجه بعد closeness و در آخر هم random.

روش greedy از نظر کیفیت خوبه اما از نظر efficiency مشکل داره و مشکل اصلی اش اینه که کنده.

Scanned with CamScanner

time Complexity روش greedy:

$O(k.n.R.M)$ ← تعداد نودهایی که قرار انتخاب کنیم

تعداد realization ها

n ← تعداد نودها، داخل هر iteration برای هر نود marigindl gain را حساب می‌کنیم

M ← تعداد realization ها (زمان که طول می‌کشد marigindl gain نود جدید را انتخاب کنیم)

(در این ها به تک تک نودها دقیق نگاه داشتیم، به نحوی که بهینه‌تر شده، چون نودها را که از همسایه‌هایشان bfs می‌زنیم و این multisource bfs زمان می‌برد $O(M)$)

مسئله‌ی دوم: Outbreak Detection (کشف شیوع)

یک گراف به ما داده شده، که این گراف شبکه‌ای را مدل می‌کند که شیوع‌های مادر آن رخ می‌دهد

یک سری اطلاعات مربوط به شیوع‌های مختلف داریم مثلا این شیوع می‌تواند آنلوگرس باشد (در مورد هر شیوع یا یک احتمالی رخ می‌دهد یک الگوی انتشار خاصی دارد.

مسئله ما اینه که ما تک تک نودها را نمی‌توانیم در نظر بگیریم، تعدادی محدودی سنسور در دست داریم و می‌خواهیم تعدادی از نودها را انتخاب کنیم تا در آن‌ها سنسور قرار بدهیم که به بهترین شکل شیوع‌ها را کشف کنیم

در حداقل زمان با کمترین تأثیر

در فیلترینگ کاربرد دارد

در بلاگ کاردار، بلاگ‌ها نودها و هایپرلینک‌ها یال هستند. پست‌های یکسیر را بازنشر می‌دهند و به این ترتیب یک cascade یا زنجیره‌ای از بازنشرها رخ می‌دهد. سوال اینه که کدام وبلاگ‌ها را بخوانیم که در کمترین زمان از پست‌های بیشتری باخبر شویم.

فرمول مسئله:

① گراف: Graph $G(V, E)$

② تعدادی outbreak و اطلاعات مربوط به آن‌ها (این outbreak ها احتمالی هستند و الگوی خاصی برای انتشار دارند)

هدف: انتخاب یک مجموعه‌ای از نودهای گراف به طوری که $f(S)$ ، max شود

Scanned with CamScanner

$$\max_{S \subseteq V} f(S) = \sum_i \underbrace{P(i)}_{\text{احتمال outbreak}} \underbrace{f_i(S)}_{\text{outbreak اگر}}$$

حایزه‌ای که S به دست می‌آورد اگر outbreak i ام را کشف کند

یک constrain هم داریم: $\text{subject to: } \text{Cost}(S) \leq B$

هزینه از بودجه زیادتر نشود ← فرق هم این مسئله با مسئله قبلی این constrain است. در حالت قبل k تا نود را انتخاب می‌کردیم اما اینجا k تا (تعداد S) برایمان مهم نیست دقت کنید Cost(S) است.

تعریف‌های متفاوتی برای $f_i(S)$ می‌توان داشت:

① مثلا زمان کشف ← هرچقدر یک مجموعه S در زمان کوتاهتری یک outbreak را کشف کند جایزه بیشتری دارد

پس یک تعریف برای $f_i(S)$ می‌شود منفی زمان کشف outbreak i ام توسط مجموعه S.

زمان کشف اولین زمانی هست که outbreak ما به یکی از نودهای داخل S می‌رسد

② تعریف دوم: تعداد افراد یا تعداد نودهای آلوده شده، هرچقدر کمتر باشد بهتر است

پس جایزه به صورت منفی تعداد نودهای آلوده شده تا زمان کشف outbreak توسط S

③ معیار سوم: کشف یا عدم کشف یا تعداد outbreak هایی که کشف می‌کند. (جمع وزن‌دار تعداد outbreak هایی که کشف می‌کند)

اثبات submodularity

تعریف از $f_i(S)$ را در نظر می‌گیریم که بازهم بود می‌خواهیم نشان بدهیم این تعریف submodular است

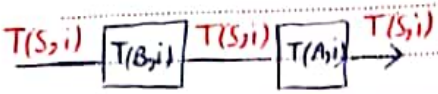
مجموعه A و B را داریم که $A \subseteq B \subseteq V$، فرض کنید S نودی است که نه در A و نه در B است،

اضافه کردن S به A marginal gain بیشتری از اضافه کردن S به B به ما می‌دهد:

$$f(A \cup \{S\}) - f(A) \geq f(B \cup \{S\}) - f(B)$$

Scanned with CamScanner

یک outbreak مثل i را در نظر بگیرید. $f_i(A)$ و $f_i(B)$ به صورت منفی اولین زمانی که outbreak توسط هر یک از A و B تشخیص داده شده تعریف می‌شود. می‌دانیم که $T(B,i) < T(A,i)$



برای S سه حالت وجود دارد: ① قبل از $T(A,i)$ کشف کند ② بین $T(A,i)$ و $T(B,i)$ کشف کند ③ بعد از $T(B,i)$ کشف کند.

حالت ۳: $T(B,i) \leq T(A,i) \leq T(S,i)$

$f_i(A \cup \{S\}) - f_i(A) = -T_a - (-T_a) = 0$

$f_i(B \cup \{S\}) - f_i(B) = -T_b - (-T_b) = 0$

$0 \geq 0$ ✓

حالت ۲: $T(B,i) \leq T(S,i) \leq T(A,i)$

$f_i(A \cup \{S\}) - f_i(A) = -T_S - (-T_a) = -T_S + T_a \geq 0$ چون: $T_a \geq T_S$

$f_i(B \cup \{S\}) - f_i(B) = -T_b - (-T_b) = 0$

$T_a - T_S \geq 0$ ✓

حالت ۱: $T(S,i) \leq T(B,i) \leq T(A,i)$

$f_i(A \cup \{S\}) - f_i(A) = -T_S - (-T_a) = T_a - T_S$

$f_i(B \cup \{S\}) - f_i(B) = -T_S - (-T_b) = T_b - T_S$

$T_a - T_S \geq T_b - T_S$ ✓

چون $f(S)$، submodular است می‌توانیم از الگوریتم greedy استفاده کنیم اما ۲ تا مشکل داریم:

① ما یک Constrain داریم. برای هر نود یک cost ای داریم و در نهایت نودهایی که انتخاب می‌شوند حاصل جمع cost شان از بودجه ما نباید بالاتر شود.

② الگوریتم greedy ما ممکن است hill climbing شود. چگونه می‌توانیم آن را بهینه کنیم؟

Scanned with CamScanner

حل مشکل اول:

یک راه حل ساده این است که در هر مرحله انتخاب بودجه ها اصلا هزینه را در نظر نگیریم و فقط انتخاب بودجه را تا جایی ادامه بدهیم که از بودجه ما بیشتر نشود.

این روش تا حدی که خواهیم بعدا می کند! با یک مثال این مورد را بیان می دهیم:

مثال: فرض کنید n تا سنسور داریم، budget: B

S_1: reward یا جایزه‌ای که می گیریم: r ، هزینه بودجه ← B

اگر بودجه ۱ را انتخاب کنیم

$S_2, S_3, \dots, S_n$ ← reward: $r-\varepsilon$, $C(S_i)=1$

اگر در هر مرحله انتخاب بودجه فقط به marginal gain توجه کنیم و cost را در نظر نگیریم، بودجه را انتخاب می کند که بیشترین reward را به ما می دهد یعنی S_1، بعد از انتخاب S_1 در iteration اول هزینه آن را از B کم کنیم و می شود و همین جا الگوریتم متوقف می شود و reward هایی که به دست می آوردیم می شود r.

ولی یک حالت بهتر این بود که B تا از بودجه های دوم تا n ام را انتخاب کنیم، جایزه‌ای که به دست می آوریم برابر $B(r-\varepsilon)$ است. اگر ε را تا حد دلخواه کوچک و B را تا حد دلخواه بزرگ در نظر بگیریم جایزه‌ای که در حالت دوم به دست می آوریم تا حد دلخواه بزرگتر از حالت اول می شود.

یک الگوریتم greedy دیگر این است که نسبت reward به هزینه برای هر بودجه را در نظر بگیریم. در هر iteration بودجه را انتخاب کنیم که این نسبت برای آن max باشد.

$$S_i = \arg\max_{s \in V} \frac{f(A_{i-1} \cup \{s\}) - f(A_{i-1})}{C(s)}$$

اما می توان مثال هایی پیدا کرد که این روش هم تا حد دلخواه بد عمل کند.

Scanned with CamScanner

آذر | ۴ | 25 Nov 2021
پنجشنبه | ۱۹ ربیع الثانی ۱۴۴۳

مثال: فرض کنید ۲ تا سنسور می توانیم انتخاب کنیم:

S_1: $C(S_1)=\varepsilon$, $f(S_1)=2\varepsilon$

S_2: $C(S_2)=B$, $f(S_2)=B$

الگوریتم greedy سنسور 1 را انتخاب می کند زیرا $\frac{B}{B}=1 < \frac{2\varepsilon}{\varepsilon}=2$ و بعد از آن می تواند

جلوتر بزند چون هزینه $B+\varepsilon$ می شود. حایزه کمایی هم 2ε می شود.

اما حالت بهتر این است که S_2 را انتخاب کنیم و جایزه B بشود، اگر ε را تا حد دلخواه کوچک یا B را به اندازه دلخواه بزرگ در نظر بگیریم در اینصورت تا هر حد دلخواهی این الگوریتم بد عمل می کند. پس در این حالت هیچ bound وصلی برای کیفیت الگوریتم نداریم

راه حل: الگوریتم CELF

در این الگوریتم هر دو تا معیاری که گفتیم را استفاده می کنیم. یعنی یک بار یک الگوریتم greedy اجرا کنید

به طوری که معیار انتخاب شما فقط mariginal gain ای باشد که

آذر | ۵ | جمعه
26 Nov ■ ۲۰ ربیع الثانی

هر بود دارد و هزینه (cost) را در نظر نگیریم و فقط در انتهای هر iteration هزینه را از بودجه کم کنیم تا بیشتر از بودجه نشود. در اثر اجرای این الگوریتم یک مجموعه S به دست می آید.

سپس از معیار دوم استفاده می کنیم: مجددا الگوریتم Greedy را اجرا می کنیم اما این بار معیار انتخاب reward (mariginal Gain) به هزینه (cost) باشد. یعنی علاوه بر اینکه هرگاه یک بود انتخاب شد هزینه اش را از بودجه کم می کنیم، در انتخاب بود هم هزینه را دخالت می دهیم. نتیجه ی اجرای این روش مجموعه S' به دست می آید.

روز بسیج مستضعفان

Scanned with CamScanner

۱۴۰۰

در نهایت برای S' و S'' مقدار f آنشان را مقایسه کنیم، هرکدام f بزرگتری داشت آن جواب است. (S' یا S'')

Final solution:

$$S = \arg\max\left(f(S'), f(S'')\right)$$

این روش همیشه یک باندی دارد اینطوریست که به اندازه دل خواهی بد عمل کند. کیفیت آن حداقل این دارد که $\frac{1}{2}(1-\frac{1}{e})$ است و از این بدتر عمل نمی کند. یعنی اون S ای که با این الگوریتم به دست می آید f آن نسبت به f(OPT) تا حد دلخواهی پایین نیست. حداقل $\frac{1}{2} \times 0.63$ هست.

حل مشکل دوم: سرعت الگوریتم greedy بالا ببریم

ما باید در هر iteration در الگوریتم greedy بیایم marigin gain را برای تک تک نودها حساب کنیم و این نصانبر است.

یک نود دلخواه u را در نظر بگیرید. فرض کنید در iteration های مختلف انتخاب نشده است.

marginal gain ای که نود u نسبت به مجموعه های مختلف از نودهای انتخاب شده مثلا $S_1, S_2, \dots$ و به طور کلی S_i دارد، نزولی (غیر صعودی) است.

علت: فرض کنید i و j شماره دو تا iteration هستند که $j > i$ و نود u در هیچ کدام از این iteration ها انتخاب نشده است.

marginal gain نود u در iteration = i : $f(S_i \cup \{u\}) - f(S_i)$

" " " " = j : $f(S_j \cup \{u\}) - f(S_j)$

S_i زیرمجموعه S_j است چون هرچقدر جلوتر برویم به مجموعه قبلی مقداری نود اضافه می شود. از طرف

Scanned with CamScanner

آذر | ۷ | 28 Nov 2021 | ۲۲ ربیع الثانی ۱۴۴۳ | یکشنبه

تابع f Submodular است. بنابراین داریم:

$$f(S_i \cup \{u\}) - f(S_i) \geq f(S_j \cup \{u\}) - f(S_j)$$

بنابراین نتیجه می‌گیریم هرچقدر iteration ها را جلوتر رویم marginal gain نودها نسبت به S کوچکتر می‌شود.

از این نتیجه‌گیری می‌توانیم به این صورت استفاده کنیم:

نودی مثل v داشته باشیم و marginal gain آن را در iteration i حساب کنیم و مشاهده می‌کنیم که مقدار آن از marginal gain نودهای دیگر در iteration قبلی بیشتر است. در این صورت می‌توانیم نتیجه بگیریم که marginal Gain نود v در iteration جدید نیز مقدارش از بقیه بیشتر است چون هرچه iteration جلوتر رود مقدار marginal gain نودها کاهش پیدا می‌کند یا ثابت می‌ماند. بنابراین بدون اینکه برای نودهای دیگر (غیر از v) در iteration جدید مقدار marginal gain را حساب کنیم مستقیماً نود v را به خروجی می‌فرستیم.

در عمل چه اتفاقی می‌افتد؟

فرض کنید که تا iteration i ام جلو رفته‌ایم و مجموعه‌مان را انتخاب کرده‌ایم و حالا در iteration i+1 ام هستیم و می‌خواهیم نود جدید را انتخاب کنیم. تا تکرار i+1 ام هر کدام از نودها یک marginal gain داشتند که بعضی از این marginal gain ها مربوط به تکرار i+1 ام یا تکرارهای قبلی باشد. نودها را از ابتدا تا تکرار i+1 ام به صورت مرتب شده نگه داشته‌ایم. (به صورت نزولی) در تکرار i+1 ام نقطه‌ای که بالای لیست قرار دارد را انتخاب می‌کنیم و برای این نود marginal gain جدید

Scanned with CamScanner

marginal gain ها را حساب می‌کنیم. اگر marginal gain این نود از بقیه‌ی marginal gain ها بزرگتر بود اصلا نیازی به حدس زدن نیست و همین نود را انتخاب می‌کنیم.

اگر این اتفاق نیفتاد یعنی یک نودی بود که marginal gain اش در شمارش قبلی بیشتر از marginal gain این نود جدید بود، میایم این نود جدید را در لیست مرتبمان در جای مناسبش قرار می‌دهیم. بعد میایم برای عنصر دوم در لیست قبلی که حال شده عنصر اول، marginal gain جدیدش را حساب می‌کنیم و با marginal gain تک تک نودهای دیگر مقایسه می‌کنیم. اگر بزرگ‌ترین بود همین نود را انتخاب می‌کنیم و اگر نبود آن را در جای مناسبش قرار می‌دهیم و نود بعدی را برمیداریم و همین روند را تکرار می‌کنیم.

در بدترین حالت یک دور تا انتهای لیست می‌رویم و برای تک تک نودها marginal gain حساب می‌کنیم که مثل الگوریتم greedy می‌شود اما این اتفاق در عمل نمی‌افتد.

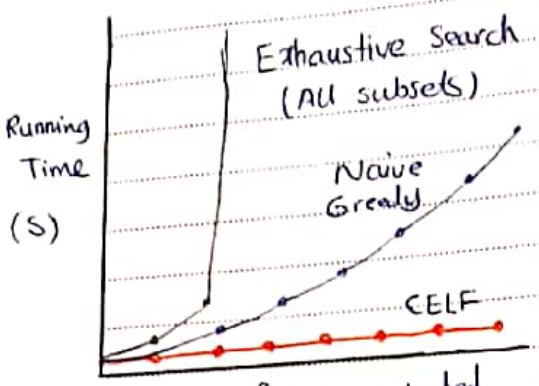
در عمل معمولا چند تا نود اول را که marginal gain شان را انتخاب بکنیم بقیه شان marginal gain اش از بقیه نودها بالاتر خواهد شد و به خروجی فرستاده می‌شود.

این الگوریتم lazy است. (lazy Hill climbing)

lazy evaluation یعنی در محاسبه‌ی marginal gain ها تنبلانه عمل می‌کنیم.

Scanned with CamScanner

جلسه ششم ـ ۱۴۰۰/۷/۲۷



با توجه به این نمودار الگوریتم CELF که خط بازی است بهترین الگوریتم greedy است.

همانطور که np hard راه‌حل مسئله را نشان می‌دهد.

Data Dependent Bound

فرض کنید S یک مجموعه از نودها هست که به‌عنوان راه‌حل پیدا شده، تفاوتی ندارد که با چه الگوریتمی این S پیدا شده است. جواب optimal: $OPT=\{t_1,\dots,t_k\}$ هم مونوتون و submodular است و $|S|\le K$

برای تک تک نودهای گراف mariginal gain اش نسبت به S را حساب می‌کنیم

$$\delta(u)=f(S\cup\{u\})-f(S)$$

بدین mariginal gain ها sort کنیم به گونه‌ای که اون که بیشترین MG را دارد نود ۱، نود ۲ و همین بیشترین MG را دارد و ...

$$\delta(1)\ge\delta(2)\ge\dots$$

$$f(OPT)\le f(S)+\sum_{i=1}^{k}\delta(i)$$

می‌خواهیم نشان دهیم چنین باندی داریم:
بین تفاوت $f(S)$ و $f(OPT)$ حداکثر چند برابر مجموع MG مربوط به k تا نود اول است.
همانطور که مشاهده می‌شود این bound، additive است درحالی که bound قبلی multiplictive بود. (۰٫۶۳ داشت ضرب می‌شد). ولی با باند جدید (تفاوت $f(S)$ و $f(OPT)$) به‌دنبای جمع استگی دارد. حسن این باند این است که به هیچ الگوریتم بستگی ندارد. کاهش حداکثر از ۰٫۶۳ دگاهی هم بدتر عمل می‌کند.

اثبات:

$$f(OPT)\le f(OPT\cup S)$$ ← چون f مونوتن است

$$=f(S)+\sum\left[f(S\cup\{t_1\dots t_i\})-f(S\cup\{t_1\dots t_{i-1}\})\right]$$ چون MG ای که روی t_i می‌دهد بیشتر از MG ای است که روی $S\cup\{t_1\dots t_{i-1}\}$ می‌دهد

$$\le f(S)+\sum_{i=1}^{k}\left[f(S\cup\{t_i\})-f(S)\right]$$

$$=f(S)+\sum_{i=1}^{k}\delta(t_i)\le f(S)+\sum_{i=1}^{k}\delta(i)$$

$$\Rightarrow f(T)\le f(S)+\sum_{i=1}^{k}\delta(i)$$

به جای نودهای k تایی که بیشترین MG را دارند فرار دادیم

Scanned with CamScanner

بررسی چند Case study:

۱) شبکه آب ← الگوریتم CELF برنده شد

وقتی تابع f را تغییری هم جایگذاری سنسورها خیلی تغییری کند

2) شبکه weblog ها: ما دقیقه زمان داریم، کدام وبلاگ ها را بخوانیم؟

ما در تئوری در مورد الگوریتم CELF گفتیم که ابتدا الگوریتم greedy را run می‌کنیم، یک بار وقتی که معیارها را انتخاب نمودیم فقط MG است و یک بار هست MG به هزینه (cost) را در نظر می‌گیریم. این کار را برای هر دو حالت می‌کنیم که ما در تئوری داشتیم.

ما در عمل با احتمال زیادی benefit/cost نتیجه بهتری می‌دهد.

می‌توانیم فقط benefit/cost را در نظر بگیریم و تا حد خوبی مطمئن باشیم که بهینه عمل می‌کند.

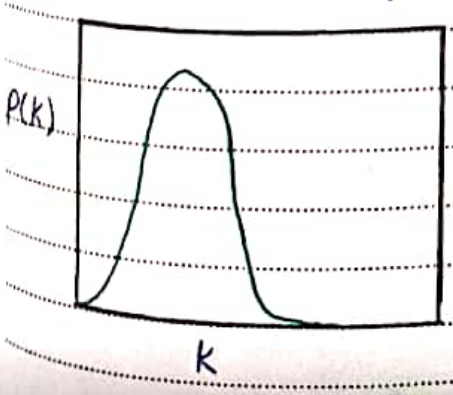
توزیع درجه توانی و اتصال ترجیحی

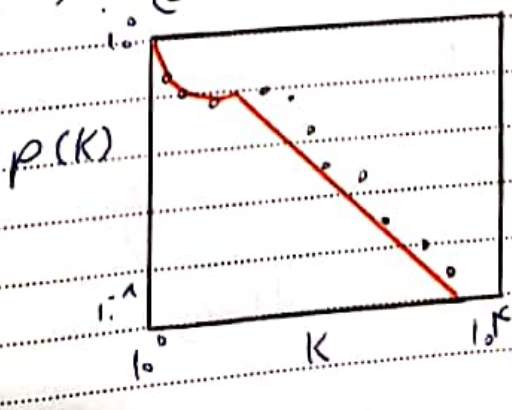
ما یک سری پدیده در شبکه‌های دنیای واقعی مشاهده می‌کنیم و بعد سعی می‌کنیم یک مدل ارائه دهیم که این پدیده‌ها را توضیح دهد. مثلا small world ← اینکه قطر شبکه‌های پیچیده دنیای واقعی $O(\log n)$ است

پدیده دیگری که مشاهده می‌شود ← توزیع درجه (در کنار ضریب خوشه‌بندی بالا)

توزیع توانی: $p(k) \propto k^{-\alpha}$ ← در شبکه‌های دنیای واقعی این مشاهده می‌شود

شکل جف توزیع درجه Gnp و سمت راست توزیع درجه گراف دنیای واقعی را داریم. (سمت راست در مقیاس log-log)





روز مجلس

Scanned with CamScanner

در گراف Gnp درجه اکثر نودها نزدیک به میانگین درجه است در نتیجه احتمال اینکه نودهای با درجه خیلی بالا داشته باشیم به صفر میل می‌کند در صورتی که گراف دنیای واقعی اینطور نیست. احتمال داشتن نودهای با درجه بالا کم است اما به صفر میل نمی‌کند. (تعداد نودهای با درجه بالا کم است ولی صفر نیست)

اگر نمودار توزیع درجه را در مقیاس log_log رسم کنیم یک خط با شیب $-\alpha$ می‌شود. این نشان می‌دهد که توزیع درجه power_law است نه exponential

$$y = f(x) = e^{-x} \rightarrow \log(y) = -x = -e^{\log x}$$

پس نمودارش برای توزیع نمایی در مقیاس log_log را ما خطی می‌بینیم

$$y = x^{-\alpha} \rightarrow \log(y) = -\alpha \log(x)$$

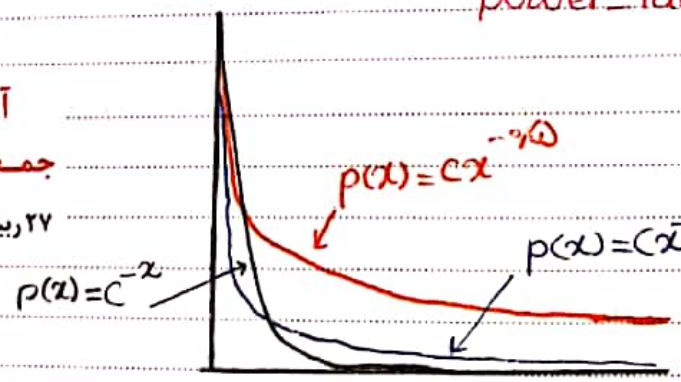
در این حالت $\log(y)$ یک تابع خطی برحسب $\log(x)$ است:

$$Y = -\alpha X$$

شیب خط توان power law است.

تقریباً همان چیزی است که در نمودار مشاهده می‌شود. به همین دلیل نتیجه می‌گیریم که توزیع درجه گراف‌های واقعی power law است

تفاوت توزیع نمایی و power_law

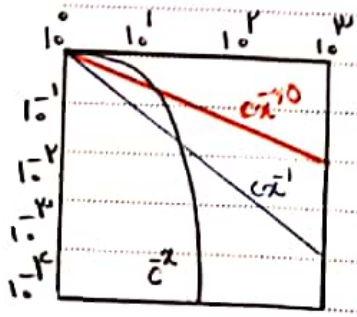


خط سیاه توزیع نمایی است و آبی دارای توزیع power law با α های مختلف است.

در توزیع نمایی دنباله درجه خیلی زود به سمت صفر میل می‌کند. (heavy tail)

اصطلاحا دنباله‌اش کوتاه است.

توزیع power_law خیلی کند به سمت صفر میل می‌کند. اصطلاحا دنباله‌اش بلند است (long tail)



در مقیاس log_log که رسم بگیریم تفاوت‌ها خیلی واضح می‌شود.

توزیع‌های power_law به صورت خطی در می‌آیند و خط مشکی هم حالت exponential دارد. (نمایی نزولی)

مثال توزیع نرمال و binomial توزیع‌های exponential هستند.

α در شبکه‌های پیچیده دنیای واقعی معمولا بین ۲ و ۳ است $2 < \alpha < 3$

Scanned with CamScanner

شبکه‌های مستقل از مقیاس (Scale_Free)

یک شبکه پیچیده که توزیع درجه‌اش power_law است مستقل از مقیاس نامیده می‌شود.

یک تابع (در فیزیک) Scale_Free گفته می‌شود اگر ما مقیاس x را عوض کنیم تغییری که در $f(x)$ مشاهده می‌شود این است که مقیاسش عوض می‌شود و تغییر دیگری مشاهده نمی‌کنیم.

$$f(x) = x^{\lambda} \Rightarrow f(ax) = a^{\lambda} x^{\lambda} = a^{\lambda} f(x)$$

مثال:

این ویژگی مشاهده می‌شود پس Scale_Free است. ← یعنی مقیاس x را عوض کردیم مقیاس y عوض شد.

ولی مثلا $\log(x)$ اینطور نیست و همینطور exponential:

$f(ax) = \log(ax) = \log(a) + \log(x) = \log(a) + f(x)$ → اگر به $f(x)$ شیفت داده شده، scale نشده

$f(ax) = \exp(ax) = \exp(x)^{a} = f(x)^{a}$ → یعنی مقیاس x را عوض کردیم $f(x)$ به توان رسید

توزیع Heavy Tail (دُم سنگین)

یک توزیع Heavy Tail است اگر در مقایسه با توزیع نمایی دُمش ضخیم‌تر باشد (یعنی کندتر از توزیع نمایی به صفر میل کند)

اما معیار مقایسه، توزیع نمایی است، (و ما برای مقایسه، xهای خیلی بزرگ را در نظر می‌گیریم یعنی برای xهای خیلی بزرگ باید مقدار تابع ما (که یک تابع احتمالی است) از توزیع نمایی بزرگتر باشد.

$X \leftarrow$ متغیر تصادفی

$x \to \infty$

x را به سمت ∞ میل می‌دهیم، احتمال اینکه توزیع تجمعی ما بزرگتر از x باشد نسبت به $e^{-\lambda x}$ باید به ∞ میل کند.

$$\lim_{x \to \infty} \frac{P(X > x)}{e^{-\lambda x}} = \infty$$

Scanned with CamScanner

چون می خواهیم رفتار توزیع را برای x های خیلی بزرگ بررسی کنیم x را به ∞ میل می دهیم.

صورت کسر نشان می دهد که توزیع p برای x های خیلی بزرگ چطور رفتار می کند و توزیع تجمعی اش به چه صورت است. مخرج کسر همان صورت کسر است وقتی که $p(x)$ برابر توزیع نمایی است یعنی داریم: $p(x) = \lambda e^{-\lambda x}$

وقتی که صورت کسر را با فرض اینکه $p(x)$ نمایی است حساب کنیم، به مخرج کسر می رسیم. علت اینکه مخرج کسر را این گونه گذاشته ایم این است که مبنای مقایسه ها، توزیع نمایی است.

مثال هایی از توزیع هایی که Heavy Tail نیستند:

① توزیع نرمال (Normal PDF):

$$p(x) = \frac{1}{\sqrt{2\pi}\sigma} e^{-(x-\mu)^2/2\sigma^2}$$

② توزیع نمایی (Exponential PDF):

$$p(x) = \lambda e^{-\lambda x}$$

$$P(X>x) = 1 - P(X \leq x) = e^{-\lambda x}$$

مثال هایی از توزیع Heavy Tail:

به طور کلی هر جا x در توان نباشد یا اگر در توان هست به گونه ای خنثی شده باشد می توان نتیجه گرفت توزیع Heavy Tail است

power law $p(x) \propto x^{-\alpha}$

power law with cutoff stretched exponential $\begin{cases} x^{-\alpha} e^{-\lambda x} \\ x^{\beta-1} e^{-\lambda x^\beta} \end{cases}$

long_normal $\frac{1}{x} \exp\left(-\frac{(\ln x - \mu)^2}{2\sigma^2}\right)$

گفتیم $p(x) \propto x^{-\alpha}$ پس نیاز به یک ضریب هست که مطلوبیت آن می گویند Normalization constant (ضریب نرمال سازی)

که متناسب بودن را به تساوی تبدیل کند.

Scanned with CamScanner

۱۴۰۰

چجوری از شرط توزیع احتمالی بودن $p(x)$ استفاده می‌کنیم یعنی انتگرال روی x های مختلف باید برابر با ۱ بشود:

$$p(x) = Z x^{-\alpha}$$

$$\int p(x)dx = 1 \rightarrow \int_{x_m}^{\infty} p(x)dx = Z\int_{x_m}^{\infty} x^{-\alpha}dx$$

$$= -\frac{Z}{\alpha-1}\left[x^{-\alpha+1}\right]_{x_m}^{\infty} = -\frac{Z}{\alpha-1}\left[\infty^{1-\alpha} - x_m^{1-\alpha}\right]$$

$$\Rightarrow Z = (\alpha-1)x_m^{\alpha-1} \quad \text{Need } \alpha>1 \Rightarrow \boxed{p(x) = \frac{\alpha-1}{x_m}\left(\frac{x}{x_m}\right)^{-\alpha}}$$

یادآوری:

$$\int (ax)^n = \frac{(ax)^{n+1}}{a(n+1)}$$

بررسی امید ریاضی توزیع درجه وقتی که توزیع درجه power law است:

$$E[X] = \int_{x_m}^{\infty} x p(x)dx = Z\int_{x_m}^{\infty} x^{-\alpha+1}dx = \frac{Z}{2-\alpha}\left[x^{2-\alpha}\right]_{x_m}^{\infty}$$

$$= \frac{(\alpha-1)x_m^{\alpha-1}}{-(\alpha-2)}\left[\infty^{2-\alpha} - x_m^{2-\alpha}\right] \quad \text{Need } \alpha>2 \Rightarrow \boxed{E[X] = \frac{\alpha-1}{\alpha-2}x_m}$$

امید ریاضی یا میانگین درجه

همین فرآیند را می‌توان برای محاسبه واریانس انجام داد و باید $\alpha>3$ باشد

محاسبه α:

α توان یا exponent برای توزیع توانی ما هست: $p(x) = Z x^{-\alpha}$

α را باید estimate کنیم. چیزی که ما در اختیار داریم یک توالی از درجه‌هاست که از انتگرال دنیای واقعی به دست آمده. فرض می‌کنیم توزیع درجه‌ها power law است و ما باید پارامتر α را تخمین بزنیم. مسئله ما یک مسئله تخمین پارامتر است. تکنیکی که زیاد استفاده می‌شود max log likelihood

Scanned with CamScanner

مایک دیتایی را دیده ایم که توزیعی دارد. می خواهیم پارامتر آن را حدس بزنیم. likelihood همان احتمال است ولی احتمال برای چیزی که مشاهده شده. ولی یک دیتایی هنوز مشاهده نشده معمولا بحث میکنن احتمال ولی اگر دیتا مشاهده شده به آن احتمال، likelihood می گویند.

این کلی maximum likelihood:

از این منطق استفاده می کند که اگر دیتایی مشاهده شده، حتما احتمال زیادی داشته. حالا که مشاهده شده باید likelihood آن max شود.

در این جا دیتایی که مشاهده شده دهم نوع ها هست: $d_1, d_2, \dots, d_n$

پس احتمال $d_n, \dots, d_2, d_1$ باید max شود: $p(d_1, d_2, d_3, \dots, d_n) \to max$

معمولا از یک فرض ساده سازی استفاده می کنیم: فرض استقلال ← اینکه $p(d_1)$ و $p(d_2)$ و ... انهم مستقل اند.

$$p(d_1, d_2, d_3, \dots, d_n) = p(d_1)\, p(d_2) \cdots p(d_n)$$

معمولا به جای maximize کردن این احتمال لگاریتم آن را max می کنیم چون ساده تر است و مشکلی هم به وجود نمی آورد. اگر تابع در یک نقطه max شود لگاریتم آن هم در همان نقطه max می شود.

پس داریم:

$$L(\alpha) = \ln\left(\prod_i^n p(d_i)\right) = \sum_i^n \ln p(d_i)$$

داشتیم: $p(x) = \frac{\alpha-1}{x_m}\left(\frac{x}{x_m}\right)^{-\alpha}$ ← به جای x، d_i قرار می دهیم: $p(d_i) = \frac{\alpha-1}{x_m}\left(\frac{d_i}{x_m}\right)^{-\alpha}$

بعد ln از روی آن اعمال می کنیم:

$$L(\alpha) = \sum_i^n \left(\ln(\alpha-1) - \ln(x_m) - \alpha \ln\left(\frac{d_i}{x_m}\right)\right)$$

برای یافتن max، برحسب α از آن مشتق می گیریم:

$$\frac{dL(\alpha)}{d\alpha} = 0 \Rightarrow \frac{n}{\alpha-1} - \sum_i^n \ln\left(\frac{d_i}{x_m}\right) = 0 \Rightarrow \boxed{\hat{\alpha} = 1 + n\left[\sum_i^n \ln\left(\frac{d_i}{x_m}\right)\right]^{-1}}$$

Scanned with CamScanner

توضیحات تکمیلی از جلسه قبل

آذر | ۱۷ | 8 Dec 2021
چهارشنبه | ۳ جمادی الاولی ۱۴۴۳

۱۴۰۰

جلسه دهم - ۱۴۰۰/۸/۴

اتصال ترجیحی ← هرجا اتصال ترجیحی داشته باشیم توزیع درجه ما power_law می‌شود

اتصال ترجیحی به این معناست که هرگاه نود جدیدی وارد شبکه می‌شود به صورت تصادفی به نودهای موجود متصل نشود بلکه به نودهای با درجه بالاتر با احتمال بیشتری متصل شود

یک مدل اتصال ترجیحی پایه: نودها در گام‌های زمانی گسسته وارد می‌شوند: 1, 2, 3, ... n ← نودی که در زمان i وارد شده باشد با i نشان می‌دهیم. حالا گام زمانی دلخواه را در نظر بگیرید، هر کدام از نودهایی که در گام‌های زمانی قبل از آن آمده اند درجه‌ای دارند ← d_i $(i<j)$ فرض کنیم هر نود جدیدی که وارد می‌شود قرار است m تا یال داشته باشد.

میایم برای هرکدام از m تا یال به این صورت عمل می‌کنیم: احتمال اینکه برای هرکدام از یال‌ها که مقصدشان نود i است) مقصدشان نود i باشد، این احتمال متناسب است با d_i

$$p(j \to i) = \frac{d_i}{\sum_k d_k}$$

پس این جا اتصال ترجیحی داریم و مقصد یال را مثل مدل ER تصادفی انتخاب می‌کنیم

در نتیجه پولدارتر می‌شود ← نود با درجه بالاتر، درجه‌اش می‌رود بالاتر. نود با درجه بیشتر احتمال اینکه یال جدید را بگیرد بیشتر است.

در خیلی جاها اتفاق می‌افتد مثلا در citation، احتمال اینکه مقاله‌ای که بیشتر cite شده را cite کنیم بیشتر است.

الان باید اثبات کنیم این مدل اتصال ترجیحی توزیع درجه power law به ما می‌دهد. برای این کار مدل را ساده‌تر می‌کنیم:

نودهای ما در time step های گسسته وارد می‌شوند: 1, 2, 3, ..., n

وقتی یک نود جدید وارد می‌شود: با احتمال p مقصدش به صورت تصادفی یکنواخت انتخاب می‌شود و با احتمال 1-p اتصال ترجیحی اتفاق می‌افتد و مقصدیال متناسب با درجه وزنی هرکدام از نودهای موجود انتخاب می‌شود

(دقیقا یک یال رسم می‌شود از نود وزنی جدید به سایر نودهای موجود)

Scanned with CamScanner

درجه خروجی هر نود دقیقاً ۱ است و گراف جهتدار است. هر نود فقط وقتی وارد می‌شود یک یال خروجی جدید اضافه می‌شود و دیگر اضافه نمی‌شود.

می‌خواهیم اثبات کنیم درجه ورودی هر نود که مکانیسم اتصال ترجیحی دارد، توزیع درجه power law پیدا می‌کند.

پس $d_i(t)$ را در طول زمان بررسی می‌کنیم:

نود i در زمان $t=i$ می‌رسد پس وقتی $t=i$: $d_i(t)=0$ (یال ورودی ندارد در ابتدا)

فرض کنید در زمان t ام هستیم و نود i ام یک درجه ورودی مانند $d_i(t)$ دارد (درجه ورودی نود i در زمان t)

حالا در زمان $t+1$ می‌شویم و می‌خواهیم ببینیم درجه ورودی نود i به چه صورت تغییر پیدا می‌کند. ۲ حالت داریم:

در زمان $t+1$ یک نود جدید وارد می‌شود و می‌خواهد مقصد یالش را انتخاب کند. با احتمال p مقصد یالش را به صورت تصادفی یکنواخت انتخاب می‌کند و نود i هم یکی از t تا نود موجود است. پس در این حالت با احتمال $p \times \frac{1}{t}$ نود i انتخاب می‌شود. در حالت احتمال ترجیحی که احتمالش $1-p$ است نود i ام متناسب با درجه‌اش انتخاب می‌شود

یعنی $\frac{d_i(t)}{\sum \text{درجه ورودی نودها}}$ ← حاصل جمع درجه‌های ورودی نودها در زمان t برابر با t

است چون در هر گام زمان فقط یک یال وارد شده. پس در حالت کلی:

$$p \times \frac{1}{t} + (1-p)\frac{d_i(t)}{t}$$

۱۹ | آذر | جمعه
10 Dec ■ ۵ جمادی الاولی

اگر t خیلی بزرگ باشد

$d_i(t)$ چگونه تغییر می‌کند؟ می‌خواهیم نشان بدهیم وقتی t به اندازه کافی بزرگ است احتمال اینکه درجه ورودی نود i برابر با k باشد متناسب است با $(q=1-p)$ $k^{-(1+\frac{1}{q})}$

$$P(d_i=k) \propto k^{-(1+\frac{1}{q})} \rightarrow \underset{(\alpha=1+1/q)}{\text{power_law}}$$

می‌خواهیم بررسی کنیم وقتی از گام زمان t به $t+1$ می‌رویم

میزان تغییرات درجه برای نود i چقدر است:

$$\underbrace{d_i(t+1)-d_i(t)}_{} = p\,\frac{1}{t} + (1-p)\frac{d_i(t)}{t}$$

$$\frac{d\,d_i(t)}{dt} = p\,\frac{1}{t} + (1-p)\frac{d_i(t)}{t} = \frac{p+q\,d_i(t)}{t} \qquad q=(1-p)$$

Scanned with CamScanner

$$\frac{1}{p+q\,d_i(t)}\,d\,d_i(t) = \frac{1}{t}\,dt$$

تقسیم بر $p+q\,d_i(t)$

$$\int \frac{1}{p+q\,d_i(t)}\,d\,d_i(t) = \int \frac{1}{t}\,dt$$

انتگرال گیری

$$\frac{1}{q}\ln(p+q\,d_i(t)) = \ln(t) + C$$

$A = e^C$ (ثابت)

$$p+q\,d_i(t) = e^C t^q \Rightarrow \boxed{d_i(t) = \frac{1}{q}(At^q - p)}$$

مقدار A را باید به دست بیاوریم. از شرط مرزی استفاده می کنیم:

$$d_i(i) = 0 \Rightarrow d_i(i) = \frac{1}{q}(Ai^q - p) = 0 \Rightarrow A = \frac{p}{i^q}$$

$$\boxed{d_i(t) = \frac{p}{q}\left(\left(\frac{t}{i}\right)^q - 1\right)}$$

از این عبارت نتیجه می گیریم که هر چقدر i کوچکتر باشد یعنی نود قدیمی تر باشد این عبارت بزرگتر می شود

نودهای قدیمی تر با i کوچکتر درجه بزرگتری دارند

حالا می خواهیم اینکه $P(k)$ (توزیع درجه) را حساب کنیم، توزیع تجمعی $F(k)$ را حساب می کنیم و از آن مشتق می گیریم تا $P(k)$ به دست بیاید.

$$d_i(t) = \frac{p}{q}\left(\left(\frac{t}{i}\right)^q - 1\right) < k \longrightarrow i > t\left(\frac{q}{p}k+1\right)^{-1/q}$$

$F(k)$ یعنی چه؟ یعنی چه نسبت تعداد نودهایی که درجه‌شان کوچکتر مساوی k است به کل نودها.

نودهایی که بزرگتر از $t\left(\frac{q}{p}k+1\right)^{-1/q}$ هستند درجه‌شان کوچکتر از k است. تعداد این نودها چند تاست؟

یک range بین 1 تا t داریم که به ازای i از 1 تا $t\left(\frac{q}{p}k+1\right)^{-1/q}$ درجه نودها بیشتر از k است و به ازای

Scanned with CamScanner

تعداد از $t_1, t_2, t_3 \dots t(\frac{q}{p}k+1)^{-1/q}$ درجه نودهای کوچکتر از k است. پس تعداد نودهایی که درجه‌شان کوچکتر از k است برابر است با: $t - t(\frac{q}{p}k+1)^{-1/q}$

و در کل نود k هم t است. وقتی تقسیم کنیم $F(k)$ حاصل می‌شود:

$$F(k) = 1 - \left[\frac{q}{p}k+1\right]^{-1/q}$$

$$F'(k) = p(k) = \frac{1}{p}\left[\frac{q}{p}k+1\right]^{-1-\frac{1}{q}} \Rightarrow \alpha = 1 + \frac{1}{1-p}$$

در نتیجه یک توزیع power law داریم و در آینده احتمال ترجیحی به ما توزیع درجه power law می‌دهد.

سوال: چرا فقط از احتمال ترجیحی استفاده نکردیم و با احتمال p تصادفی یکنواخت انتخاب کردیم؟ نقش پارامتر p چیه؟

اگر p صفر باشد توان توزیع نمایی همیشه ۲ می‌شود اما در شبکه‌های واقعی α های مختلف داریم. وقتی پارامتر p را استفاده می‌کنیم این قابلیت را پیدا می‌کنیم که بتوانیم توزیع‌های با α های متفاوت داشته باشیم.

مشکلات حالت پایه مدل احتمال ترجیحی:

یکی از عیب‌هاش اینه که Age_degree Correlation بالایی دارد. وابستگی سن - درجه بالاست، یعنی نودهای قدیمی درجه بالا دارند ولی در دنیای واقعی این پدیده را نداریم.

برای حل آن یک راه پیشنهاد شد: در فاز احتمال ترجیحی مقصد یال را متناسب با درجه انتخاب نمی‌کنیم، میایم از یک توزیع power law یک وزن تولید می‌کنیم و به نود جدید می‌دهیم. (وزن عددی بین ۰ و ۱ است) در فاز احتمال ترجیحی متناسب با این وزن‌ها یال را اتصال می‌دهیم.

مشکل دوم:

در مدل احتمال ترجیحی، دو نود که درجه خیلی زیادی دارند هم‌دیگر را می‌شوند. چون یکی از آن‌ها ابتدا آمده و قدیمی‌تر است و احتمال خیلی بالا به او یال می‌دهد ولی در دنیای واقعی این اتفاق نمی‌افتد. نودهای با درجه بالا معمولا هم‌دیگر را ندارند.

Scanned with CamScanner

یک مشکل همیشگی اینه که clustering coefficient آن خیلی پایین است. به صورت تئوری تا به حال نتوانسته اند ضریب خوشه بندی را برای آن به دست آورند اما کارهای عملی زیادی صورت گرفته و نشان داده که وقتی n خیلی زیاد می شود در مدل preferential attachment (اتصال ترجیحی)، clustering coefficient به سمت صفر میل می کند. درحالی که شبکه های پیچیده دنیای واقعی اینطور نیستند و برای ضریب خوشه بندی شان یک حد پایینی دارند و از آن کمتر نمی شود. هرچقدر اندازه گراف کوچک یا بزرگ باشد، ضریب خوشه بندی به سمت صفر میل نمی کند.

مکانیسم های preferential attachment

① مکانیسم کپی کردن ← یک نود را انتخاب می کنیم، یکی از یال های آن نود را به صورت تصادفی یکنواخت انتخاب می کنیم و نود جدید به مقصد آن یال اضافه می شود. نود جدید یک یال می دهد به مقصد آن یال انتخاب شده. این جا هم به جوری مکانیزم preferential attachment را داریم اجرا می کنیم به این صورت که اگر نودی درجه اش بالا باشد تعداد یال های زیادی دارد، در نتیجه مقصد تعداد یال های زیادی هست و با احتمال بالاتری انتخاب می شود.

② walking on a Network

یک نود تصادفی را انتخاب می کنیم و نود جدید را به این نود تصادفی متصل می کنیم. سپس همسایه های مرتبه اول این نود را می یابیم و نود جدید را به آن ها هم متصل می کنیم. بود برای همسایه های مرتبه دوم... این جا هم مکانیزم preferential attachment را داریم؛ هر نودی که درجه اش خیلی زیاد باشد، همسایه هایش خیلی زیاد است و با احتمال بالاتری جزو همسایه های مرتبه اول یا دوم یک نود تصادفی انتخاب شده قرار می گیرد و با احتمال بالاتری نود جدید بهش وصل می شود.

Scanned with CamScanner

Attaching to edges (۲)

یک یال را به صورت تصادفی یکنواخت انتخاب می‌کنیم. نود جدید را به این یال وصل می‌کنیم. مکانیزم preferential attachment ← نودی که درجه‌اش زیاده تعداد یال‌هایش هم زیاد است پس با احتمال بیشتری انتخاب می‌شود

Node duplication (۳)

یک نود را با همسایه‌هایش انتخاب می‌کنیم، نود جدید را به همه‌ی همسایه‌های این نود موجود وصل می‌کنیم با اینکه به هر همسایه‌ای با یک احتمال p متصل می‌کنیم.

این جا هم مکانیزم اتصال ترجیحی را داریم به این صورت: نودی که درجه بالایی داشته باشد احتمال اینکه به عنوان همسایه‌ی یک نود تصادفی انتخاب شده باشد بیشتر است. چون نودی که درجه بالایی دارد همسایه‌های بیشتری دارد.

Scale Free Network

همان طور که گفتیم α باید بین ۲ و ۳ باشد تا average تعریف شده باشد.

سوال: اگر α خیلی بزرگ باشد، (مثلا بزرگتر از ۳ باشد) چه اتفاقی می‌افتد؟ با افزایش α در توزیع درجه power_law چه اتفاقی می‌افتد؟

$$K^{-\alpha} = \left(\frac{1}{K}\right)^{\alpha}$$

$p(k)$ کوچک می‌شود.

اگر K بزرگ باشد $\left(\frac{1}{K}\right)^{\alpha}$ برای α بزرگ به صفر میل می‌کند. یعنی توزیع شبیه توزیع‌های خانواده‌ی نمایی (exponential) می‌شود و از توزیع شبکه‌های پیچیده دنیای واقعی فاصله می‌گیرد.

نتیجه: باید α را بین ۲ و ۳ انتخاب کنیم

Scanned with CamScanner

Link Analysis : page Rank and Hits

قبل از اینکه موتورهای جستجو به وجود بیایند چگونه ساختار محتوای وب را سازمان دهی می کردیم؟

از یک ابزار ساده ای به نام web directory یا web category استفاده می شد. یک دسته بندی کلی از سایت های وب داشت. هر category زیرشاخه هایی داشت تا اینکه در نهایت به یک وب سایت می رسید. خیلی سخت بود و هزینه زمانی زیادی داشت و در عین حال برای web با اندازه بزرگ اصلا جوابگو نبود. در نتیجه موضوع جستجوی وب به وجود آمد. موتورهای جستجو پیشنهاد شدند و فیلد information retrival به وجود آمد.

اما مشکلی که وجود داشت هم وب سایت ها معتبر و هم نامعتبر وجود داشتند و موتور جستجو مجموعه عظیمی از وب سایت های مرتبط با جست وجو را روش بازیابی اطلاعات پیدا می کردند. کدام یک از این ها را باید به کاربرشان می دادند؟

تا اینکه موضوع ranking یا رتبه بندی صفحات وب به وجود آمد. به هرکدام از صفحات وب یک رتبه (امتیاز) اختصاص می دهد و این صفحات برحسب امتیازشان به کاربرشان داده می شوند. ابتدا صفحات با امتیاز بالا بعد صفحات با امتیاز پایین.

در این صورت می توانند حتی کم خیلی زیادی از Document ها را prone کنند. آن هایی که رتبه و اهمیتشان پایین است را حذف کنند.

این الگوریتم ranking که وب را متحول کرد و وب بعد از آن گسترش یافت، الگوریتم page Rank بود.

Scanned with CamScanner

آذر | ۲۵ | 16 Dec 2021
پنجشنبه | ۱۱ جمادی الاولی ۱۴۴۳

چالش‌های web Search

① به چه کسی باید اعتماد کنیم؟

معمولا وب سایت‌های مرتبطی که پیدا می‌کنیم خیلی اندک و زیاد است. در نتیجه باید روشی بیابیم که این نحوه را کوچک کنیم به گونه‌ای که مفیدها باقی بمانند و غیرمفیدها حذف شوند.

یک insight ای که برای پاسخ به سوال "به چه کسی اعتماد کنیم" می‌توانیم داشته باشیم این است که صفحاتی که مطمئن هستند معمولا به یکدیگر اشاره می‌کنند.

② کاربر یک پرس و جو (query) دارد، مثلا "newspaper"، بهترین جواب کدام است؟

یک جواب می‌تواند این باشد: وب سایت‌هایی که expert هستند در مورد newspaper ها بگویند کدام جواب بهتر است یا در حقیقت point کنند به وب سایت‌هایی که حاوی جواب بهتر هستند.

روز پژوهش

آذر | ۲۶ | 17 Dec
جمعه | ۱۲ جمادی الاولی

در رده‌بندی نودها و همچنین اهمیت سایت‌های مختلف در وب diversity بالایی داریم

پس از این خاصیت می‌توانیم استفاده کنیم: ساختار لینک برای تعیین اهمیت صفحات استفاده می‌شود. اما در حوزه این ساختار لینک، همه لینک‌ها یک اندازه اهمیت ندارند. به طور کلی از ساختار لینک برای رتبه‌بندی نودها استفاده می‌کنیم و به گونه‌ای که همه‌ی لینک‌ها هزینه و تاثیر یکسان ندارند.

معیارهای Centrality ← اهمیت یک نود را در ارتباط با بقیه نودها تعیین می‌کند (یا یال در ارتباط با بقیه یال‌ها)

① رده ← رده بالاتر اهمیت بیشتر

① Betweenness: بستگی از shortest path هایی که از آن نود می‌گذرند. هرچه بیشتر باشد اهمیت نود بیشتر است

② Classness: فاصله هر نود با تک تک نودهای دیگر را اندازه بگیرید. نودی که این میانگین فاصله برایش کمتر باشد اهمیتش بیشتر است.
میانگین

③ Eigenvector: برای گراف یک ماتریس در نظر می‌گیریم. بعد یک بردار ویژه برای آن استخراج می‌کنیم. مقدار هر خانه در این بردار، اهمیت نود متناظر با آن را نشان می‌دهد.
متناظر با بزرگترین بردار ویژه

Scanned with CamScanner

جلسه یازدهم - ۱۴۰۰/۸/۹

الگوریتم Hits

رتبه‌بندی واهمیت صفحات براساس لینک‌ها (گراف ما جهتدار است)

↳ مثلا جواب query را خوبی دهند

هدف الگوریتم این است که نه تنها نودهایی که content provider خوبی هستند را پیدا کند و به آن‌ها نمره بالایی بدهد بلکه نودهایی را هم معرفی کند که در یک زمینه‌ای expert (متخصص) هستند

به هر نود ۲ تا score اختصاص می‌دهیم: ① expert score و Hub score

تعریف ← مجموع نمرات authority لینک‌های خروجی یک نود

یک نود Hub score بالایی دارد اگر اولا تعداد نودهای خروجی اش زیاد باشد و دوما به نودهایی که content provider خوبی هستند اشاره کند.

② Content provider و authority score

لینک‌های ورودی یک نود را در نظر می‌گیریم و برای هر لینک ورودی expert score و Hub score آن لینک ورودی را در نظر می‌گیریم. یک نود authority بالایی دارد یا content provider خوبی است اگر تعداد زیادی لینک با Hub score بالا به آن اشاره کنند.

ما در این جا بیشتر به Hub score و authority score براساس Hub score و همچنین authority score براساس Hub score تعریف می‌شود. یعنی تعریف بازگشتی داریم.

نودهایی که authority بالایی دارند content provider خوبی هستند و نودهایی که Hub score بالایی دارند معمولا expert خوبی هستند مثلا دانشگاه‌های معتبر را لیست می‌کنند یا روزنامه‌های مهم را لیست می‌کنند.

نودهایی که authority بالا دارند مثلا وب سایت یک روزنامه، دانشگاه و ...

Scanned with CamScanner

یک مزایده بازگشتی داریم ← یک Hub خوب باید به مقدار زیادی authority خوب اشاره کند و یک authority خوب باید مورد اشاره ی مقدار زیادی hub خوب قرار بگیرد

روی کاغذ الگوریتم Hits

به هر نود یک Hub score و یک Authority score اختصاص می دهیم. بردارهای H و A را داریم که به ترتیب نشان دهنده ی یک نود است و مقدار امتیاز Hub و Authority برای آن نود را نگه میدارد.

در ابتدا به A و H مقدار اولیه $\frac{1}{\sqrt{n}}$ به ازای هر عنصر اختصاص می دهیم بعد وارد فرآیند iterative می شویم.

یک شرط توقف در نظر در می گیریم. در هر iteration این فرض را داریم که مقادیر Hub و Authority نود ها در تکرار tام را داریم و می خواهیم این مقادیر را در تکرار $t+1$ام به دست آوریم.

- $\forall i : \text{Authority} : a_i^{(t+1)} = \sum_{j \to i} h_j^{(t)}$ مقدار authority به این صورت آپدیت می شود:
- $\forall i : \text{Hub} : h_i^{(t+1)} = \sum_{i \to j} a_j^{(t)}$
- $\forall i : \text{Normalize} : \quad \sum_i \left(a_i^{(t+1)}\right)^2 = 1 \,,\ \sum_j \left(h_j^{(t+1)}\right)^2 = 1$

شرط های توقف که می توانیم داشته باشیم: ① تعداد تکرار ها عدد ثابتی باشد (مثلا ۱۰ تا ۲۰ تا)

② تفاوت نود ها در تکرار $t+1$ام نسبت به تکرار tام خیلی زیاد نباشد: (ε را خودمان set می کنیم)

$$\sum_i \left(a_i^{(t)} - a_i^{(t+1)}\right)^2 < \varepsilon \qquad \sum_i \left(h_i^{(t)} - h_i^{(t+1)}\right)^2 < \varepsilon$$

طریقه نرمال سازی: باید کاری کنیم که بردارهای hub و authority که در تکرار $t+1$ام به دست آورده ایم نرم ۲ شان برابر با ۱ باشد. نرم 2 بردار برابر با جمع توان ۲ تک تک اعضای بردار است (کاما جزو هم گرفته می شود) در نرمال سازی هر مولفه ی بردار را تقسیم بر نرم 2 می کنیم. نتیجه این است که نرم 2 بردار برابر ۱ می شود.

Scanned with CamScanner

۱۴۰۰/

فایده رتبه‌سازی: باعث می‌شود بتوانیم بردارهای hub در تکرارهای مختلف را به‌راحتی بهم مقایسه کنیم (همین طور authority). این مقایسه را در شرط توقف نیاز داریم. اگر رتبه‌سازی انجام ندهیم score ها مرتباً زیاد می‌شوند و هیچ‌گاه همگرا نمی‌شوند. ما در رتبه‌سازی می‌کنیم طول بردار ثابت است و فقط چرخش بردار را در نظر می‌گیریم.

فرم برداری الگوریتم Hits:

Vector $a=(a_1 \dots a_n)$ $\quad h=(h_1 \dots h_n)$

Adjacency Matrix $A_{(n\times n)}$: $A_{ij}=1$ if $i \to j$

$\Rightarrow h_i = \sum_{i\to j} a_j$ $\quad$ $h_i = \sum_j A_{ij} \cdot a_j$

این رابطه می‌گوید همانا نام بردار h برابر است با ضرب داخلی سطر i ام ماتریس A در بردار a. پس داریم:

$h = A \cdot a$, $a = A^T \cdot h$

پس الگوریتم ما به این صورت برداری می‌شود:

$h^{(t+1)} = A \cdot a^{(t)}$

$a^{(t+1)} = A^T \cdot h^{(t)}$

Normalize $a^{(t+1)}$ and $h^{(t+1)}$

$a = A^T \cdot h$

$a = A^T \cdot (A \cdot a)$

(new h) (new a)

$a = A^T(Aa) = (A^T A)a$ ← a بعد از ۲ گام (t+1 و t-1) آپدیت می‌شود

$h = A(A^T h) = (AA^T)h$ ← h بعد از ۲ گام

به طور کلی می‌توان ۲k گام در نظر گرفت:

$a = (A^T \cdot A)^k \cdot a$

$h = (A \cdot A^T)^k \cdot h$

Scanned with CamScanner

eigenVector (بردار ویژه) برای ماتریس R: بردار x را یک بردار ویژه برای ماتریس R می‌گوییم اگر:

$R.x = \lambda x$

x نباید صفر باشد و λ می‌تواند هر عدد حقیقی باشد (صفر هم می‌تواند باشد)

به λ مقدار ویژه متناظر با بردار ویژه x می‌گوییم.

وقتی steady state داریم یعنی خرات دیگر تغییر پیدا نمی‌کند.

a در زمان t-1 و t+1 باهم برابر است

$A^T.A.a^{t-1} = c'.a^{t+1}$ ← به‌خاطر شرط نرمال‌سازی

این رابطه‌شان می‌دهد که a بردار ویژه A^TA است. بردار authority بردار ویژه‌ی متناظر با بزرگترین مقدار ویژه‌ی A^TA است

$A.A^T.h = c''.h$ ← دوتا h باهم برابر

این رابطه‌شان می‌دهد hub score بردار ویژه‌ی ماتریس AA^T است. اثبات می‌شود که متناظر با بزرگترین مقدار ویژه‌ی AA^T است.

برای محاسبه مقدار ویژه مستقلی نداریم به‌خصوص بردار ویژه‌ی متناظر با بزرگترین مقدار ویژه که با یک تکرار شونده در زمان $O(n^2)$ قابل انجام است اما مشکل اصلی محاسبه ضرب ماتریس A^TA و AA^T است که $O(n^3)$ هستند. به‌همین دلیل الگوریتم تکرار شونده‌ی قبلی ترجیح داده می‌شود.

الگوریتم pageRank

درباره این سوال کلیدی مطرح می‌شود که اهمیت هر لینک چقدر است؟ آیا دونود که هردو دارای لینک هستند اهمیت یکسانی دارند؟

سوال دومی که مطرح می‌شود این است که چه لینک‌هایی را در نظر بگیریم؟ ورودی یا خروجی یا هردو؟

الگوریتم Hits هردو را در نظر می‌گرفت. اگر بخواهیم یکی را در نظر بگیریم لینک‌های ورودی مهم‌تر است چون مختار دست ماست، بقیه باید به ما لینک بدهند. امکان دستکاری کردن لینک‌های ورودی کمتر است.

شب یلدا

Scanned with CamScanner

pageRank برهین اساس از لینک های ورودی استفاده می کند

pageRank به صورت یک مدل حریص در نظر گرفته می شود. یک Score ای داریم که بین نودهای مختلف جابجا می شود تا به حالت سکون برسد. (ثابت)

اصول pageRank

۱) اهمیت هر لینک متناسب است با Source آن لینک

۲) فرض کنید اهمیت یا Score برای نود i برابر r_i و d_i درجه خروجی i باشد. یک نود اهمیتش را به طور مساوی بین لینک های خروجی تقسیم می کند پس به هر لینک خروجی: $\frac{r_i}{d_i}$ می رسد.

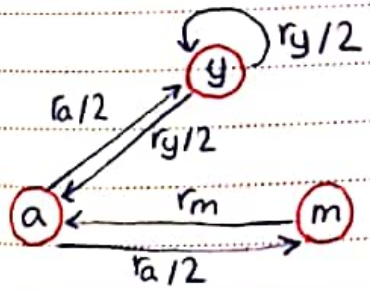
۳) Score یک نود برابر است با حاصل جمع Score های لینک های ورودی آن نود.

نتیجه: یک نود زمانی مهم می شود که لینک های ورودی زیادی داشته باشد و هر لینک ورودی Score بالایی داشته باشد.

$$r_j = \sum_{i \to j} \frac{r_i}{d_i}$$

تعریف rank برای نود j: (سیگما روی لینک های ورودی j)

اولین وبی که در سال 1839 وجود داشت:



معادلات حریص

$$\begin{cases} r_y = r_y/2 + r_a/2 \\ r_a = r_y/2 + r_m \\ r_m = r_a/2 \end{cases}$$

این معادلات را می توان با دستگاه معادلات خطی حل کرد چون اولا درست نیست، سمت چپ زمان $t+1$ است و سمت راست زمان t. دوما همینه نیست. حل معادلات خطی $O(n^3)$ است.

گام زمان

دستگاه

Scanned with CamScanner

فرم ماتریسی pageRank

ماتریس مجاورت stochastic را تعریف می‌کنیم:

به ازای هر لینکی که از نودی مثل j به نودی مثل i داریم: $M_{ij} = \frac{1}{d_j}$ ← میزان اسکوری که از j می‌گیرد

d_j ← تعداد درجه خروجی نود j است

M یک ماتریس stochastic ستونی است چون جمع عناصر ستونش برابر ۱ است. هر ستون مثل توزیع احتمال عمل می‌کند

Rank vector r ← یک بردار page rank تعریف می‌کنیم که در هر iteration نمره هر نود را می‌دهد.

خودمان این شرط را قائل می‌شویم: همیشه دنباله‌ارهای مختلف حاصل جمع scoreها برابر ۱ باشد $\sum_i r_i = 1$

همیشه یک جریان ۱ داریم و در iteration های مختلف بین نودهای مختلف پخش می‌کنیم تا زمانی که scoreها تغییر پیدا نکنند

از روی رابطه $r_j = \sum_{i \to j} \frac{r_i}{d_i}$ طبق تعریف ضرب ماتریسی می‌توان این رابطه را نتیجه گرفت:

$$r^{(t+1)} = M \cdot r^{(t)}$$

وقتی در حالت steady state، rها با هم برابر می‌شوند و r می‌شود بردار ویژه‌ی M

تفسیر قدم زدن تصادفی از pageRank:

قدم زدن تصادفی روی گراف: ابتدا یکی از نودها را به صورت تصادفی انتخاب می‌کنید. در هر مرحله t، برای رفتن به مرحله‌ی $t+1$، یکی از لینک‌های خروجی نود فعلی را به صورت تصادفی انتخاب می‌کنید و به آن نود می‌روید.

سوال: احتمال اینکه در زمان $t+1$ در نود i ام باشید چقدر است؟ (در قدم زدن تصادفی)

برای هر تایم t، بردار $p(t)$ را در نظر می‌گیریم که نشان می‌دهد احتمال اینکه در زمان t روی هرکدام از نودها باشیم چقدر است. $p(t)$ یک توزیع احتمال است.

احتمال اینکه در $t+1$ امین iteration روی نود i ام باشیم برابر است با احتمال اینکه در تکرار t ام روی نود i باشید تقسیم بر تعداد لینک‌های خروجی نود i + احتمال اینکه در تکرار t روی نود k باشید تقسیم بر تعداد

Scanned with CamScanner

۱۴۰۰

لینک‌های خروجی i_2 + ... + i_1 + i_2 + i_3 و ... لینک‌های ورودی

به نود j هستند. پس احتمال اینکه روی یک نود خاص (j) باشیم

برابر است با:

$$r_j = \sum_{i \to j} \frac{r_i}{d_{out}(i)}$$

که این رابطه همان رابطه‌ی محاسبه‌ی Score نود در pageRank است

(البته گراف حتما باید جهت‌دار باشد) پس random walk همان page Rank است.

در Random walk دنبال به steady state ی رسیم یعنی p(t+1) برابر p(t) است.

$$p(t+1) = M \cdot p(t)$$

$$\text{steady state} \rightarrow p(t+1) = M \cdot p(t) = p(t)$$

بنابراین p(t) همان stationary distribution برای random walk است.

پس برای محاسبه page rank score یک قدم زدن تصادفی روی گراف ایجاد می‌کنیم به مقدار زیادی این فرآیند را انجام می‌دهیم. در نهایت مقدار دهی که هر نود visit شده تقسیم بر کل گام‌های زمانی می‌شود احتمال دیدن آن نود. در حقیقت توزیعی به ما می‌دهد که همان stationary distribution قدم زدن تصادفی است

• که برابر با page rank score است.

چگونه در عمل page rank را حساب کنیم؟

این رابطه‌ی کلی را داریم:

محاسبه page rank در عمل

$$r_j^{(t+1)} = \sum_{i \to j} \frac{r_i^{(t)}}{d_i}$$

مشابه مثال همان درآمد iterative را داریم. از یک مقدار دلخواه اولیه شروع می‌کنیم، یک شرط توقف هم داریم:

$$\sum_i \left| r_i^{(t+1)} - r_i^{(t)} \right| < \varepsilon$$

معمولا در page rank نرم ۱ را استفاده می‌کنیم در حالی که در Hits نرم ۲ استفاده می‌کردیم.

دوتا بردار score داریم مربوط به تکرار t و t+1، قدر مطلق تفاضل‌شان را می‌گیریم و این‌ها را با هم جمع می‌کنیم

Scanned with CamScanner

که می‌شود برابر تفاضل. اگر برابر تفاضل از یک ε که کاربر Set می‌کند کوچکتر شد، الگوریتم را متوقف می‌کنیم. فرقی که با Hits دارد: عزات زیاد نمی‌شوند، یعنی خیلی نیازی به Normalization نیست.

power Iteration ← یک الگوریتم اولیه و ساده برای محاسبه page rank score

ابتدا score هر نود را با $\frac{1}{N}$ تقریب می‌زنیم.

set $r_j \leftarrow 1/N$

برای هر نود j، r'_j را محاسبه می‌کنیم که از رابطه‌ی بازگشتی محاسبه می‌شود:

1: $r'_j \leftarrow \sum_{i \to j} \frac{r_i}{d_i}$

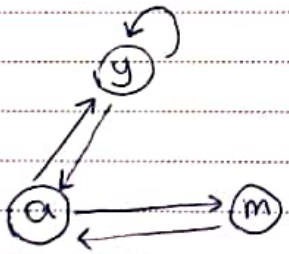
r' را در r می‌گذاریم. r' در حقیقت بردار score ای است که به صورت موقتی

2: $r \leftarrow r'$

دستور جدید محاسبه می‌شود در نهایت جای r می‌نشیند

if $|r - r'| > \varepsilon$ goto1

مثال:



	y	a	m
y	1/2	1/2	0
a	1/2	0	1
m	0	1/2	0

بعد از بار تکرار score ها دیگر تغییری نمی‌کند

$$\begin{bmatrix} r_y \\ r_a \\ r_m \end{bmatrix} = \begin{matrix} 1/3 & 1/3 & 5/12 & 9/24 & \cdots & 6/15 \\ 1/3 & 3/6 & 1/3 & 11/24 & \cdots & 6/15 \\ 1/3 & 1/6 & 3/12 & 1/6 & & 3/15 \end{matrix}$$

r_m، score کمتری پیدا می‌کند نسبت به بقیه چون تنها یک لینک ورودی دارد و بقیه دو لینک ورودی دارند. از نظر اهمیت این لینک‌ها هم با هم فرق دارند.

Scanned with CamScanner

جلسه دوازدهم - ۱۴۰۰/۸/۱۱

سوالی که مطرح می‌شود این است که در چه موقعیت‌هایی الگوریتم page rank دچار مشکل می‌شود؟

① Spider trap

وقتی به وجود می‌آید که ما حداقل یک نود در گراف داشته باشیم که فقط یک لینک خروجی دارد و آن لینک خروجی به خودش است

(a) (b)

این نود کل score شبکه را به خودش اختصاص می‌دهد score بقیه نودها را صفر می‌کند

حتی اگر google را با یک نود spider

iteration	0	1	2	3
ra	1	0	0	0
rb	0	1	1	1

② Dead end

وقتی نودی داشته باشیم که هیچ لینک خروجی ندارد

(a) (b)

iteration	۰	۱	۲	۳
ra	1	0	0	0
rb	0	1	0	0

Scanned with CamScanner

راه‌حل Spider Trap:

با احتمال β در هر نود یکی از لینک‌های خروجی نود فعلی را دنبال کنید و با احتمال $1-\beta$ به صورت تصادفی یکی از نودهای دیگر را بروید.

در روش random walk پیچ لینک دیگر وارد Spider Trap نمی‌شویم دیگر نمی‌توانیم

تقسیم page rank: در هر نود نسبت β از score این نود را بین لینک‌های خروجی نود تقسیم می‌کنیم

و نسبت $1-\beta$ از score را بین همه‌ی نودها تقسیم می‌کنیم.

در شکل مقابل random walk وقتی به نود m می‌رسد متوقف می‌شود

وقتی وارد نود m شدیم با احتمال 1 به صورت تصادفی یکی

همان‌طور که google پیشنهاد داد فقط spider trap را حل می‌کند:

$$r_j = \sum_{i \to j} \beta \frac{r_i}{d_i} + (1-\beta)\frac{1}{n}$$

الگوریتم page rank ای که الان استفاده می‌شود هم spider trap و هم handle ای Dead End

می‌کند

Scanned with CamScanner

دوره‌ی:

دی | ۹ | 30 Dec 2021
پنجشنبه | ۲۵ جمادی الاولی ۱۴۴۳

personalized page Rank

یک مسئله خیلی مهم که در گراف وجود دارد میزان proximity دو تا نود است. دو تا نود را به ما می دهند می خواهیم میزان شباهت ساختاری شان (شباهتشان از روی ساختار گراف) را کشف کنیم.

مثال: گراف نظ نویسنده ها کنفرانس ها.

می خواهیم بدانیم کنفرانس های شبیه ICDM کدام هستند؟

معیارهای proximity

۱) shortes path ← معیار proximity دو تا نود را برابر

معیار خوبی نیست مثلا در شکل مقابل نودهای A و B تنها دوست های

هستند

دی | ۱۰ | 31 Dec
جمعه | ۲۶ جمادی الاولی

Scanned with CamScanner

② Network Flow ← مثلا دریایی را از A شروع کنیم و بررسی کنیم چقدر از آن به B رسیده

این معیار هم خوب نیست. مثال:

ویژگی های معیار

اتصالات چندگانه

کیفیت مسیر را در نظر بگیرد

آیا در این مسیر نودهای درجه ۱ داریم

random walk with Restart

روشی است که استفاده می شود برای اینکه شباهت

همان random walk است با ۲ تا تفاوت: ① نود آغازین ما تغییر نکند و همیشه یکی است

Scanned with CamScanner

(۴) با احتمال β در هر مرحله لینک های خروجی را دنبال می کنیم و با احتمال $1-\beta$ به نود آغازین برمی گردیم

مثلا برای ۱۰۰۰ بار random walk را انجام می دهید، هر نودی بیشتر دیده شود proximity بیشتری با نود آغازی دارد

personalized page Rank

هدفش این است: شما یک گراف دارید، می خواهید نودهای این گراف را نه تنها براساس اهمیتشان بلکه براساس میزان re شان به مجموعه S از نودها sort کنید.

کاری که انجام

۱) نود آغازین شما یکی از نودهای S است

۲) Teleport شما به یکی از نودهای S صورت می گیرد

تغییر Score ←

$$\begin{cases} M'_{ij} = \beta M_{ij} + (1-\beta)/|S| & \text{if } i \in S \\ M'_{ij} = \beta M_{ij} & \text{otherwise} \end{cases}$$

M'_{ij} همان M است

Scanned with CamScanner

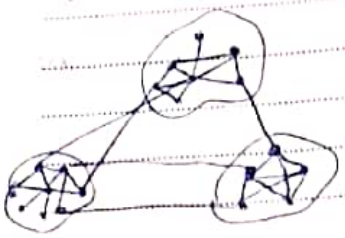
دی
دوشنبه | ۱۳ | 3 Jan 2022
۲۹ جمادی الاولی ۱۴۴۳

۱۴۰۰

جلسه سیزدهم — ۱۴۰۰/۸/۱۶

سوال: pinterest چگونه Visual search انجام میدهد؟

Scanned with CamScanner



ساختار Community در شبکه‌های پیچیده

در مورد موضوعی که از شبکه‌های پیچیده داریم به این صورت است:

در یک سری کپش‌های dense داریم که بین آن‌ها لینک‌های کمی وجود دارد

یک سوالی که این حا وجود دارد این است که در شبکه‌ی پیچیده اطلاعات چگونه منتشر می‌شود؟

لینک‌هایی که در شبکه‌های اجتماعی موجب پخش اطلاعات می‌شوند از نظر ساختاری به چه صورت هستند؟

لینک‌های short و long داریم. ارتباط این لینک‌ها با اطلاعاتی که پخش می‌کنند چگونه است؟

یک موردی که در شبکه‌های اجتماعی دیده می‌شود این است که شما در شبکه‌های اجتماعی چگونه از eventها مطلع می‌شوید؟ حتما از طریق دوستانتان و کسانی که به آن‌ها لینک دارید مطلع می‌شوید ولی کدام دوستانتان؟

نکته جالب این بود که اطلاعات جدید را (مثلا در مورد یک موقعیت شغلی جدید) از دوستان تر لینک دریافت نمی‌کنند بلکه از دوستان دور و آشنایان دریافت می‌کنند. اصطلاحا به آن‌ها acquaintances گفته می‌شود. از نظر ساختاری لینک بین دو نود یا کپش‌ها dense یا sparse گراف باشد. یا نود تر لینک یا دو نود دور را هم وصل کنند.

از نظر اجتماعی هم یک یال می‌تواند بالی باشد که خیلی استفاده می‌شود و لینک فعالی است. این دو نود رفت و برگشت بیشتر را می‌بینند، با هم صحبت می‌کنند... یا اینکه بالی است که active نیست زیاد و این دو شخص آشنایی دارند و با هم دارند و خیلی کم یکدیگر را می‌بینند.

از نقطه نظر اجتماعی یک یال می‌تواند قوی یا ضعیف باشد. یال قوی یالی است که از نظر اجتماعی خیلی زیاد استفاده می‌شود. یال ضعیف یالی است که از نظر اجتماعی زیاد استفاده نمی‌شود.

بستار سه‌گانه (قلتی) ← اصل Triadic Closure

اگر دو نود یک همسایه مشترک داشته باشند با احتمال قابل توجهی بین آن‌ها لینک به وجود می‌آید.

Gronovetter یک ارتباط بین نقش ساختاری و نقش اجتماعی یک یال برقرار کرد.

Scanned with CamScanner

1) یال هایی که از نظر ساختاری embeded هستند یعنی در قسمت های dense گراف هستند و معنا قوی را هم وصل می کنند که همسایه های مشترک زیادی دارند، این ها از نظ اجتماعی هم قوی هستند.

2) یال هایی که از نظر گراف و ساختاری در قسمت های dense نیستند و در قسمت های sparse هستند و دو تا نود را به هم وصل می کنند که همسایه مشترک ندارند، از نظر اجتماعی هم یال های ضعیفی هستند و زیاد active نیستند.

3) یال هایی که قسمت های مختلف گراف را هم متصل می کنند و از نظر ساختاری ضعیف هستند، از نظر فراهم کردن اطلاعات خیلی قوی هستند. چون موجب می شوند دو بخش دور گراف به هم متصل بشوند، اطلاعاتی که در یک قسمت وجود دارد به قسمت دیگر منتقل بشود ولی یال هایی که از نظ structure خیلی قوی هستند، از نظر فراهم کردن اطلاعات ضعیف هستند یعنی اطلاعاتی به ما نمی دهند که از راه های دیگر هم نمی توانند بدست آورید.

دستار مثلثی (Triadic closure) یکی از عامل های اصلی ضریب خوشه بندی بالا در شبکه های پیچیده است

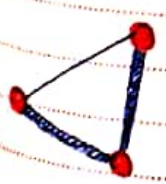
Granovetter از عاکرد یال هایی که از نظر ساختاری قوی هستند از نظر اجتماعی هم قوی هستند و یال هایی که از نظر ساختاری ضعیف هستند از نظر اجتماعی هم ضعیف

تعریف: یال bridge (پل) یالی است که اگر حذف شود تعداد کامپوننت های گراف زیاد می شود.

local bridge: یالی است که span اش بزرگتر از ۲ است.

← span یک یال: فاصله ی بین دو تا endpoint یک یال اگر آن یال وجود نداشته باشد

Strong Triadic Closure: (دستار مثلثی قوی) ← ۳ تا نود را در نظر بگیرید، اگر بین دو تا جفت از آن ها یال اجتماعی قوی داشته باشیم بین جفت سوم هم حتما یک یال خواهید داشت



حالا یکه از نظر اجتماعی قوی باشه یا ضعیف باشه

ارعای Granovetter این بود که اگر در شبکه ی پیچیده ای دستار مثلثی قوی برقرار باشد، یال هایی که از نظر structural ضعیف هستند از نظر اجتماعی هم ضعیف هستند.

یعنی local bridge ها

bridge ← local ها از نظر اجتماعی ضعیف هستن

bridge یه نوع local bridge هست که span اش بی نهایته.

Scanned with CamScanner

دی پنجشنبه | ۱۶ | 6 Jan 2022 | ۳ جمادی الثانیه ۱۴۴۳

این ادعا را به صورت رسمی تبیان می کنیم:

فرض کنید یک نود A داریم که اصل strong Triadic closure را برآورده می کند و نود A حداقل ۲ یال قوی دارد. درنتیجه هر local bridge ای که نود A داشته باشد weak است و نمی تواند strong باشد.

یعنی اگر A یک نود باشد که این اصل را برآورده کند و حداقل ۲ یال قوی داشته باشد، هر یالی که به A می رسد و از نظر ساختاری ضعیف است (یعنی local bridge است)، این یال از نظر اجتماعی هم باید ضعیف باشد.

اثبات با برهان خلف: فرض کنید یال بین A و B (در شکل اسلاید)، strong باشد، طبق اصل strong Triadic closure باید بین B و C یک یال وجود داشته باشد که در این صورت span(A,B) می شود ۲ و یک A و B دیگر local bridge نیست ← تناقض

شهادت حضرت فاطمه (س) (۱۱ هـ ق) (تعطیل)

پس لبه هایی که از نظر ساختاری ضعیف هستند یعنی span شان

دی جمعه | ۱۷ | 7 Jan ■ ۴ جمادی الثانیه

بزرگتر از ۲ است، از نظر اجتماعی هم ضعیف هستند.

برای شبکه های عاشق های تلفنی نیز بررسی شد:

در این شبکه یک یال قوی در نظر گرفته می شود اگر تعداد عاشق هایی که توسط آن یال گرفته شده از یک حدی بالاتر باشد. از نظر اجتماعی

از نظر ساختاری هم یک معیار متقن، overlap میان دو تا endpoint یک یال است:

Edge overlap: $O_{ij} = \frac{|N(i) \cap N(j)|}{|N(i) \cup N(j)|}$

برای اینکه یک یال local bridge باشد باید overlap اش صفر باشد.

اگر یال ها را برحسب قدرت اجتماعی کم به زیاد حذف کنیم گراف زودتر درهم می شکند نسبت به حالتی که

Scanned with CamScanner

۱۴۰۰

یال ها را از قدرت اجتماعی زیاد به کم حذف کنیم. یعنی یال هایی که از نظر اجتماعی ضعیف تر هستند آن هایی هستند که بین گروه ها قرار دارند. این یال ها بین گروه های مختلف قرار می گیرند و آن ها را هم وصل می کنند و اخباری که داخل یک گروه وجود دارد را به گروه دیگر می رسانند. دیگه از دوستان نزدیک اخبار جدید نمی شنویم بلکه از دوستان دور اخبار جدید را می شنویم.

یال های از نظر اجتماعی قوی داخل گروه ها و ضعیف ها بین گروه ها هستند. از طریق یال های ضعیف هست که اطلاعات به دست ما می آید نه قوی.

کریتیک ها را از overlap کم به زیاد حذف کنیم گراف زودتر می شکند (سست به زیاد به کم)

سوال: Community (cluster) ها را در گراف چگونه پیدا کنیم؟

تعریف Network Community مشابه cluster است: شباهت داخلی شان زیاد (فاصله کم) و شباهتشان با بقیه کم (فاصله زیاد) است ← اینجا هم Community مجموعه ای از نودهاست که بینشان تعداد لینک ها زیاد است و با بقیه نودها تعداد لینک هایشان خیلی کم است

یک راه برای یافتن Community ها Visualize کردن است ← برای گراف های بزرگ ممکن نیست حتی برای گراف های کوچک هم همیشه درست Visualize می کنیم نمی توانیم Community ها را تشخیص بدهیم

می خواهیم در مورد الگوریتم های مختلف پیدا کردن Community در یک شبکه بحث کنیم

روش اول: Strength of weak Ties (قدرت یال های ضعیف) (Girvan_Newman)

یعنی دسته کمال حاضر ضعیف هستن اما از یک جهت خیلی قوی هستند ← تعداد shortest path هایی که از یال های ضعیف عبور می کند خیلی زیاد است معمولا چون دوتا دسته از گراف را به هم وصل می کنند غیر از این یال هیچ راه ارتباط دیگری با هم ندارند

Scanned with CamScanner

Edge betweenness ← تعریف غیررسمی: تعداد shortest path هایی که از روی یک یال عبور می‌کنند

node betweenness : تعداد shortest path هایی که از روی یک نود عبور می‌کنند

(در عمل معمولا نسبت می‌گیریم ← یعنی نسبت shortest path هایی که عبور می‌کنند)

یک راه‌حل برای پیدا کردن Community ها این است که یال‌هایی که از نظر اجتماعی ضعیف هستند را حذف کنیم. الگوریتم Girvan_Newman در واقع همین.

ماتریس مجاورت ساختاری

اما گراف‌های دنیای واقعی را که به ما می‌دهند در آن مشخص نیست که کدام یال‌ها قوی و ضعیف هستند.

باید یک معیار مناسب داشته باشیم. الگوریتم Girvan_Newman از عکس Edge betweenness استفاده می‌کند که به خوبی قدرت اجتماعی یال‌ها را منعکس می‌کند.

الگوریتم Girvan_Newman یک الگوریتم hierarchical (سلسله مراتبی) است. تعداد iteration دارد که می‌تواند دلخواه باشد یا با یک معیاری تعیین شود. در هر iteration یک گراف بزرگ را ممکن است به ۲ یا چند گراف کوچک تقسیم کنیم.

① در هر iteration یک edge betweenness تک تک یال‌های موجود در گراف را حساب می‌کنیم

② یال یا یال‌هایی با بالاترین edge betweenness را از گراف‌مان حذف می‌کنیم

در نهایت که متوقف می‌شویم Connected component هایی که به دست می‌آورید خوشه‌های ما هستند

برای یافتن Connected component ها هم می‌توان از یک الگوریتم ساده (مثلا BFS یا DFS) استفاده کرد، هر بار که می‌خواهیم شروع کنیم به جست‌وجو در گراف یک Connected component جدید پیدا کنیم.

الگوریتم Girvan_Newman از نظر کیفیت خیلی خوب است در عمل و Community های خوبی استخراج می‌کند.

اما مشکلی که دارد زمان زیادش است. پیچیدگی $O(nm)$ است. n تعداد نودها و m تعداد یال‌ها

مربوط به iteration است.

Scanned with CamScanner

محاسبه‌ی edge betweenness در هر بار برای گراف بدون وزن $O(nm)$ زمان می‌برد.

برای گراف‌های وزن‌دار $O(nm + n^2 \log n)$ زمان می‌برد.

برای گراف‌های وزن‌داری که وزن منفی هم بتوانند باشند مسئله np hard می‌شود و حل نمی‌شود.

n و m عدایی می‌شوند.

نکته: برای گرافی که از اجرای iteration قبلی حاصل شده دوباره حساب می‌کنیم edge betweenness را در هر iteration

Scanned with CamScanner

جلسه پانزدهم - ۱۴۰۰/۸/۲۳

Modularity Matrix: $B_{ij} = A_{ij} - \frac{K_i K_j}{2m}$

Membership: $S = \{-1, +1\}$

$$Q(G,S) = \frac{1}{4m} \sum_{i \in N} \sum_{j \in N} \left(A_{ij} - \frac{K_i K_j}{2m}\right) S_i S_j = \frac{1}{4m} \sum_{i,j \in N} B_{ij} S_i S_j$$

$$= \frac{1}{4m} \sum_i S_i \underbrace{\sum_j B_{ij} S_j}_{= B_i \cdot S} = \frac{1}{4m} S^T B S$$

$$\boxed{Q(G,S) = \frac{1}{4m} S^T B S}$$

که هرکدام از خانه‌های بردار S یا +1 است یا -1.

B متقارن است چون گراف ما غیرجهت‌دار است و ماتریس مجاورت آن یعنی A متقارن است، از طرفی $\frac{K_i K_j}{2m}$ برای (i و j) و (j و i) با هم برابر است و $B_{ij} = A_{ij} - \frac{K_i K_j}{2m}$. پس $B_{ij} = B_{ji}$ و B متقارن است.

اثبات می‌شود که B یک ماتریس positive semi-definite است.

تعریف: ماتریس A را positive semi-definite گویند اگر بتوان آن را به این صورت نوشت: $A = U \cdot U^T$

که ستون‌های U بر هم عمودند (سطرهای U^T بر هم عمودند)

بردار x برای ماتریس A یک بردار ویژه است اگر و تنها اگر: ① $x \neq 0$

② عدد λ وجود داشته باشد به گونه‌ای که $A \cdot x = \lambda \cdot x$ (λ می‌تواند ۰ باشد)

در این صورت λ را مقدار ویژه‌ی متناظر با بردار ویژه‌ی x می‌گوییم. در حقیقت بردارهای ویژه‌ی A بردارهایی هستند که وقتی تحت A متقل می‌شوند، جهت آن‌ها عوض نمی‌شود فقط طولشان λ برابر می‌شود.

ثابت می‌شود که یک ماتریس متقارن (با احتساب مراتب) n تا مقدار ویژه‌ی حقیقی دارد:

$\lambda_1 \leq \lambda_2 \leq \dots \leq \lambda_n$

ممکن است بعضی از این مقادیر ویژه تکراری باشند چون ریشه‌های معادله مشخصه هستند. برای ماتریس‌های غیرمتقارن مقادیر ویژه می‌تواند غیرحقیقی باشد

Scanned with CamScanner

فرض کنید x_i بردار ویژه‌ی متناظر با λ_i است

برای ماتریس‌های متقارن x_1 تا x_n برهم عمود هستند هر دو تا بینشان (orthogonal).

اگر هم normalize کنیم و طول هر کدام ۱ باشد orthonormal می‌گیند.

فقط در مورد ماتریس‌های متقارن این دیتا وجود دارد.

اگر ماتریس A علاوه بر متقارن بودن، positive semi-definite باشد مقادیر ویژه‌ی آن بزرگتر مساوی صفر هستند. ($\lambda_i \geq 0$)

قضیه تجزیه طیفی (Eigen Decomposition theorem)

یک ماتریس متقارن را می‌توان به این صورت نوشت: $A = \sum_i x_i \cdot \lambda_i \cdot x_i^T$

عوض کتاب‌ها گران‌ها را عنوان جدولی گذاشتیم که اگر اتفاقات ماتریس کنارشان متقارنه

Modularity Optimization

می‌خواهیم این تابع هدف را max کنیم به طوری که S مثبت و منفی ۱ است $\leftarrow Q(G,S) = \frac{1}{4m} S^T B S$

$$= S^T \left[\sum_{i=1}^{n} x_i \lambda_i x_i^T \right] S = \sum_{i=1}^{n} S^T x_i \lambda_i x_i^T S = \sum_{i=1}^{n} (S^T x_i)^2 \lambda_i$$

تا زمانی که S فقط بتواند $\{-1, +1\}$ باشد این کمینه‌سازی np-hard است.

چون B متقارن است می‌توانیم به صورت تجزیه طیفی آن را بنویسیم.

$x_i^T S$ یعنی ضرب داخلی x_i در S

$S^T x_i \leftarrow$ یعنی ضرب داخلی S در x_i $\leftarrow S^T x_i, x_i^T S$

فعلا $\sum_{i=1}^{n} (S^T x_i)^2 \lambda_i$ همچنان np-hard است چون تقریبی نزده‌ایم هنوز

اگر شرط اینکه $S \in \{+1, -1\}$ را برداریم و S هر مقداری بتواند داشته باشد، مسئله در زمان چندجمله‌ای به صورت کمینه حل شود. به آن quadratic form optimization می‌گویند. که یک مسئله‌ی شناخته‌شده در حوزه‌ی است. مقدار این max می‌شود λ_n (بزرگترین مقدار ویژه‌ی ماتریس B) و S ای که این max را می‌دهد، بردار ویژه‌ی متناظر با λ_n است:

$$S = x_n$$

Scanned with CamScanner

با این حا این رابطه را داریم و مسئله‌ی ما هنوز np_hard است:

$$\max Q(G,s) = \sum_{i=1}^{n} (S^T x_i)^2 \lambda_i \simeq (S^T x_n)^2 \lambda_n$$

در گام اول به‌های اسلاید هدی λها را در نظر نگیریم فقط λ_n و x_n را در نظر بگیریم چون در گراف‌های دنیای واقعی یک Gap بزرگ بین λ_n و بقیه λها وجود دارد و می‌توانیم این بقیه را حذف کنیم. معمولا λ_n مقدار قابل توجهی بزرگتر از λ_{n-1} است.

در گام دوم λ_n را هم می‌توانیم حذف کنیم، چون همیشه مثبت است و تاثیری در maximization ندارد.

در گام سوم توان 2 را هم حذف می‌کنیم چون $S^T x_n$ یک عدد است و اگر بدانیم مقدار ماکزیمم آن مثبت است پس همان مقداری که $(S^T x_n)^2$ را max می‌کند، $S^T x_n$ را هم max می‌کند.

مانده پیدا کردن مقداری برای S هستیم که $(S^T x_n)^2$ را max کند پس می‌توانیم $S^T x_n$ را به‌جای آن در نظر بگیریم. پس در کفایت به این عبارت می‌رسیم:

maximize: $\sum_{j=1}^{n} s_j \cdot x_{nj}$ ← ضرب داخلی S^T در x_n را می‌توان اینگونه نوشت

x_n بردار ویژه‌ی dominant برای ماتریس B است

$$s_j = \begin{cases} +1 & \text{if } x_{nj} \geq 0 \\ -1 & \text{if } x_{nj} < 0 \end{cases}$$

برای max کردن این عبارت، هرگاه x_{nj} مثبت بود s_j را $+1$ و هرگاه x_{nj} منفی بود s_j را -1 می‌گیریم یعنی برای هر بود خانه‌ی متناظرش را در x_n در نظر می‌گیریم اگر مثبت بود کلاستر آن بود را $+$ و اگر منفی بود کلاستر آن بود را منفی می‌گیریم و می‌توان همین فرآیند را به صورت سلسله مراتبی انجام داد. در هر مرحله یک گراف به دو تا کلاستر تقسیم می‌شود.

اصطلاحا به آن Fast Modularity optimization گفته می‌شود چون در زمان polynomial مسئله را حل می‌کند

شمای کلی الگوریتم به این صورت است:

Scanned with CamScanner

۱۴۰۰ $B_{ij} = A_{ij} - \frac{k_i k_j}{2m}$ ماتریس Modularity یعنی B را پیدا می‌کنیم

① ابتدا ماتریس

② dominant eigen vector (بردار ویژه‌ی متناظر با بزرگترین مقدار ویژه) را پیدا می‌کنیم

③ علت نمودهای مختلف در x_n را چک می‌کنند. هر بار که حاصل متناظرش در x_n مثبت بود

+1 می‌شود در غیر این صورت -1. در نتیجه گراف به دو بخش تقسیم می‌شود

همین کار را به صورت سلسله مراتبی می‌توان ادامه داد تا کلاسترهای بیشتر حاصل شود و یک معیار متوقف

(dominant eigen Vector of B) است x_n یافتن ، iteration فازی که زمان‌بر است ، در هر

power Method ← برای هر ماتریس (نه فقط مقارن) یک الگوریتم خیلی معروف برای این کار:

از بردار دلخواه $v^{(0)}$ آغاز می‌کنند. بعد در یک فاز iterative فرض می‌کنیم

$$v^{(t+1)} = \frac{B v^{(t)}}{\| B v^{(t)} \|}$$

یک بردار در تکرار t دارید و یک بردار در تکرار t+1 می‌خواهید

تولید کنید. v^{t+1} از روی v^t اینگونه محاسبه می‌شود:

خرج کسر فقط برای نرمال سازی است. بعد از مقداری گام برداری به دست می‌آید که تا حدخوبی به دست

dominant eigen vector همگرا شده است.

time Complexity ← ضرب یک ماتریس در یک بردار داریم ← $O(n^2)$ ← تا حلقه تودرتو

$$n \times T_1 \times T_2$$

پس time complexity کل برآیند می‌شود:

T_1 ← مقدار iteration های power Method

T_2 ← " " " الگوریتم Fast Modularity optimization

T_1 معمولا ثابته (۱۰، ۱۲ تا کافیه)

T_2 هم می‌تونیم ثابت در نظر بگیریم یدیکه درکل می‌شود $O(n^2)$

الگوریتم Girvan_Newman ← $O(nm)$ (برای گراف بدون وزن)

Fast Modularity optimization ← $O(n^2)$ ✓ سریعتره

Scanned with CamScanner

بهمن | ۷ | 27 Jan 2022
پنجشنبه | ۲۴ جمادی الثانیه ۱۴۴۳

Spectral clustering for Graph partitioning

معمولا ۳ تا مرحله دارند الگوریتم‌های خوشه‌بندی طیفی:

① pre-processing: در این مرحله یک تغییر ماتریسی از گراف می‌سازیم. مثلا ماتریس مجاورت یا لاپلاسین

② Decomposition (تجزیه): یک عمل تجزیه روی ماتریس انجام بدهیم

یعنی ماتریس را به صورت حاصل ضرب دو یا بیشتر ماتریس نوشته شود. مثال‌هایی از تجزیه ماتریسی: SVD، مقادیر ویژه، QR و... اگر ماتریس را به صورت حاصل ضرب دو تا ماتریس نوشته باشیم

این ماتریس‌ها یک سری latent factor ها (یعنی dimension یا dimension اصلی) معرفی می‌کنند

ماتریس دوم تخصص یا مشخصات (این latent factor ها) را به ما می‌دهد. اگر بتوانیم داده را به یک dimension اصلی منتقل کرده ایم و در این dimension های اصلی براساس تخصص clustering را انجام می‌دهیم. (دو مرحله بود)

③ Grouping

در نتیجه گام دوم، هر نقطه در فضای جدید تبدیل شده که در این مرحله این نقاط را clusterبندی می‌کنیم

بهمن | ۸ | 28 Jan
جمعه | ۲۵ جمادی الثانیه

فرض کنید با گراف‌های غیرجهتدار کار می‌کنیم. معمولا وقتی با طیف گراف کار می‌کنیم فرض می‌کنیم گراف غیرجهتدار.

فرض کنید تسک شما Bi_partitioning است یعنی گرافی که دارید را می‌خواهید تقسیم کنید به ۲ تا گراف کوچکتر.

سوال اول اینه که یک partition خوب را چطور تعریف می‌کنید؟ چطور دو تا پارتیشن‌بندی را مقایسه کنیم؟

مسئله‌ی efficiency است.

یک partitioning خوب مثل clustering خوب باید تعداد لینک‌ها

Scanned with CamScanner

یک معیار برای کار ما: edge cut

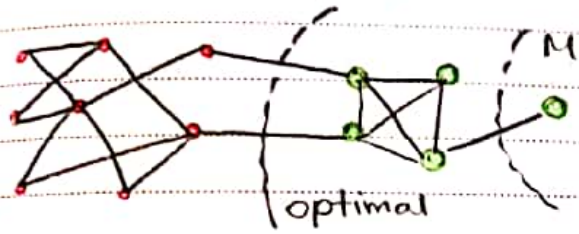
Cut(A,B): مقدار یال هایی که یک سرشان در پارتیشن A و یک سرشان در پارتیشن B است.

$$cut(A,B) = \sum_{i \in A, j \in B} w_{ij}$$

اگر یال ها وزن دار باشند:

یک راه حل ساده این است که تمام پارتیشن بندی ها را بیابیم و cut را برای آن ها حساب کنیم. هرکدام مقدار cut کمتری داشت آن را در نظر می گیریم

اما این معیار در برخی موارد خوب عمل نمی کند. اگر گراف داشته باشید که نوری با تک یال داشته باشد، همیشه آن را انتخاب می کند. مثلا:



در این جا نوری که تک یال دارد Min Cut را دارد

علتش این است که فقط تعداد یال های بین دو partion را در نظر می گیرد و density داخلی partition ها را در نظر نمی گیرد.

معیار Conductance (ضریب رسانایی یا انقال)

$$\phi(A,B) = \frac{cut(A,B)}{\min(vol(A), vol(B))}$$

← این مقدار را می خواهیم بیابیم Min

Vol(A) ← حاصل جمع درجه نود های A (اگر گراف وزن دار باشد درجه ی وزن دار در نظر می گیریم)

این معیار خوب کار می کند چون در خرج کسر density داخلی هر پارتیشن را در نظر گرفته ایم

تمام partition های مختلف A و B را در نظر می گیریم و این را انتخاب می کنیم که Conductance کمتری دارد.

در عمل خیلی کمتر از cut عمل می کند بجز حالت های degenerate که در cut داشتیم را ندارد.

مسئله ی پیدا کردن افرازی که Conductance برای آن کمینه باشد، np_hard است.

Scanned with CamScanner

ماتریس A ← ماتریس مجاورت غیر مختار

x یک بردار است در فضای n بعدی $x = (x_1, x_2, \dots, x_n)$

$$\begin{bmatrix} a_{11} & \dots & a_{1n} \\ \vdots & & \\ a_{n1} & \dots & a_{nn} \end{bmatrix} \begin{bmatrix} x_1 \\ \vdots \\ x_n \end{bmatrix} = \begin{bmatrix} y_1 \\ \vdots \\ y_n \end{bmatrix} \qquad y_i = \sum_{j=1}^{n} A_{ij} x_j = \sum_{(i,j) \in E} x_j$$

می‌توانیم به هر مقداری که نود x_n دارد به چشم label برای آن نود نگاه کنیم

y_i می‌شود ضرب داخلی سطر i ام ماتریس A در بردار x . می‌توانیم سیگما را روی یال‌ها ببندیم و A_{ij} را حذف کنیم چون در جاهایی که یال داریم 1 است و در سایر جاها 0 است. در حقیقت می‌توان بشکل sum بستن برچسب همسایه‌ها برای هر نود به ضرب ماتریس در بردار نگاه کنیم.

ضرب ماتریس در بردار یک label جدید به هر نود اختصاص می‌دهد که برایند جمع لیبل‌های قبلی همسایه‌های آن نود است.

Spectral Graph Theory

در مورد spectrum یا طیف یک گراف صحبت می‌کند.

spectrum یعنی مقادیر ویژه‌ی ماتریس تان را پیدا کنید، بردارهای متناظر با این مقادیر ویژه را هم پیدا کنید آن‌ها را مرتب کنید

$$\lambda_1 \leq \lambda_2 \leq \dots \leq \lambda_n$$

اثبات می‌شود که همیشه n تا مقدار ویژه‌ی حقیقی داریم (با احتساب مراتب)

d-regular Graph

فرض کنید ما یک گراف متصل منظم است که درجه‌ی هر نود در آن d است. می‌خواهیم یک مقدار و بردار ویژه برای آن حدس بزنیم

$$A \cdot x = \lambda \cdot x \Rightarrow x = (1, 1, \dots, 1)$$

$$A \cdot x = (d, d, \dots, d) = \lambda \cdot x \Rightarrow \lambda = d$$

Scanned with CamScanner

Scanned with CamScanner

ماتریس Laplacian

$L = D - A$

بردار $x = (1, \dots, 1)$ یک بردار ویژه برای L است که مقدار ویژه‌ی متناظر با آن ۰ است

$L.x = 0 \Rightarrow \lambda = \lambda_1 = 0$

ماتریس L یک ماتریس positive semi-definite است

پس ۲ ویژگی مهم در مورد L: ① مقادیر ویژه‌ی آن

②

یک ماتریس po است اگر و تنها اگر یکی از این ۳ شرط را برآورده کند:

①

② برای هر x: $x^T L x = \sum_{ij} L_{ij} x_i x_j > 0$

③ $L = N^T . N$

Scanned with CamScanner

۱۴۰۰

فرض کنید یک ماتریس متقارن مثل M داریم. می خواهیم x را به گونه‌ای که عبارت زیر را min کند پیدا کنیم:

$$\frac{x^T M x}{x^T x}$$

مقدار max را برای این کسر به دست آوردیم که برابر λ_n بود و x ای که آن را max می کرد x_n بود

مقدار min هم در جبر خطی حل شده که λ_1 است و x آن هم x_1 است

اگر این شرط را بگذاریم که: $x^T w_1 = 0 \leftarrow x : x^T w_1 = 0$ یعنی بردارهای x ای که بر w_1 عمود باشند

حواب آن λ_2 است و x ای که این شرط را دارد x_2 است.

$$\lambda_2 = \min_{x : x^T w_1 = 0} \frac{x^T M x}{x^T x}$$

گاهی اوقات قید طول x برابر 1 را اضافه می کنیم

و خرج کسر را حذف می کنیم چون وقتی طول x برابر 1 باشد:

$$x^T x = |x|^2 = 1$$

$$x^T L x = \sum L_{ij} x_i x_j = \sum (D_{ij} - A_{ij}) x_i x_j$$

$$= \sum D_{ii} x_i^2 - \sum_{(i,j) \in E} 2 x_i x_j$$

$$= \sum_{(i,j) \in E} (x_i^2 + x_j^2 - 2 x_i x_j) = \sum_{(i,j) \in E} (x_i - x_j)^2$$

Scanned with CamScanner

خلاصه

۱) بردار x یک unit vector

۲) می‌دانیم که x عمود بر ۱ است

$$\sum_i x_i \cdot 1 = \sum_i x_i = 0$$

۳)

$$\lambda_2 = \min \frac{\sum_{(i,j)\in E} (x_i - x_j)^2}{\sum_i x_i^2}$$

آیا می‌توانیم مسئله‌ی clustering را در قالب این مسئله که جواب آن را می‌دانیم بیان کنیم؟

ما دو تا کلاستر داریم: +1 و -1. فرض کنیم تعداد عناصر این دو کلاستر با هم برابر است

$$y_i = \begin{cases} +1 & \text{if } i \in A \\ -1 & \text{if } i \in B \end{cases}$$

در نتیجه مجموع عنصر y برابر صفر است: $|A| = |B| \Rightarrow \sum_i y_i = 0$

برای این که یک طرف آن در A و یک طرف آن در B است هزینه ۴ را در نظر می‌گیریم. (همان cut است با این تفاوت که هزینه به جای ۱، ۴ است)

$$\arg\min_{y \in [-1,+1]^n} f(y) = \sum_{(i,j)\in E} (y_i - y_j)^2$$

Scanned with CamScanner

۱۴۰۰

در واقع در این رابطه یک Cut از Graph variation را در نظر گرفته اید

$$\min \quad f(y) = \sum_{(i,j)\in E} (y_i - y_j)^2$$

① مرحله pre-processing ←

② مرحله Decomposition ←

Scanned with CamScanner

بهمن | ۱۷ | 6 Feb 2022
یکشنبه | ۴ رجب ۱۴۴۳

مرحله Grouping ← دعل

مثلا در فضای 1 بعدی یک پارتیشن بندی خوب این است که مقادیر هشت وهفت را 2 تا کلاستر بگیریم.

یک روش دیگر: میانگین یا میانه داده ها را در نظر بگیریم. میانگین یا میانه را به دست می آوریم، آن هایی که کوچکتر از آن هستند یک کلاستر و آن هایی که بزرگتر هستند یک کلاستر.

یک روش دیگر: واقعا در این فضای 1 بعدی پارتیشن بندی را انجام دهیم. بزرگترین Gap را پیدا کنیم، آن هایی که قبل از Gap هستند در یک کلاستر و آن هایی که بعد از Gap هستند در یک کلاستر دیگر.

K way Spectral Clustering

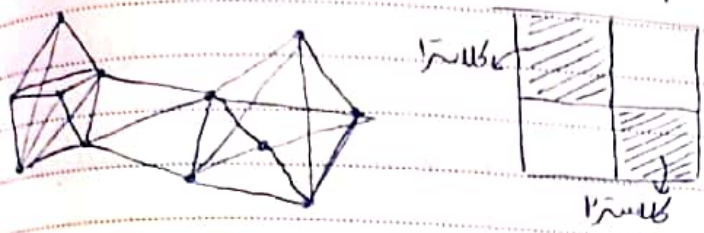
راه این است که

Scanned with CamScanner

جلسه هفدهم - ۱۴۰۰/۸/۲۰

کشف جوامعی که overlap دارند

setting ای که تاالان داشتیم: یک گراف داریم که به چند تا کلاستر تقسیم می‌شود. ماتریس مجاورت به این صورت است:



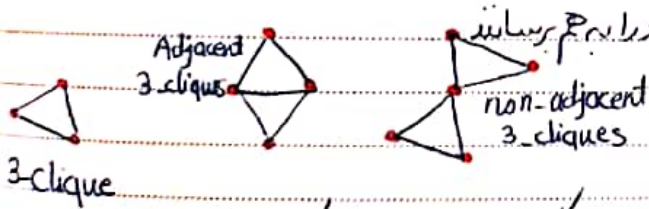
داخل کلاسترها dense است ماتریس مجاورت. پس کلاسترها تقاطع کمی دارند.

اما در دنیای واقعی در بسیاری از کاربردها نودها به بیش از یک Community تعلق دارند. چگونه در این حالت Community ها را پیدا کنیم؟

Clique Percolation Method (CPM)

این الگوریتم تعریف می‌کند: دو تا نود در یک Community هستند اگر یک chain (زنجیره یا دنباله) از (و تنها اگر)

clique های k تایی همسایه باشند که این دو تا نود را به هم برسانند

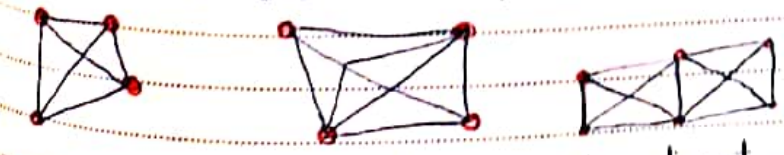


clique: زیرگراف کامل

دو تا clique با اندازه k را همسایه (Adjacent) می‌گوییم اگر k-1 نود مشترک داشته باشند. (مجاور)

K_clique Community:

مجموعه‌ای از نودها که از طریق توالی‌ای از k_clique ها قابل رسیدن (همسایه) هستند، یعنی فرض می‌کنیم نودهایی که از طریق یک توالی از k_clique های همسایه همدیگر را می‌رسند، داخل یک Community هستند



در این شکل اگر k=۳ باشد ۲ تا اجتماع پیدا می‌شود اگر k=۴ فقط ۱ تا Community می‌شد

(رسیدن از طریق k_clique ها خاصیت transivity ندارد)

Scanned with CamScanner

CPM Steps

گام اول: maximal_clique ها را پیدا می‌کنیم

maximal_clique : clique ای است که هیچ اسپرگرافی نداشته باشد که آن هم clique باشد

یعنی هیچ نود دیگه‌ای نمی‌توانیم به این clique اضافه شود که

گام دوم: اسپرگراف clique overlap را می‌سازیم به این صورت که هر maximal_clique یک نود در این گراف است و بین دو تا maximal_clique یال وجود دارد اگر و تنها اگر K-1 adjacent باشند

یعنی با هم همسایه باشند و K-1 نود مشترک داشته باشند (مثال در اسلاید)

گام سوم: Connected Component های اسپرگراف را در نظر می‌گیریم. هر Connected Component یک Community را به ما نشان می‌دهد

نکته: اگر گراف منفصل باشد و K=2 آن گاه هر گروه‌ها در یک Community قرار می‌گیرند.

فرمول مقداری برای K خوب است که در آن Community های با اندازه مختلف بیشتری داشته باشیم

Clique overlap Matrix (مثال اسلاید)

هر clique در این ماتریس یک سطر و یک ستون دارد. خانه‌ی متناظر هر سطر و ستون نشان می‌دهد که clique های مربوط به آن سطر و ستون چند تا نود مشترک دارند

حتی این ماتریس ساخته شد، خانه‌هایی که از K-1 بزرگتر هستند را ۱ و آن‌هایی که از K-1 کوچکتر هستند را صفر set می‌کنیم. چون می‌خواهیم فقط clique هایی را در نظر بگیریم که حداقل K-1 تا نود مشترک دارند

از روی این ماتریس مجاورت به دست آمده می‌توان Connected component ها را پیدا کرد که هر CC یک Community را نشان می‌دهد.

نکته: مثلا وقتی K=4 ست شود نودهایی که عضو هیچ clique چهارتایی نباشند (هیچ اجتماعی قرار نمی‌گیرند)

Scanned with CamScanner

۱۴۰۰

maximal_Clique ← پیدا کردن یک maximal clique است np-hard

یک مسئله‌ی تصمیم‌گیری می‌ماند ← پیدا کردن maximal_Clique

np_hard پیدا کنید K با اندازه clique یک ... اینکه یک گراف به شما داده شود و از شما خواسته‌اند یک

است و ورژن decision آن np_complete است. (clique با اندازه K دارد یا نه؟)

الگوریتم تقریبی:

از یک نود آغاز این شروع می‌کنید و آن را توسعه می‌دهید تا clique ای که این نود به آن تعلق دارد بیابید تا جایی که

دیگر نتوان این clique را توسعه داد. در این حالت یک maximal clique یافت شده و آن را به خروجی می‌فرستید.

• چه نودهایی می‌تواند به عنوان Seed انتخاب شوند؟

اگر بخواهیم clique ای را بیابیم که نود A به آن تعلق دارد باید نودهایی که همسایه A هستند را به عنوان

Seed انتخاب کنیم زیرا در یک clique همه‌ی نودها با هم لینک دارند.

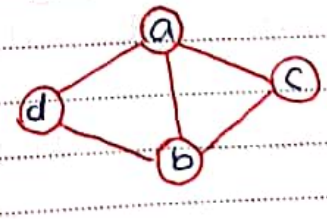
بعد از اینکه یک نود مثل C انتخاب شد، نودهای بعدی که می‌توانند اضافه شوند، باید همسایه‌ی A و C باشند.

پس در ابتدا لیست داریم: Q ← clique فعلی را نشان می‌دهد R ← کاندیدها را نشان می‌دهد، کاندیداهایی

که ممکن است به clique ما اضافه شوند تا آن را توسعه دهند تا جایی که به یک maximal clique برسیم.

در ابتدای کار Q و R خالی هستند. همسایه‌های نود u را با $\Gamma(u)$ نشان می‌دهیم.

①	②
$Q = \{a\}$	$\{a,b\}$
$R = \{b,c,d\}$	$\{b,c,d\} \cap \Gamma(b) = \{c,d\}$



③	④
$Q = \{a,b,c\}$	backtrack $\{a,b,d\}$
$R = \{d\} \cap \Gamma(c) = \{\}$	$\{c\} \cap \Gamma(d) = \{\}$

و تا maximal_clique پیدا کردیم

Scanned with CamScanner

بهمن | ۲۱ | 10 Feb 2022
پنجشنبه | ۸ رجب ۱۴۴۳

Q current clique
R candidate vertices

```
Expand (R, Q)
while R ≠ {}
  p= vertex in R
  Qp= Q U {P}
  Rp = R ∩ Γ(p)
  if Rp ≠ {} ، Expand (Rp, Qp)
    else: output Qp
  R= R-{P} ⟶ backTracking
```

شبه کد الگوریتم

expand با توسعه‌ی (R, Q) به این صورت است:

تا زمانی که R تهی نیست یک نود p از R انتخاب کن

و آن را به Q اضافه کن و R را آپدیت کن به این صورت که R جدید

فی ستون اشتراک R قبلی با همسایه های p . اگر R جدید تهی شد

زیرا باید clique فعلی را توسعه میدهیم ولی اگر تهی شد باید این Q که maximal است را به خروجی بفرستیم

یک مشکلی که وجود دارد این است که یک clique را ممکن است چند بار تولید کنیم. چگونه می‌توان جلوی آن را گرفت؟ راه حل آن این است که از ترتیب lexicographically استفاده کنیم برای تولید clique ها و extention آن‌ها و نود آغازین. مثلا اگر از b شروع کردیم a d غیرقابل قبول است. ترتیب الفبایی رعایت نشده.

همیشه نودهایی که از R انتخاب می‌کنیم باید نودی باشند که کمترین id را داردون از نودهایی که در clique است id بزرگتری دارد.

(یک راه حل هم اینه و تمام کلیک‌ها را تولید می‌کنید با ترتیب الفبایی بفرستید خارج اگر تکرار تولید شده‌اند آن هم در ترتیب الفبایی است و کلیک تو)

Community Structure

بهمن | ۲۲ | جمعه
11 Feb | ۹ رجب

در حالتی که Community ها با هم overlap دارند ساختار Community چگونه خواهد بود؟

(هسته پیرامونی)

Core_periphery

بعضی Community ها overlap دارند و Nested (تودرتو) هستند ساختار هستند. یک هسته داریم که یک گراف dense است و در اطراف آن periphery، یک سری گراف‌های خیلی کوچک که خودشان dense هستند ولی با تعداد خیلی کمی یال به هسته متصل شده‌اند.

خود هسته هم به صورت تودرتو همین ساختار را دارد یعنی یک هسته‌ی dense دارد و پیرامون آن یه سری زیرگراف dense که با یال کمی به هسته متصل‌اند.

پس یک Nested Core_periphery دارید.

Scanned with CamScanner

Network Community profile

۱۴۰۰

conductance معیار ... می خواهیم بررسی کنیم یک مجموعه از نودها مثل S تا چه اندازه کلاستر خوبی است. معیار

را داشته باشیم: تعداد لینک هایی که یک طرفشان داخل S است و یک طرفشان در کل شبکه تقسیم

بر حاصل جمع درجه ی نودهایی که در S هستند:

$$\phi(S) = \frac{|\{(i,j) \in E ;\ i \in S ,\ j \notin S\}|}{\sum_{s \in S} d_s}$$

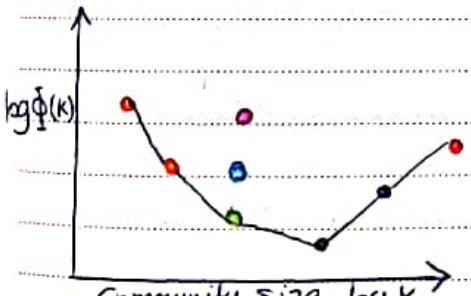
هرچقدر کمتر باشد کیفیت Community بیشتر است

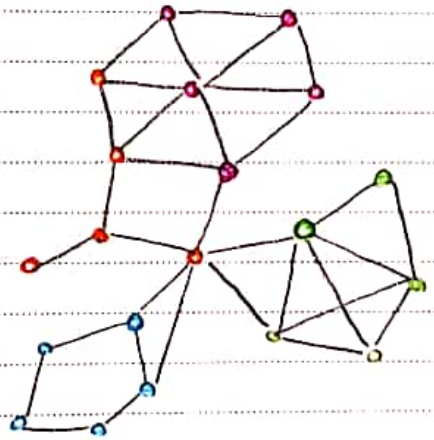
Network Community profile plot (NCP) ← یک پلات است که در کل الگوریتم ها و ساختار اجتماع به ما کمک می کند

k های مختلف را برای تعداد نودها در نظر می گیریم. برای تک تک Community های با اندازه k مقدار

Conductance را حساب می کنیم برای هرکدام که min شد می شود $\Phi(k)$:

$$\Phi(k) = \min_{S \subset V, |S|=k} \phi(S)$$





یکی از کاربردهای آن در مقایسه ی الگوریتم های مختلف است.

یکی از کاربردهای دیگرش برای کلیل Community های خودشبکه است. برای یک k خاص تک تک اجتماع ها

گراف که اندازه شان k است در نظر گرفته و conductance را برای آن ها حساب می کنیم و اونی که min شد

برابر $\Phi(k)$ است. برای مقایسه الگوریتم ها اینگونه عمل می کنیم: به ازای هر k ای، اجتماع هایی با اندازه k

که توسط آن الگوریتم یافت شده اند را در نظر می گیریم و معیار conductance شان را حساب کرده و هرکدام min شد

NCP آن الگوریتم را تشکیل می دهد. هر الگوریتمی که خط NCP اش پایین تر باشد بهتر است.

برای کلیل خودشبکه بیشتر الگوریتم های خوب را اعمال می کنیم، هرکدام پیشرفت کلاسترهای می سازند. اجتماع خروجی های آن ها را در نظر می گیرم

Scanned with CamScanner

دوره برای یک k خاص بررسی می‌کنید چند کلاستر دارید. هرکدام‌شان ممکن است توسط یک الگوریتمی تولید شده باشد.
سپس $\Phi(k)$ را ثبت می‌کنید و نمودار NCP به دست می‌آید که هرچقدر مقدارش کمتر باشد نشان‌گر این است که از نظر Community well_structured ترهستش.

پارامتری دیگر به این صورت بود که NCP به صورت یک خط شیب دار است به سمت پایین که شیب این خط با $\frac{-1}{dim(G)}$ مناسب است. (dim = dimension)

اینها و از آن یک مطالعه گسترده روی گراف های واقعی صورت گرفت و نتایج متعددی حاصل شد. مشاهده شد که NCP از ۲ بخش تشکیل شده است: ① downward ② upward

ابتدا تا یک جایی با افزایش k کلاسترهای بهتری پیدا می‌کنیم، بعد از آن k (اندازه کلاسترها) که بیشتر می‌شود کیفیت کلاسترها کمتر می‌شود

توضیح قسمت upward

وقتی اندازه خوشه‌ها از یک حدی بزرگتر می‌شوند تعداد لینک‌هایی که با بخش‌های خارج پیدا می‌کنند خیلی سریعتر از تعداد لینک‌هایی که داخل خودشان دارند رشد می‌کند. در نتیجه کیفیتشان پایین می‌آید.

چون کلاسترهای کوچک با هم ترکیب شده‌اند این کلاستر بزرگ را ساخته‌اند و کلاسترهای واقعی نیستند.

توضیح قسمت Downward

شبکه‌های پیچیده دنیای واقعی یک core دارند و یک سری کلاسترهای خیلی کوچک. این کلاسترهای کوچک کلاسترهای خیلی خوبی هستند و بهشدت dense هستند و با لینک کمی به core وصل‌اند.

در ابتدای کار شما این چنگک‌ها (کلاسترهای کوچک) را پیدا می‌کنید. تا زمانی که بزرگترین چنگک را پیدا نکنید کیفیت بهتر و بیشتر می‌شود. چنگک‌های بزرگ density داخلی بالایی دارند و با لینک کمی به core وصلند

بنابراین تا زمانی که همه این چنگک‌ها پیدا شوند کیفیت خوشه‌ها کمتر می‌شود.

Scanned with CamScanner

۱۴۰۰

سوال: اگر این کلاسترهای خوب (چگال‌ها) را حذف کنیم نمودار NCP چه تغییری می‌کند؟
هیچ تغییری نمی‌کند! چون ساختار Nested داریم، خود هسته از یک هسته و تعدادی چگال تشکیل شده است (فقط یک مقداری کیفیت خوشه‌ها کمتر می‌شود)

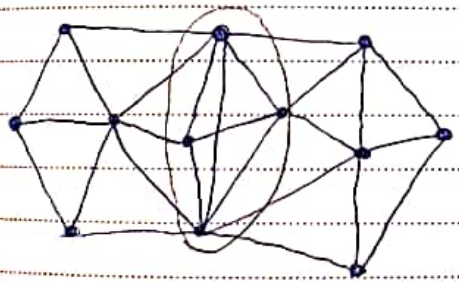
جلسه چهارم - ۱۴۰۰/۹/۲

در ساختار Nested هرچقدر داخل‌تر می‌رویم هسته‌ها dense‌تر می‌شوند

الگوریتم‌های اولیه‌ای که برای کشف Communityهای overlaping پیشنهاد شدند به‌سری فرض‌ها
ضمنی دارند: اینکه قسمت overlaping ما چگالی‌اش کمتر است و اسپارس است (CPM این فرض‌ها)

یک مثال از CPM با kهای مختلف و حدسی که تشخیص می‌دهد:

چرا الگوریتم CPM فرض می‌کند قسمت مشترک چگالی کمتری دارد؟ خروجی الگوریتم CPM به مقدار k بستگی دارد:



اگر $k=3$ ← کل گراف را یک Community می‌گیرد
چون هر یزر به یک clique سه‌تایی تعلق دارد و

اگر $k=4$ ← فقط قسمت ۴ ضلعی وسط را به عنوان یک community تشخیص می‌دهد.

پس الگوریتم‌های اولیه وقتی که قسمت overlaping چگال‌تر باشد درست کار می‌کند.

سوال: شبکه‌های پیچیده‌ای واقعی قسمت مشترک dense‌تر است یا اسپارس‌تر؟
مطالعات نشان داده که dense‌تر هست. با افزایش تعداد Communityهای مشترکی که دو تا نود بهش تعلق دارند احتمال اینکه بینشان یال بیاد و وصل باشند بیشتر می‌شود در نتیجه قسمت مشترک dense‌تر می‌شود

در مدل Nested هم در واقع به این شکل است که چگال‌های ما ادامه دارند و در قسمت core اشتراک دارند به همین دلیل core چگال‌تر است.

پس الگوریتم‌های اولیه خوب کار می‌کنند و الگوریتم‌های دیگری ارائه شدند:

Scanned with CamScanner

بهمن | ۲۶ | 15 Feb 2022
سه‌شنبه | ۱۳ رجب ۱۴۴۳

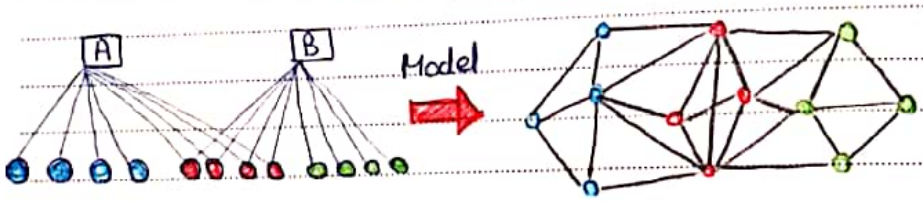
الگوریتم AGM (Community-Affliation Graph Model)

به دو صورت می‌تواند استفاده شود: ① Generative Model (مدل مولد) ← برای تولید گراف‌هایی که از نظر ساختار Community های overlaping شبیه گراف‌های دنیای واقعی هستند

پارامترهایی که می‌گیرد: ① تعداد نودها (V) ② تعداد Community ها (C) ③ Membership (M) ④ Function (تابع عضویت)

یک نود را به ۱ یا ۰ یا بیشتر Community مَپ می‌کند

برای هر Community مثل C یک احتمال P_c داریم که این را نشان می‌دهد که هر دو نود در آن Community با چه احتمالی به هم متصل اند (یا چه احتمالی یک یال به وجود می‌آید)



دو نود u و v را در نظر بگیرید، به ازای هر اجتماع C که هم u و هم v به آن تعلق دارند با احتمال P_c، u و v را به هم وصل می‌کنیم. این کار را برای جوامع مختلف که u و v دارند به طور مستقل در نظر می‌گیریم

$$P(u,v) = 1 - \prod_{c \in M_u \cap M_v} (1 - P_c)$$

پس هر چقدر Community های مشترک بیشتری داشته باشند (u و v)

↓

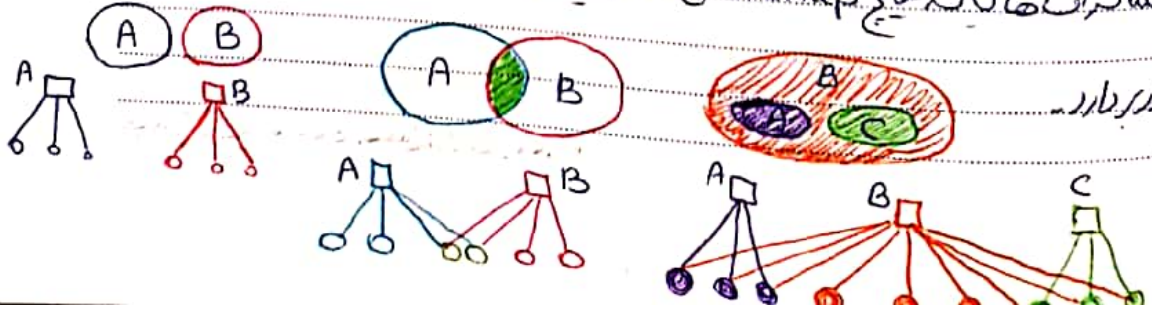
احتمال اینکه u و v یال داشته باشند

احتمال اینکه به هم متصل بشوند بالاتر است.

$P(u,v)$ ← احتمال اینکه بین u و v یال داشته باشیم = ۱ منهای احتمال اینکه بین u و v یال نداشته باشیم

به ازای هر Community مشترک مثل C احتمال اینکه u و v یال نداشته باشند $1 - P_c$ است و این احتمال‌ها را در هم ضرب می‌کنیم

این مدل یک تنوع گسترده‌ای از گراف‌ها را از نقطه نظر وضعیتی که Community ها نسبت به هم دارند، تولید می‌کند. مثلا گراف‌هایی که هیچ overlap ای ندارند یا overlap دارند با یک Community تکه‌ای Community را در بر دارد.



Scanned with CamScanner

تسک دوم AGM: یک گراف دنیای واقعی داریم، چگونه از AGM برای کشف Community های overlaping گراف دنیای واقعی استفاده کنیم؟

فرض می‌کنیم گراف دنیای واقعی توسط مدل AGM تولید شده است، پارامترهای آن را تخمین می‌زنیم.

با تخمین پارامترها، Community هایی که بیشترین احتمال در این گراف پیچیده یافت می‌شوند شبکه‌هایی dense تر هستند.

$B(V, C, M, \{P_c\})$ ← می‌توانیم هرزمان Set کنیم

V ← همان رئوس گراف دنیای واقعی

دو تا پارامتر چالش برانگیز داریم: ① Membership function ② P_c ← احتمالی که density هر اجتماع را مشخص می‌کند

روشی که برای تخمین پارامترها از آن استفاده می‌کنیم: max log likelihood (MLE)

گفتیم تا زمانی که چیزی مشاهده نشده اصطلاح "احتمال" را به کار می‌بریم و وقتی مشاهده شد "likelihood" را به کار می‌گیریم.

ایده کلی MLE: فرض می‌کنیم یک شبکه در دنیای واقعی مشاهده شده بنابراین حالا که مشاهده شده احتمالش خیلی زیاد بوده است. فرض می‌کنیم گراف ما توسط مدل AGM تولید شده، احتمالش هم زیاد بوده پس میایم likelihood آن را maximize می‌کنیم.

MLE همیشه از فرض استقلال هم استفاده می‌کنیم. دنیای ما مجموعه‌ای از نودهاست که بین آن‌ها یال داریم و مجموعه‌ای از نودها که بین آن‌ها یال نداریم.

$$\arg\max_{B} P(G|B) = \prod_{(i,j)\in E} p(i,j) \prod_{(i,j)\notin E} (1 - p(i,j))$$ → objective function

$$p(i,j) = 1 - \prod_{c \in M_i \cap M_j} (1 - P_c)$$

پس ما یک تابع هدف داریم که P_c در آن وجود دارد که density هر کلاستر را مشخص می‌کند و Membership function هم داریم در این تابع هدف که آن را هم می‌خواهیم estimate کنیم. با maximize کردن این تابع هدف، P_c و Membership function، learn می‌شوند.

Scanned with CamScanner

بهمن ۲۸ 17 Feb 2022
پنجشنبه ۱۵ رجب ۱۴۴۳

از Stochastic GD استفاده می‌کنیم. 2 خانواده از پارامترها داریم ← ① پارامترهای مربوط به P_c

② پارامترهای مربوط به Membership function ها

حالا که دو نوع متفاوت از پارامترها را داریم بهتر است این‌ها را جدا از هم در نظر بگیریم.

۱) ابتدا P_c را برای همه‌ی Community ها مقدار fix در نظر می‌گیریم، بعد از Stochastic GD استفاده می‌کنیم تا Membership function ها را بیابیم.

۲) در مرحله‌ی دوم، M ای که یافته‌ایم را fix در نظر می‌گیریم و P_c را برای اجتماع‌های مختلف learn می‌کنیم (با Stochastic GD)

پیش‌بینی پیوند (Link prediction)

به طور سطح بالا ← پیش‌بینی می‌کنیم در آینده‌ای نزدیک بین دو تا نودی که الان به هم متصل نیستند آیا لینکی به وجود می‌آید یا نه؟

کاربردهای زیادی دارد: suggestion ها در facebook، linkedIn و ...

بهمن ۲۹ جمعه 18 Feb ■ ۱۶ رجب

در Recommendation کاربرد دارد.

تعریف رسمی: فرض کنید گرافی را در بازه‌ی زمانی $G[t_0, t_0']$ داریم. این گراف را به عنوان ورودی به ما داده‌اند. می‌خواهیم پیش‌بینی کنیم که در این گراف ورودی بین چه نودهایی لینک تولید می‌شود که در زمان آینده نزدیک به هم متصل خواهند شد و به هم متصل نیستند.

پس برای ارزیابی الگوریتم‌مان این‌گونه عمل می‌کنیم:

گراف را در بازه‌ی زمانی بعدی $G[t_1, t_1']$ در نظر می‌گیریم. به گراف قبلی یک سری یال‌ها اضافه شده است. فرض کنید تعداد این یال‌های جدید n باشد. از بین مدتی که الگوریتم‌ها تولید کرده‌اند n تای اول را برمی‌داریم. حالا دو تا مجموعه‌ی n تایی از یال‌ها داریم. بررسی می‌کنیم که این دو مجموعه چقدر با هم overlap دارند. هرچقدر اشتراک‌شان بیشتر باشد الگوریتم‌ها بهتر عمل کرده است.

Scanned with CamScanner

گاهی وقت‌ها لیست دوم را هم به صورت مرتب شده نگه میدارند مثلا برحسب زمان دوره‌ها و این دو لیست را این طور مرتب شده باهم مقایسه می‌کنند یعنی نه فقط براساس تعداد عناصر مشترک بلکه براساس ترتیب و جایگاه عناصر قبل هم مقایسه را انجام می‌دهند.

بهمن | ۳۰ | 19 Feb 2022
شنبه | ۱۷ رجب ۱۴۴۳

۱۴۰۰ خیلی وقت‌ها به مسئله‌ی Link prediction می‌توانیم به چشم classification نگاه کنیم.

الگوریتم گراف را ورودی می‌گیرد و به ازای هر جفت نودی که به هم متصل نیستند خروجی می‌دهد که این‌ها به هم متصل خواهند شد یا نه. (لیبل + و -)

بدترین گراف دوم را که می‌دهند به عنوان داده‌ی دست است و با معیارهایی که برای اندازه‌گیری دقت classification وجود دارند می‌توانیم دقت الگوریتم را به دست آوریم.

دو دسته‌ی کلی از الگوریتم‌ها وجود دارند: ① الگوریتم‌هایی که از یادگیری ماشین استفاده می‌کنند از نظر محاسباتی efficient هستند و نسبتا ساده‌ترهستند. شباهت یا proximity بین دو نود را حساب می‌کنند و هرچقدر این شباهت بیشتر باشد احتمال اینکه این نودها کم لینک بهم بدهند بیشتر است.

② از یادگیری ماشین استفاده می‌کنند و میزان شباهت بین دو نود را یاد می‌گیرند و از این proximity یا گرفته شدن برای پیش‌بینی لینک استفاده می‌کنند.

یک دسته‌ی سوم هم اخیرا معرفی شده ← روش‌های مبتنی بر embedding

link prediction در گراف‌های dense راحت‌تر است و به دقت‌های بالایی می‌رسد. چالش اصلی در مورد گراف‌های sparse است.

نودهای که درجه‌شان کوچکتر از ۲ هست را در نظر نمی‌گیریم در link prediction. چون این اینفلوئنس‌ها برای اینگونه نودها اصلا خوب کار نمی‌کنند. به همین دلیل گراف core را در نظر می‌گیریم (درجه ۲ حذف شده)

چون این روش‌ها خیلی ساده هستند performance درجه بالایی ندارند اما از نظر محاسباتی کم هزینه هستند

متودولوژی کلی: یک جفت نود که به هم متصل نیستند را در نظر می‌گیریم. با استفاده از یک معیار proximity میایم یک value برای ارتباط یا تردیکی آن دو نود پیدا می‌کنیم. $(x,y) \rightarrow c(x,y)$

مثلا $c(x,y)$ می‌تواند تعداد همسایه‌های مشترک x و y را نشان بدهد.

Scanned with CamScanner

در میانگین جفت‌هایی که کمم متصل نیستند را بر اساس معیار proximity مرتب می‌کنیم، به صورت نزولی. قطعا جفت نودهایی را انتظار می‌کنیم که درجه هرکدام از نودهایشان نزدیک مساوی ۳ باشد.

بعد در نهایت ۱۰ تا از اول این لیست را با یال‌های جدید گراف مقایسه می‌کنیم و میزان overlap را حساب می‌کنیم

معیارهای Proximity

shortest path Length - distance

① Graph distance ← فاصله بین دو تا نود (معمولا منفی فاصله را در نظر می‌گیریم)

② Common neighbors ← تعداد همسایه‌های مشترک: $|\Gamma(x) \cap \Gamma(y)|$ هرچه بیشتر بهتر

③ Jaccard's Coefficient ← تعداد همسایه‌های مشترک را نرمالایز کرده.

$$\frac{|\Gamma(x) \cap \Gamma(y)|}{|\Gamma(x) \cup \Gamma(y)|}$$

(تقسیم بر اجتماع همسایه‌ها کرده)

④ Adamic / Adar

$$\sum_{z \in \Gamma(x) \cap \Gamma(y)} \frac{1}{\log |\Gamma(z)|}$$

معیار دوم به هر همسایه مشترک وزن اعمال کرد. مثلا ممکن به همسایه مشترک متقاعین دو نود x و y را به عنوان همسایه داشته باشند ولی به همسایه مشترک یک اعضای نا همسایه داده که x و y هم جزو نا همسان. در حالت اول وزن این همسایه مشترک باید بیشتر باشه پس به یک همسایه مشترک وزن متناظر با داروی تعداد همسایه‌هایش می‌دهیم (معمولا log هم می‌گیرند) چون متقاعین تمایل دارند دوست با دارو اعمال اینکه آن‌ها با هم آشنا کنند بیشتر از حالت دوم است.

⑤ preferential Attachment ← ضرب تعداد همسایه‌های x و y

$$|\Gamma(x)| \cdot |\Gamma(y)|$$

⑥ pageRank

$$r_x(y) + r_y(x)$$

یک pagerank personalized با شروع از x اجرا می‌کنیم، score مربوط به y را به دست می‌آوریم و برعکس. بعد این دو تا score را با هم جمع می‌کنیم.

قبلا گفتیم که در personalized pagerank، jumping هم به نود آغازین داریم.

⑦ Katz

مسیرهای با طول مختلف بین دو تا نود را در نظر می‌گیریم. هرچند تعداد این Simple_path ها بیشتر باشد proximity دو تا نود بیشتر است.

weighted Katz ← به مسیرهای مختلف با طول‌های مختلف وزن‌های متفاوت می‌دهیم.

Scanned with CamScanner

⑧ low_rank inner product

برای هر بردار یک بردار استخراج می‌شود. میزان proximity این دو تا بردار می‌شود شباهت این دو بردار.
یکی از معیارهای شباهت دو تا بردار dot product (ضرب داخلی) است.

نکته: در اکثر بررسی‌ها، معیار adamic/adar بهترین عملکرد را دارد.

⑨ unseen bigram

ایده آن از NLP گرفته شده است. با یک معیاری (مثلا معیارهایی که تا این جا گفتیم) بین x و یک نودی که به x proximity بیشتری دارند را پیدا می‌کنیم. بعد proximity پیدا می‌کنیم. بعد top k نودی که به y proximity شان از همه بیشتر است را هم پیدا می‌کنیم. بعد proximity های این دو تا جوره را با هم مقایسه می‌کنیم. هرچقدر اشتراکشان بیشتر باشد x و y به هم شبیه‌ترند.

⑩ SimRank

یک الگوریتم iterative است که از روی میزان شباهت همسایه‌های x و y، شباهت x و y را حساب می‌کند. (یک رابطه‌ی بازگشتی مثل pagerank دارد)

⑪ hitting time

مثل personalized pagerank است با این تفاوت که یکسری jumping انجام نمی‌دهیم.
با شروع از نود x یک random walk معمولی را می‌کنیم و بررسی می‌کنیم که به صورت expected value بعد از چند تا گام y مشاهده می‌شود. هرچه این مقدار گام کمتر شود شباهت x و y بیشتر می‌شود.
هرچقدر با مقدار گام کمتری y را مشاهده کنیم میزان شباهت y به x بیشتر می‌شود.

Scanned with CamScanner

اسفند | ۳ | 22 Feb 2022
سه‌شنبه | ۲۰ رجب ۱۴۴۳

جلسه نوزدهم — ۱۴۰۰/۹/۷

روش‌هایی که از یادگیری ماشین برای Link prediction استفاده می‌کنند: در این روش‌ها ما با هفت مبنا نودها یاد گرفته می‌شود

(SRW) Supervised Random walks for Link prediction

مثال facebook: روش‌های اکتشافی تا حدخوبی جواب‌های خوبی دارند. مثلا در مورد facebook بررسی‌ها نشان داده که با افزایش حاصلضرب بین دو نفر احتمال دوست شدنشان بیشتر می‌آید (تعداد دوست‌هایی که دارند) که نشان می‌دهد فاصله معیار خوبی است برای Link prediction. هم‌چنین با افزایش تعداد همسایه‌های مشترک، احتمال با تعداد اندازه‌ای که با هم دوست می‌شوند بیشتر می‌شود، و معیار خوبی است. ۹۲٪ دوستی‌ها friend of friend است.

تعریف دقیق task: یک گراف داریم و یک نود در آن مشخص شده، می‌خواهیم بیستر از افراد را به این نود نشان بدهیم که احتمال می‌دهیم دوست این نود شوند. برای این کار از دنیای یادگیری با نظارت (supervised Machine learning) استفاده می‌کنیم.

ما گراف‌مان را در ۳ تا snapshot (بازه) داریم: ① یک گراف داده‌اند و یک نود را در آن مشخص کرده‌اند

② در بازه دوم یک سری example مثبت و منفی به ما می‌دهند. نودهایی که با آن‌ها لینک برقرار کرده example‌های مثبت و نقیه نودها example منفی‌اند

③ در بازه زمانی سوم متدی را که با استفاده از بازه زمانی اول و دوم توسعه داده‌ایم، ارزیابی می‌کنیم. در این بازه نودهایی که با S لینک داده‌اند مشخص می‌شوند بعد بررسی می‌کنیم لیست پیشنهادی ما با این نودها چقدر overlap دارد.

در حقیقت با استفاده از گرافی که در snapshot اول به ما داده‌اند example‌های مثبت و منفی که در بازه دوم داریم یک مدل می‌سازیم و این مدل یک لیست مرتب ایجاد می‌کند: $\{d_1, \dots, d_K\}$

تعداد snapshot سوم بررسی می‌کنیم این لیست چقدر با آن‌هایی که واقعا با S لینک داده‌اند، overlap دارد.

Scanned with CamScanner

الگوریتم چگونه این لیست را مشخص می‌کند؟

به هر نود یک Score براساس میزان شباهتش با نود S نسبت داده می‌شود. این Score ها با یک مکانیزم supervised مشخص می‌شود. بدد K تا نودی که بیشترین Score را دارند d_1 تا d_K را تشکیل می‌دهند و به خروجی فرستاده می‌شوند

در واقع در این جا شباهت نودها را با این گریم هان الگوریتم هایی که قبلا داشتیم

مثلا personalized pr را تغییرش می‌دهیم، یک جزء یادگیری هم به آن اضافه می‌کنیم که عملکردش کمتر شود.

در ابتدا گراف را وزن دار می‌کنیم، به هر یال یک strength نسبت می‌دهیم.

strength یک یال شباهت دو نودی که دو سر آن قرار دارد را نشان می‌دهد. این شباهت را براساس ویژگی هایی که هر نود دارد تعیین می‌کنیم و به هر یال یک وزن می‌دهیم. → features

سپس از نود آغازین یک personalized pagerank اجرا می‌دهیم ولی به صورت وزن دار یعنی در هر مرحله با یک احتمالی به نود آغازین می‌پریم و با $1-\alpha$ همان احتمال یکی از نودهای خروجی را متناسب با وزن یالش انتخاب می‌کنیم و به آن می‌رویم.

در اثر اجرای این عمل هر نود یک score ای پیدا می‌کند و آن هایی که امتیاز بیشتری دارند را به خروجی می‌فرستیم

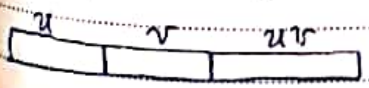
← چگونه گراف را وزن دار کنیم؟

وزن هر یال در واقع یاد گرفته می‌شود. یک تابع f داریم که یک بردار پارامتر دار ← β . وروری این تابع یک جفت نود و خروجی اش یک عدد است.

$$a_{uv} = f_\beta(u,v) = \exp\left(-\sum_i \beta_i \cdot x_{uv}[i]\right)$$

این عدد خروجی شباهت جفت نود وروری را نشان می‌دهد. a_{uv} خروجی f است.

برای جفت u و v یک بردار x_{uv} می‌سازیم: این بردار از چسپاندن سر هم قرار دادن فیچرهای u، فیچرهای v و فیچرهای یالی که u و v را به هم وصل می‌کند، به دست می‌آید.



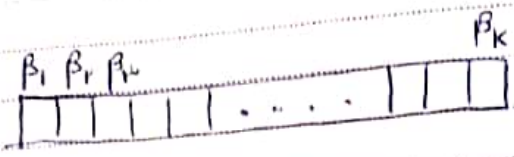
β یاد گرفته می‌شود. β به هرکدام از خانه های x_{uv} یک وزن اختصاص می‌دهد و شباهت u و v را می‌سنجد.

به توان جمع وزن دار خانه های بردار x_{uv}.

Scanned with CamScanner

اسفند | ۵ | 24 Feb 2022
پنجشنبه | ۲۲ رجب ۱۴۴۳

β ← یک بردار پارامتر دارد



حالا که وزن یال‌ها مشخص شده Random walk with restart را به صورت وزن‌دار اجرا می‌کنیم.

سوال: پارامتر β را چگونه یاد بگیریم؟

روش: example های t تا - به همین دلیل به ما داده می‌شود که بتوانیم پارامتر β را یاد بگیریم.

personalized pageRank

ماتریس انتقال

$$Q'_{uv} = \begin{cases} \frac{a_{uv}}{\sum_w a_{uw}} & if\ (u,v) \in E \\ 0 & otherwise \end{cases}$$

→ اگر از u به v یال نباشد با احتمال صفر از u به v می‌رویم

اگر از u به v یال وجود داشته باشد با احتمال متناسب با وزن بین u و v از u به v می‌رویم. هر یک همسایه خروجی نود u است. Σ روی همسایه‌های نود u (وزن‌ها) داریم (خروجی).

اگر teleport (jump) داشته باشیم:

۶ | اسفند | جمعه | ۲۳ رجب | 25 Feb

$$Q_{ij} = (1-\alpha) Q'_{ij} + \alpha 1\ (j=S)$$

با احتمال α - یکی از نودهای خروجی را انتخاب می‌کنیم و با احتمال α به نود آغازین S می‌رویم. اگر دو همان نود آغازین S باشد با احتمال α به S می‌رویم. 1 نشان می‌دهد اگر $S=j$ باشد 1 است وگرنه 0 است.

$$P = P^T Q$$

رابطه‌ی بازگشتی این personalized pagerank وزن دار به این صورت است:

همان روش iterative که تا حالا داشتیم این رابطه‌ی بازگشتی حل می‌شود ولی Q در نتیجه P تابعی از β هستند و β را باید یاد بگیریم و نیاز به تابع هدف داریم. به این منظور example هم داریم.

positive examples: $D = \{d_1, \dots, d_k\}$ Negative examplex: $L = \{otherwise\}$.

$$\min_{\beta} F(\beta) = \|\beta\|^2$$

ابتدا β را طوری learn می‌کنیم که $F(\beta)$ ، min شود:

Scanned with CamScanner

این یک تکنیک خیلی استاندارد در ML است. برای اینکه از overfitting جلوگیری شود هرچقدر مدل learn می‌شود ساده‌تر باشد و وزن فیچرهای مختلف کم باشد خیلی از فیچرها وزنشان صفر باشد احتمال اورفیت کم می‌شود. در نتیجه سعی می‌شود که تا جایی که امکان دارد همه‌ی بردار β کم باشد. برای این کار $\min \lambda \|\beta\|^2$ می‌کنند.

اما یک سری قید هم داریم که همان example های $+t$ هستند. β باید به گونه‌ای باشد که به example مثبت امتیاز بیشتری از example های منفی بگیرند. پس برای هر جفت example که یکی $+t$ دیگری - است این شرط را می‌گذاریم که score مربوط به Negative example کوچکتر از score مربوط به positive example باشد:

$$\forall d \in D, l \in L : p_l < p_d$$

مسئله‌ای که وجود دارد این است که ممکن شرط‌های ما اینقدر زیاد باشند که اصلا جواب نداشته باشد و هیچ مقداری برای β یافت نشود. میایم مقادیر قیدها را soft می‌کنیم:

$$\min_{\beta} F(\beta) = \sum_{d \in D, l \in L} h(p_l - p_d) + \lambda \|\beta\|^2$$

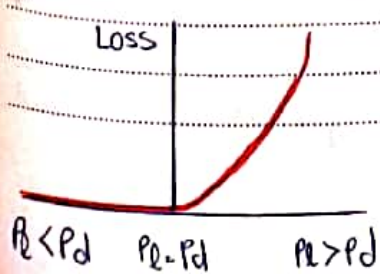
در واقع شرط سخت در سخت قرار می‌دهیم که حتما باید برآورده شود.

λ یک ابرپارامتر است که خودمان set می‌کنیم.

این تابع هدف از دو قسمت تشکیل شده: یک قسمتش همان تابع هدف قبلی است $\|\beta\|^2$ ، λ را هم خودمان مشخص می‌کنیم که اهمیت این دو قسمت را نسبت به هم تعیین می‌کند.

قسمت جدیدی که اضافه شده به ما می‌گوید اگر قیدها را بیایید بگذاریم حتما برآورده شوند این اجازه را می‌دهیم که قیدها برآورده نشوند ولی اگر هرکدام از قیدها برآورده نشوند یک هزینه‌ای پرداخت شود. به ازای هر d و یک example مثبت است و هر l که example منفی است اگر $p_l < p_d$ نبود یک هزینه‌ای پرداخت شود.

$h(x) = 0$ if $x < 0$, or x^2 else



Scanned with CamScanner

حالا باید با minimize کردن objective function، β را یاد بگیریم. برای این کار از stochastic GD استفاده می‌کنیم:

۱) β را با یک مقدار دهی اولیه (دلخواه) می‌کنیم. مثلا همه‌ی وزن‌ها را ۱ قرار می‌دهیم.

۲) وارد فاز iterative می‌شویم. فرض کنید در تکرار tام مقدار β را داریم و می‌خواهیم مقدار آن را در تکرار t+1ام بیابیم. مشتق با گرادیان تابع هدف نسبت به β را حساب می‌کنیم و از $\beta^{(t)}$ کم می‌کنیم.

$$\beta^{(t+1)} = \beta^{(t)} - \eta \frac{\partial F(\beta^{(t)})}{\partial \beta}$$

η یک ثابت است ← learning rate، گاهی اوقات ثابت است و گاهی در iteration‌های مختلف تغییر می‌دهند. می‌توان به عنوان یک پارامتر به آن نگاه کرد.

چالشی که باقی مانده است، محاسبه‌ی مشتق تابع هدف نسبت به β است.

$$\frac{\partial F(\beta)}{\partial \beta} = \sum_{l,d} \frac{\partial h(p_l - p_d)}{\partial \beta} + 2\lambda\beta = \sum_{l,d} \underbrace{\frac{\partial h(p_l-p_d)}{\partial(p_l-p_d)}}_{\text{این قسمت راحته}} \left(\underbrace{\frac{\partial p_l}{\partial \beta} - \frac{\partial p_d}{\partial \beta}}_{\text{سخته}} \right) + \underbrace{2\lambda\beta}_{\text{ساده}}$$

$h(x) = \max\{x, 0\}^2$ به طور کلی مشتق بردار p نسبت به β محاسبه‌ی این مشکل هست

این روابط را داشتیم:

$$p = p^T Q \Rightarrow p_u = \sum_j p_j Q_{ju}$$

حالا باید $\frac{\partial Q_{ju}}{\partial \beta}$ ، $\frac{\partial p_j}{\partial \beta}$ را

$$\frac{\partial p_u}{\partial \beta} = \sum_j Q_{ju} \frac{\partial p_j}{\partial \beta} + p_j \frac{\partial Q_{ju}}{\partial \beta}$$

محاسبه کنیم. محاسبه‌ی اولی ممکن است چون این روابط را داریم:

$$Q_{uv} = \begin{cases} \frac{a_{uv}}{\sum_w a_{uw}} & \text{if } (u,v) \in E \\ 0 & \text{otherwise} \end{cases}$$

$$a_{uv} = f_\beta(u,v) = \exp\left(-\sum_i \beta_i \cdot x_{uv}[i]\right)$$

حالا باید $\frac{\partial p_j}{\partial \beta}$ را محاسبه کنیم. اگر به جای مشتق بردار p نسبت به β، x قرار دهیم در رابطه‌ای که داشتیم، این رابطه حاصل می‌شود:

$$x = Q \cdot x + z$$

Scanned with CamScanner

$x = Q \cdot x + Z$

بردار برای مشتق p نسبت به β یک رابطه‌ی بازگشتی داریم:

حل این رابطه‌ی بازگشتی با استفاده از iteration امکان‌پذیر است.

یعنی برای محاسبه‌ی مشتق p نسبت به β به صورت iterative آن را حل می‌کنیم. یک روش عددی برای محاسبه‌ی مشتق p نسبت به β استفاده می‌کنیم:

① یک مقدار اولیه‌ی دل‌خواه برای $\frac{\partial p^{(0)}}{\partial \beta}$ در نظر می‌گیریم. مثلا مقدار هر خانه را ۱ قرار می‌دهیم.

② بعد می‌آییم p در تکرار $t+1$ را با استفاده از رابطه‌ی بازگشتی update می‌کنیم:

$$\frac{\partial p^{(t+1)}}{\partial \beta} = Q \cdot \frac{\partial p^{(t)}}{\partial \beta} + \frac{\partial Q_{ju}}{\partial \beta} \cdot p$$

نکته: این فرآیند iterative که یک روش عددی برای محاسبه‌ی p است در دل فرآیند iterative قبلی که مربوط به روش SGD است قرار دارد و با هم فرق دارند.

توضیح سطح بالاتر: ابتدا یک β دل‌خواه انتخاب می‌کنید $\beta^{(0)}$

بعد فرض می‌کنید می‌خواهید از تکرار t به $t+1$ بروید. $\beta^{(t)}$ را دارید و می‌خواهید $\beta^{(t+1)}$ را بیابید. با استفاده از $\beta^{(t)}$ ورودی q را حساب می‌کنید (q_{uv})، بعد Q را حساب می‌کنید که با وزن یال‌ها محاسبه می‌شود. بعد هم p را حساب می‌کنیم.

بعد دوباره یکی از این مقادیر را استفاده می‌کنید و مشتق تابع هزینه نسبت به β را محاسبه می‌کنید. (Q و $p^{(t)}$ و $\beta^{(t)}$ را دارید)

بعد با استفاده از رابطه‌ی update، β را به روز می‌کنیم $\beta^{(t+1)} \leftarrow \beta^{(t+1)} = \beta^{(t)} - \eta \frac{\partial F(\beta^{(t)})}{\partial \beta}$

نکته: هزینه‌ی محاسباتی این روش بالا است ولی این هزینه را در فاز training می‌پردازیم. در فاز training، β را پیدا می‌کنیم. در فاز تست وزن هر کدام از فیچرها را داریم. کار را وزن‌داری می‌کنیم و weighted personalized pr می‌زنیم. در فاز تست بین این الگوریتم و personalized pagerank تفاوت چندانی وجود ندارد.

Scanned with CamScanner

SRW
مثال از به کارگیری الگوریتم در Face Book

فیس بوک اسپند را انتخاب کردند. تعداد نودها: 174000
Avg. Degree: 168 ← نودها به شدت باهم در ارتباط اند
تعداد friend per month ای که به ازای هر نفر اضافه می شود:

(جزئیات مثال در اسلاید)
positive examples = {دوستهایی که نود S در Nov 2009 پیدا کرده} = D
Negative = {نودهایی که S بهشون لینک نداده در این بازه} = L

با چون تعداد L خیلی بالایی شود، به 3 Friend of Friend دوری می شود به تعداد ۲۰،۰۰۰.
یعنی friend of friend نودهایی که دوست دوست این نود هستند ولی با آن دوست نشده اند.
برای نیچرها برای نودها و یال ها دسته بندی گرفته شده از Concat شبکه آنها بردار به دست می آید
precision at top 20 ← معیار ارزیابی
SRW بهبود اندکی نسبت به روش های دیگر دارد

اخیرا روش جدیدی برای Link prediction ارائه شده است. در این روش برای هر نود یک بردار تعبیه یا embedding یاد می گیرند. بعد برای هر جفت نود، بردارهای آن ها را با یک عملگر جفت می کنند و یک بردار تعبیه برای جفت نود می سازند. مثلا یک راه ضرب Hadamard (ضرب به ضرب) است. روش های دیگری مثل جمع، Concatenation، Xor و ... هم استفاده می شود.

در مرحله سوم این بردارها را به یک دسته بندی می دهند. classifier یاد می گیرد که بین این جفت نودها یال ایجاد می شود یا خیر.
یعنی مسئله Link prediction تبدیل می شود به یک مسئله classification.

در این حالت هم بیای آموزش ما مثل قبل داده می شود، یعنی یک snapshot از گراف.

راه اول: یک گراف که از آن استفاده می کنند و برای هر نود یک تعبیه یاد می کنند و برای جفت نودها هم همینطور
راه دوم: به بیای example مثبت و منفی داده می شود، طبقه بند آموزش داده می شود. به بیای بردار دارید
بعلاوه برچسب هایشان و به طور عادی یک دسته بند را روی این ها آموزش می دهید.

در راه دوم سوم یک گراف داده می شود که میل را روی آن تست می کنیم.
اگر نود جدید اضافه شود در ستش اینه که درباره train کنیم اما می توانیم لینک های یاد گرفته شده را استفاده کنیم و بساخت این نود جدید را با توجه آثارین S حساب کنیم. حتی می توانیم این وزن ها را در یک گراف جدید به کار ببریم که از نقطه نظر نیچرهای نودها و یال ها قبل گراف قبلی است. مثلا فیس بوک اسپند را یاد گرفته ایم و وزن های آن را برای فیس بوک مثلا ندیه کار ببریم.

Scanned with CamScanner

جلسه سیستم — ۱۴۰۰/۹/۹

روی نودهای یک گراف می خواهید یک تسک ML انجام بدهید. سوال این است که چگونه این کار را با دقت بالا انجام بدهیم؟ گفتیم که به Link prediction هم می توان به عنوان classification نگاه کرد.

در خیلی از تسک های ML یک مسئله نیاز است دنبال بشود که نیاز اول آن پیدا کردن فیچرهای مناسب برای ادامه کار است. دیتای که به ما داده می شود معمولا دیتای خام است. باید از این دیتا feature استخراج کنید به طوری که هر دیتا پوینت یک بردار مپ بشود. به این بردار، بردار فیچرهای گوینده این بردارها را در یک ماتریس در کنار هم قرار می دهیم.

بعد این ماتریس را به الگوریتم ماشین لرنینگ می دهید، لرنینگ انجام می شود و یک مدل به دست می آید.

الان دیتای ما یک گراف است، گاهی به صورت دستی می شود انجام داد. مثلا برای هر نود ویژگی های ماتریس، حسنیت، کمیلات و... را در نظر گیریم یا اینکه به صورت اتوماتیک learn کنیم و یک بردار ویژگی به دست آوریم.

یکی representation learning این است که این فرآیند را به صورت اتوماتیک انجام بدهید یعنی از دیتای خام به دیتای ساختار مند به صورت اتوماتیک برسیم. مزایا: هزینه کمتر، کیفیت بیشتر.

(خود همین فرآیند هم یک تسک ماشین لرنینگی باشد)

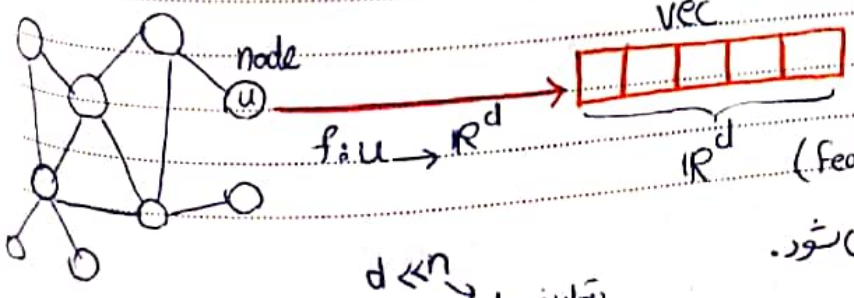
تعریف رسمی

هدف ما این است که به صورت کمینه task_independent و اتوماتیک فیچرها را learn کنیم.

task_independent ← یعنی هم نیست تسک بعدی چه چیزی باشد و بعدا از این بردارها چه استفاده ای می کنید

در نهایت هر نود map می شود به یک بردار در فضای d بعدی.

در تسک representation learning شما باید مثل f را پیدا کنید که ورودی اش یک نود و خروجی اش بردار یا نقطه ای در فضای d بعدی است.



(Feature Representation, embedding)

هر نود به یک فضای ابعاد پایین map می شود.

Scanned with CamScanner

اسفند | ۱۲ | 3 Mar 2022
پنجشنبه | ۲۹ رجب ۱۴۴۳

embedding روش های

۱) تاحالا کلی سعی میشد به صورت distributed این کار را انجام بدهیم (یعنی به صورت هم زمان تابل انجام باشد)

۲) این mapping باید Similarity preserving باشد. یعنی نودهایی که در گراف کنار هم هستند، بعد از embedding هم نزدیک باشند. (این هدف خیلی مهم است)

۳) اطلاعات ساختاری گراف مان Encode شود و در این فرآیند از بین نرود.

تاریخچه روش های Representation Learning

نسل اول: روش هایی که سعی می کردند گراف را به یک ماتریس تبدیل کنند و بعد تلاش می کردند یک شبکه عصبی وجود دارد که یک ماتریس را ورودی می گیرد و خروجی اش یک بردار بازنمایی برای هر ردیف این ماتریس است.

مثلا ماتریس مجاورت یا لاپلاسین را به شبکه عصبی بدهیم و بردارهای بازنمایی استخراج کنیم

مشکلات زیادی دارد از جمله وقتی ترتیبی از نودها را در نظر بگیریم؟ و کار این روش به ترتیب نودها خیلی وابسته است و پیدا کردن یک ترتیب خوب (canonical) مسئله ای سخت است (GI Complete)

۱۳ | اسفند | جمعه
4 Mar ■ ۱ شعبان

Graph Isomorphism Complete است، در زمان چند جمله ای الگوریتمی جواش نداریم. از نظری سازی خیلی پرچالش است. به همین دلیل این روش ها کنار گذاشته شدند. (شکل اسلاید)

نسل دوم: از روی گراف ماتریس مجاورت یا لاپلاسین می سازند و این ماتریس را به عنوان ماتریس ورودی استفاده می کنند. بعد روی این ماتریس می توان SVD، PCA و ... اجرا کنند و dimension های اصلی دیتا را استخراج کنند و همین طور تفاوت هر دیتاپوینت در این ابعاد اصلی.

بعد از اینکه SVD زدید، latent component ها را استخراج کردید، تفاوت این که هر ردیف این ماتریس روی این latent component ها چیزی می شود بازنمایی این نود.

Scanned with CamScanner

مشکلات این روش ها:

۱) time Complexity بالایی دارند، مرتبه‌ی سوم

۲) ماتریس اولیه چگونه ساخته شود؟

۳) اگر تغییر کوچکی در گراف در پنجره درمانریس صورت بگیرد، محاسبات و آپدیت کردن باید از اول انجام شود

و هزینه‌بر بود

۴) بعضی از روش‌ها را نمی‌توان تعمیم داد که مهم است.

نسل سوم: Shallow representation learning _ روش‌های یادگیری بازنمایی کم‌عمق

این روش‌ها با یک تک لایه برای هر نود یک بازنمایی یاد می‌گیرند. بردار بازنمایی هر نود، پارامترهایی است که قرار است learn بشوند. یک مسئله‌ی parameter learning داریم. مثلا از MLE می‌توانیم استفاده کنیم برای حلش.

↓

Maximum Log LikeLihood

بنابراین ایده‌ی کلی این روش‌ها این است که به بردار بازنمایی هر نود به صورت برداری از پارامترها نگاه می‌کنند.

نسل چهارم: روش‌های عمیق یادگیری بازنمایی یا GNN _ Deep Representation Algorithms

این روش‌ها performance خیلی خوبی دارند. خود GNN ها انواع مختلفی دارند.

(مثال گراف Zachary را ببینید) ← خاصیت Similarity preserving دارد

چرا یادگیری بازنمایی و به طور کلی استفاده از شبکه‌های عصبی برای گراف‌ها مشکل است؟

چون گراف‌ها انعطاف‌پذیری بالایی در ساختارشان دارند. این انعطاف‌پذیری خیلی آن‌ها را rich کرده از این نظر که می‌توانند برای مدل کردن سیاهه‌های مختلف مورد استفاده قرار بگیرند، از طرفی کار کردن با آن‌ها را سخت کرده و هزینه‌ی محاسباتی را بالا برده.

Scanned with CamScanner

مثلا CNN برای image ← پوزیشن پیکسل ها ثابت و مشخصه و می‌توانیم یک فیلتر را روی تصویر حرکت بدهید

در مورد text هم همینطور، ابتدا واژه‌های آن و ترتیب کلمات مشخص است ولی در گراف‌ها چنین ترتیب fix و مشخصی وجود ندارد. پس استفاده از شبکه‌های عصبی معمولی برای آن‌ها سر راست و راحت نیست. هم چنین نودهای ما اینطور نیست که یک کشش کوچک نیاز باشند بلکه ممکن است خیلی rich باشند و هم feature های زیادی داشته باشند

Setup

ورودی: یک گراف G ← مجموعه‌ای از نودها و یال‌ها دارد V — برای سادگی فرض می‌کنیم گراف وزن‌دار نیست

A: Adjacency Matrix

یک مشکل که روش‌های shallow دارند: نودها ممکن است از نظر اطلاعات جانبی و feature ها خیلی rich باشند که خیلی مفید است از این موارد استفاده کنیم ولی در این روش‌ها استفاده از این اطلاعات راحت نیست و معمولا انجام نمی‌گیرد.

هدف embedding learning

به تابع encoder داریم که یک نود را می‌گیرد و خروجی اش یک بردار برای نمایش آن نود است:

$Enc(u) \rightarrow z_u$

سعی داریم که Encode کردن ما Similarity preserving باشد

برای شباهت دو تا بردار معیاری که مناسب است ← ضرب داخلی

ولی برای شباهت دو تا نود در گراف معیار استانداردی نداریم و در الگوریتم‌های مختلف از روش‌های متفاوت استفاده می‌شود.

گام‌های اصلی یک الگوریتم shallow برای یادگیری بازنمایی:

① Define an encoder ← یک تابع کدگذار تعریف می‌کنیم که ورودی اش یک نود و خروجی اش یک بردار (objective function توسط یک embedding است. معمولا این تابع یک سری پارامتر دارد که باید یادگرفته شود

Scanned with CamScanner

۱۴۰۰

① معیاری برای میزان شباهت دو نود در فضای گراف تعریف می‌کنند. مثلا از معیارهای Common neighborhood ، hitting time ، Adamic Adar ، personalized pagerank و ... می‌توان نام برد.

② یک objective function تعریف می‌کنند که با optimize کردن آن می‌توانند پارامترهای Encoder را یاد بگیرند.

یک objective function خیلی پرکاربرد که از اصل Similarity preserving سرچشمه می‌گیرد:

$$\text{Similarity}(u,v) \approx z_v^T z_u$$

ضرب داخلی بردارهای تعبیه ← شباهت دو نود در فضای گراف

روش‌های Shallow

$$ENC(v) = Z_v \qquad Z \in \mathbb{R}^{d \times |V|} \qquad v \in I^{|V|}$$

تابع Encoder خیلی ساده است. Embedding برای نود‌ها در یک لایه یاد گرفته می‌شود.

برای هر نود یک بار یک embedding یاد می‌گیریم و تمام می‌شود. اینطور نیست که لایه‌های مختلف داشته باشیم و در هر لایه هر نود یک embedding ای داشته باشد.

ماتریس Z، $d \times n$ است (n تعداد نود‌ها). در این ماتریس به ازای هر نود یک ستون داریم. یک بردار d بعدی است.

کل ماتریس Z پارامتر است و باید یاد گرفته شود. پس تعداد پارامترهای این روش $d \times n$ است.

در حقیقت تابع Encoder یک lookup ساده انجام می‌دهد. v را می‌گیرد و ستون v ام این ماتریس را برمی‌گرداند.

بردار v یک بردار $n \times 1$ است که برای یک نود v فقط خانه‌ی v ام آن ۱ و بقیه خانه‌هایش صفر است که در Z ضرب شده

Scanned with CamScanner

اسفند | ۱۷ | 8 Mar 2022
سه شنبه | ۵ شعبان ۱۴۴۳

در نتیجه برای Z را در v ضرب می‌کنیم ستون v ام ماتریس Z را انتخاب می‌کند. بردار v، one hot indicator vector است.

مسئله‌ی اصلی این است که ماتریس Z را یاد بگیریم. روش‌های زیادی این کار را انجام می‌دهند:

node2vec, DeepWalk, LINE, HOPE

چیزی که این روش‌ها را از هم متمایز می‌کند similarity function است و انتخاب اصلی ما در این روش‌هاست.

مثلا معیارهای: متصل بودن دو نود، تعداد همسایه‌های مشترک، structural role شبیه به هم بودن، ... personalized pagerank، ؟ هستند.

روش‌های مبتنی بر Random walk ← پرطرفدارترین خانواده‌ی روش‌های shallow

میزان شباهت دو نود را با اجرای تعدادی random walk محاسبه می‌کنند.

در این حالت میزان شباهت دو نود u و v متناسب است با احتمال دیدن نود v در Random walk هایی که از نود u شروع می‌کنیم.

۱) ابتدا تخمین می‌زنیم با شروع از نود u اگر Random walk را اجرا کنیم با چه احتمالی v را می‌بینیم.

random walk های شما می‌توانند از یک استراتژی خاص پیروی کنند.

۲) embedding ها را optimize کنید یا یاد بگیرید، به طوری که similarity preserving رعایت شود.

در قدم اول این احتمال را تخمین می‌زنیم: $P_R(v|u)$ ← به عنوان میزان شباهت در نظر می‌گیریم

در گام دوم z_u و z_v را به گونه‌ای تخمین می‌زنیم که ضرب داخلی‌شان برابر $P_R(v|u)$ شود.

$z_u \cdot z_v \propto P_R(v|u)$

z_i — $\Delta\theta$ — z_j $\propto P_R(v|u)$

تنها اندازه بردارها را باید شباهت هان dot product

$z_u \cdot z_v = |z_u||z_v| \cos\theta$

شباهت کسینوسی است زیرا:

ولادت امام سجاد (ع) (۳۸ هـ ق)

Scanned with CamScanner

۱۴۰۰

چرا random walk ها معیار خوبی برای تعیین میزان شباهت دو تا نود در فضای گرافی هستند؟

① هم شباهت محلی و هم شباهت عمومی (global) دو نود را در نظر می‌گیرد. شباهت محلی (local) یعنی فقط همسایگی دو نود را در نظر بگیریم. (یعنی فقط همسایه بودن و مستقل بودن را در نظر بگیریم) ← خروجی دارد

علاوه دو تا نود دوست صمیمی خیلی شبیه به هم باشند. RW به درجه‌ای از شباهت عمومی را به local اضافه می‌کند.

② هزینه‌ی محاسباتی آن کمینه است. در عمل برای هر کدام از نودها مستقل از بقیه نودها می‌توانید random walk ها را اجرا کنید و به صورت موازی هم قابل انجام است.

دلیل دیگر اینکه اگر بخواهید k تا نودی که بیشترین شبیه هستند را بیابید نیاز ندارید که تمام نودهای دیگر را explore کنید. یعنی نیازیست شباهت کل نودهای دیگر با نود اولیه را چک کنید تا بفهمید کدام‌ها بیشترین شباهت را دارند. خیلی از روش‌ها این خروجی را دارند مثل Betweenness.

ایده کلی: نودهایی که همسایه هم هستند در فضای گرافی، در فضای embedding هم بقیه‌های آن‌ها به هم نزدیک باشند. (ترجمه‌ای از اصل Similarity preserving است)

(همسایه)

سوال: یک نود مشخص u داده شده، نودهای همسایه‌ی نزدیکش را چگونه تعریف کنیم؟ $N_R(u)$ را چگونه تشکیل دهیم؟

یک استراتژی R را برای Random walk هایمان پیاده می‌کنیم در روش‌های مبتنی بر RW.

با استفاده از نودهایی که در این RW ها دیده می‌شوند $N_R(u)$ را می‌سازیم.

بنابراین از نود u تعدادی RW با استراتژی R اجرا می‌کنیم. در اثر انجام این عمل، مجموعه‌ای از نودها دیده می‌شوند. به این مجموعه $N(u)$ می‌گوییم که embedding نودهای آن باید شبیه بقیه‌ی نود u باشد تا این فرآیند Similarity preserving باشد.

Scanned with CamScanner

بعد از انجام این random walk ها چیزی که مشاهده می‌شود نودهای مجموعه $\{N_R(u)\}$ هستند.

پس حالا که مشاهده شده‌اند احتمالشان بالا برود پس در تابع هدف احتمال یا Likelihood را برای آن‌ها max می‌کنیم:

$$\max_{Z} \sum_{u \in V} \log P(N_R(u) \mid Z_u)$$

از فرض استقلال هم استفاده می‌کنیم و فرض می‌کنیم نودهای مختلف $P(N(u))$ شان مستقل از یکدیگر است در نتیجه می‌توانیم احتمال همسایگی‌شان را در هم ضرب کنیم و احتمال کل را که احتمال خاصی است که مشاهده کردیم، به دست آوریم.

از لگاریتم استفاده می‌کنیم که راحت‌تر باشد و تبدیل به جمع شود.

مشروط به Z_u یعنی با فرض اینکه تعبیه نود u، Z_u باشد.

گام‌های این روش:

۱) با شروع از هر نود u تا k RW با استراتژی R می‌سازید. طول هر RW مثلا R باشد. طول‌شان باید کوتاه باشد.

۲) نودهایی که برای هر نود u در این walk ها جمع‌آوری می‌کنید در یک multiset اجازه می‌دهیم که تکرارها وجود داشته باشد در این صورت نودهایی که زیاد مشاهده می‌شوند اهمیت‌شان حفظ می‌شود.

۳) از اصل maximum Log LikeLihood استفاده می‌کنیم. چیزی که مشاهده کرده‌ایم یک multiset از همسایگی هر نود است و آن را برای نودهای مختلف maximize می‌کنیم.

Scanned with CamScanner

از یک سری فرض برای حل مسئله استفاده می‌کنیم:

فرض اول: یک همسایگی حاصل مثل $P(N_R)$ را در نظر بگیرید. $P(N_R)$ یعنی احتمال اینکه همسایگی N_R اول N_R مشاهده شود و عضو دوم مشاهده شود و ... اصل استقلال را داخل اعضای N_R هم اعمال می‌کنیم:

$$P(N_R) = P(\text{مشاهده عضو اول}) \times P(\text{مشاهده عضو دوم}) \times \dots$$

پس احتمال همسایگی را به حاصل ضرب تعدادی احتمال تبدیل می‌کنیم که هر احتمال روی هر یک نود است و دیگر روی یک همسایگی نیست. بعد اگر لگاریتم را از این حاصل ضرب‌ها عبور دهیم تبدیل به Σ می‌شود.

$$\log P(N_R^{(u)} | z_u) = \sum_{v \in N_S(u)} \log P(v | z_u)$$

با استفاده از اصل Similarity preserving: احتمال اینکه نود v در همسایگی نودی باشد برابر است با میزان شباهت نقیبه نود v به نقیبه نود u. پس به این رابطه می‌رسیم:

$$\log P(N_R(u) | z_u) = \sum_{v \in N_S(u)} \log P(z_v | z_u)$$

حالا $P(z_v | z_u)$ را چگونه تعریف می‌کنیم؟

دنبال embedding هستیم که میزان شباهت آن‌هایی شود احتمال آن‌ها. یعنی احتمال اینکه نقیبه‌ی v در همسایگی نقیبه‌ی u باید برابر میزان شباهت آن‌هاست. که با dot product مشخص می‌کنیم:

$$Pr(z_v | z_u) \propto z_v \cdot z_u$$

حالا می‌توانیم این شباهت را Normalize کنیم و تبدیل به احتمال کنیم ولی از تابع softmax هم استفاده می‌کنیم چون در عمل خیلی خوب جواب می‌دهد.

$$Pr(z_v | z_u) = \frac{\exp(z_v \cdot z_u)}{\sum_{n \in V} \exp(z_n \cdot z_u)}$$

Scanned with CamScanner

دیگه‌ایت objective function ای به دست می‌آید که پارامترهایش آن ظاهر شده‌اند:

منفی

$$\mathcal{L} = \sum_{u \in V} \sum_{v \in N_R(u)} -\log\left(\frac{\exp(z_u^T z_v)}{\sum_{n \in V} \exp(z_u^T z_n)}\right)$$

↙ مجموعه نودهای E روی گراف ↙ همسایگی نود u

مشکل این تابع هدف:

اگر از GD بخواهیم استفاده کنیم مشتق این تابع هدف را در هر iteration باید به دست آوریم. مشتق از Z‌ها عبوری کند. time Complexity که باید بدهیم $O(|V|^2)$ است (مرتبه دوم برحسب مقدار نودها) ابتدا یک Z روی نودها داریم در خروجی کسر هم همین طور.

این time Complexity بالا است و معمولا مرتبه دوم برای ما قابل تحمل نیست.

نکته: علت وجود علامت منفی در تابع هدف این است که می‌خواهیم عبارت را max کنیم درحالی که GD برای min کردن استفاده می‌شود به همین دلیل منفی گذاشته‌ایم تا فرمول آن را min کنیم و خود عبارت max شود.

از یک تکنیک به نام Negative Sampling استفاده می‌شود برای مشکل زمان بالا:

$$\log\left(\frac{\exp(z_u^T z_v)}{\sum_{n \in V} \exp(z_u^T z_n)}\right) \approx \log(\sigma(z_u^T z_v)) - \sum_{i=1}^{k} \log(\sigma(z_u^T z_{n_i})), \quad n_i \sim P_V$$

به جای حرکت روی کل Z‌ها داشتیم تقریبی از آن را قرار می‌دهیم

Z از 1 تا K می‌رود به جای اینکه روی تک تک نودها باشد. K پارامتر ثابتی است که خودمان Set می‌کنیم.

$n_i \leftarrow$ در هر iteration داخل Z یک نود از گراف را به صورت uniformly at random انتخاب می‌کنیم

n_i آن نودی است که انتخاب شده.

اگر k و d ثابت فرض شوند این تقریب $O(1)$ است درحالی که اصل آن $O(|V|)$ بود.

Scanned with CamScanner

هرچقدر K بزرگتر باشد تقریب بهتری می‌دهد ولی پیچیدگی محاسباتی بالاتر می‌رود.

معمولا K را بین ۵ تا ۲۰ در نظر می‌گیرند.

σ یک تابع غیرخطی (nonlinearity) است مثل Sigmoid ، Tanh یا ReLU که ورودی می‌گیرند و خروجی تابعی غیرخطی از ورودی است

جلسه بیست و یکم - ۱۴۰۰/۹/۱۴

در مورد روش‌های shallow گفتیم. در این روش‌ها تعداد مشخصی Random walk با طول مشخص از هر نود بایک استراتژی مثل R، می‌زنیم. برروما به صورت uniform به همسایه‌های هر نود می‌رویم، بستگی به استراتژی دارد.

پس برای هر نود یک ماتریس است $N_R(u)$ داریم که همسایگی نود u را نشان می‌دهد. اگر یک نود چندبار visit شده باشد در ماتریس به همان تعداد آورده می‌شود.

در نهایت بردارهای Z را به گونه‌ای می‌یابیم که تابع هدف زیر کمینه شود:

$$L=\sum_{u\in V}\sum_{v\in N_R(u)}-\log(P(v|Z_u))$$

این Random walk ها را چگونه و با چه استراتژی انجام بدهیم؟

الگوریتم Deep walk ← استراتژی random walk در این الگوریتم uniform است

Node 2 Vec ← پیشنهاد داد که random walk را parameterize کنیم، دو تا پارامتر تعریف کرد که با استفاده از آن‌ها random walk ها را به صورت non-uniform اجرا می‌کنیم. در عین حال ایده‌ی Negative Sampling را هم برای حل مسئله پیچیدگی زمانی بالا ارائه داد.

خود user می‌تواند تنظیم کند که random walks ها بیشتر در عمق باشند یا سطح.

الگوریتم Node 2 Vec ← 2nd order Random walks

این الگوریتم بایک سری پارامتر Random walk ها را انعطاف پذیری می‌کند به طوری که از همسایگی یک نود یک نگاه خیلی محلی، نگاه خیلی عمیق یا interpolate ای از این دو داشته باشیم

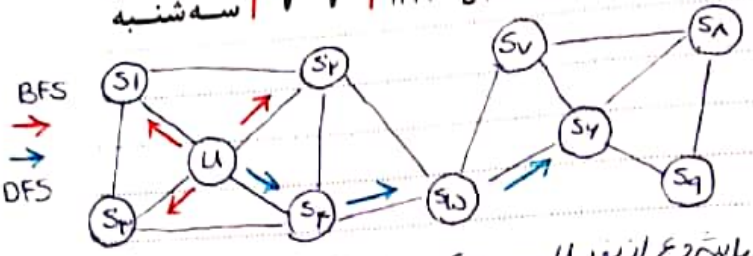
دستمان خسته نشود، اگر خواهیم به نود u خیلی محلی نگاه کنیم مثلا فقط باید نودهایی که مستقیما به آن وصلند را در نظر بگیریم

انگار که بیایم BFS بزنیم از نود u.

Scanned with CamScanner

اگر بخواهیم خیلی global عمل کنیم باید دامنه‌ی نود

را بزرگتر در نظر بگیریم و نودهای دورتر را وزنیت کنیم.



در حالت extreme این است که DFS بزنیم با شروع از نود u ← s_9, s_4, s_5, s_6

سوال این است که کدام بهتر است؟ ما نمی‌دانیم و بستگی به شبکه دارد. برای بعضی نودها شبکه روش‌های BFS بهتر باشد و برای بعضی DFS.

اگر بخواهیم یک دید microscopic داشته باشیم از همسایگی نود u و خیلی local عمل کنیم:

$N_{BFS}(u) = \{s_1, s_2, s_3\}$

بلکه نودهای خیلی دورتر از u را وزنیت کنیم

اگر بخواهیم خیلی global عمل کنیم یک دید macroscopic از همسایگی u خواهیم داشت: $N_{DFS}(u) = \{s_4, s_5, s_6\}$

چگونه بین این دو تا حالت interpolate کنیم؟ پیشنهاد دادند که رندوم walk ها را parameterize کنیم

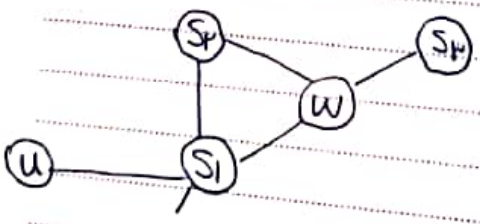
و از دو تا پارامتر استفاده کنیم:

(Return parameter p)

پارامتر p ← احتمال اینکه به نود قبلی (پدر) برگردیم

(In-out parameter q)

پارامتر q ← با چه وزنی از نود پدر دورتر شویم



فرض کنید الان یال (s_1, w) را پیموده‌ایم

Node2Vec ← با وزن $\frac{1}{p}$ به نود قبلی (پدر) می‌رویم، با وزن $\frac{1}{q}$ به

Scanned with CamScanner